

AC710N 用户手册

珠海市杰理科技股份有限公司

Zhuhai Jieli Technologyco.,LTD

版权所有，未经许可，禁止外传

2025年1月

目录

目录	2
1 序言	11
2 总体介绍	12
2.1 概述	12
2.2 系统框图	14
3 IO_CONTROL	15
3.1 概述	15
3.2 数字模块控制寄存器	15
3.2.1 PORTX_IN: portx input control register:	15
3.2.2 PORTX_OUT: portx output control register:	15
3.2.3 PORTX_DIR: portx direction control register:	15
3.2.4 PORTX_DIE: portx input enable control register:	15
3.2.5 PORTX_DIEH: portx input enable p33 control register:	16
3.2.6 PORTX_PU: portx pull up control register:	16
3.2.7 PORTX_PD: portx pull down0 control register:	17
3.2.8 PORTX_HD: portx high driver control register:	17
3.2.9 PORTX_SPL: portx pull-down resistor enable in output control register:	18
3.2.10 PORTX_OUT_BSR: portx bit set resistor in output control register:	18
3.2.11 PORTX_DIR_BSR: portx bit set resistor in dir control register:	18
3.2.12 PORTX_DIE_BSR: portx bit set resistor in die control register:	18
3.2.13 PORTX_DIEH_BSR: portx bit set resistor in dieh control register:	19
3.2.14 PORTX_PUL_BSR: portx bit set resistor in pu control register of Low group:	19
3.2.15 PORTX_PU1_BSR: portx bit set resistor in pu control register of High group:	19
3.2.16 PORTX_PDL_BSR: portx bit set resistor in pd control register of Low group:	19
3.2.17 PORTX_PDH_BSR: portx bit set resistor in pd1 control register of High group:	20
3.2.18 PORTX_HDL_BSR: portx bit set resistor in hd control register of Low group:	20
3.2.19 PORTX_HDH_BSR: portx bit set resistor in hd control register of High group:	20
3.2.20 PORTX_SPL_BSR: portx bit set resistor in spl control register:	20
3.3 USB_IO CONTROL	21
3.4 IO模拟状态设置	22
4 IO_WAKEUP	24

4.1 概述	24
4.2 Wakeup唤醒源	24
4.3 寄存器	25
4.3.1 WKUP_CON0: wakeup enable control register	25
4.3.2 WKUP_CON1: wakeup edge control register	25
4.3.3 WKUP_CON2: wakeup pending clear control	25
4.3.4 WKUP_CON3: wakeup pending register	26
5 IOMC	27
5.1 概述	27
5.2 JL_IOMC控制寄存器	27
5.2.1 SPI_IOMC0: spi iomc control register 0	27
5.2.2 SDC_IOMC0: sdc iomc control register 0	27
5.2.3 sys iomc control register 0	28
5.2.4 sys iomc control register 1	29
5.2.5 ASS_IOMC0: audio iomc control register 0	29
5.2.6 WL_IOMC0: wireless iomc control register 0	29
5.2.7 ICH_IOMC0: input channel control register0	30
5.2.8 ICH_IOMC1: input channel control register1	31
5.2.9 ICH_IOMC2: input channel control register2	31
5.2.10 ICH_IOMC3: input channel control register3	31
5.2.11 ICH_IOMC4: input channel control register4	32
5.2.12 ICH_IOMC5: input channel control register5	32
5.2.13 OCH_IOMC0: output channel control register0	32
5.2.14 OCH_IOMC1: output channel control register1	33
5.2.15 OCH_IOMC2/3/4/5: output channel control register2/3/4/5	34
5.2.16 SDPG_CON: SDPG control register	34
5.3 JL_SFC_IOMC控制寄存器	35
5.3.1 FSPG_CON: FSPG control register	35
5.3.2 IOMC0: IOMC0 register	36
5.3.3 IOMC1: IOMC1 register	36
6 PORT Control Crossbar	38
6.1 Crossbar功能结构图	38
6.2 寄存器功能	38
6.2.1 JL_OMAP-> POUT	38
6.2.2 JL_IMAP-> FUN_IN	39

7 CLOCK_SYSTEM	40
7.1 原生时钟源	40
7.2 内部PLL时钟源	40
7.3 标准时钟	40
7.4 系统时钟	41
7.4.1 HSB时钟源	41
7.4.2 LSB时钟源	42
7.5 外设时钟	43
7.5.1 SFC 时钟源	43
7.5.2 音频时钟源	44
7.5.3 WL时钟源	45
7.5.4 外设时钟源	46
7.6 数字模块控制寄存器	48
7.6.1 JL_LSBCK->SYS_CON0: lsbck system config register 0	48
7.6.2 LSB_SEL: lsb clock sel register	48
7.6.3 JL_LSBCK->STD_CON0: standard clock register 0	48
7.6.4 JL_LSBCK->STD_CON1: standard clock register 1	49
7.6.5 JL_LSBCK->STD_CON2: standard clock register 2	49
7.6.6 JL_LSBCK->PRP_CON0: peripheral clock config register 0	50
7.6.7 JL_LSBCK->PRP_CON1: peripheral clock config register 1	50
7.6.8 JL_LSBCK->PRP_CON2: peripheral clock config register 2	51
7.6.9 JL_HSBCK->SYS_CON0: hsbck system config register 0	51
7.6.10 JL_HSBCLK->HSB_DIV: hsb clock div register	52
7.6.11 JL_HSBCLK->HSB_SEL: hsb clock sel register	52
8 ADC	53
8.1 概述	53
8.2 寄存器说明	53
8.2.1 ADC_CON: ADC control register	53
8.2.2 ADC_RES: ADC result register	54
9 CRC	55
9.1 概述	55
9.2 寄存器说明	55
9.2.1 REG: CRC16 校验码	55
9.2.2 FIFO: 运算数据输入	55
10 GPCNT	57

10.1 概述	57
10.2 寄存器说明	57
10.2.1 JL_GPCNT->CON: GPCNT configuration register	57
10.2.2 JL_GPCNT->NUM: The number of GPCNT clock cycle register	58
11 GPCRC	59
11.1 概述	59
11.2 功能描述	59
11.2.1 数据处理	59
11.2.2 多项式配置	59
11.3 数字模块控制寄存器	60
11.3.1 GPCRC_CON: gpcrc control register	60
11.3.2 GPCRC_POL: gpcrc poly register	61
11.3.3 GPCRC_INIT: gpcrc initial register	61
11.3.4 GPCRC_REG: gpcrc data register	61
11.4 DEMO CODE	61
12 IIC	63
12.1 概述	63
12.2 特殊注意点	63
12.3 数字模块控制寄存器	63
12.3.1 IIC_CON0: iic control register 0	63
12.3.2 IIC_TASK: iic task register	65
12.3.3 IIC_PND: iic pending register	65
12.3.4 IIC_TXBUF: iic tx buff register	67
12.3.5 IIC_RXBUF: iic tx buff register	67
12.3.6 IIC_ADDR: iic device address register	67
12.3.7 IIC_BAUD: iic baud clk register	67
12.3.8 IIC_TSU: iic setup time register	67
12.3.9 IIC_THD: iic hold time register	68
12.3.10 IIC_DBG: iic debug register	68
12.4 基本事务操作	68
12.5 基本工作场景使用流程	69
13 MCPWM	75
13.1 概述	75
13.2 定时器 MCTIMER	75
13.3 TIMER模块寄存器	75

13.3.1 TMRx_CON: Timer contrl register	75
13.3.2 TMRx_CNT: counter initial value register	76
13.3.3 TMR_PR: counter target value register	76
13.4 模块引脚	76
13.5 模块特性	77
13.6 PWM模块控制寄存器	77
13.6.1 CHx_CON0: pwm control register0	77
13.6.2 CHX_CON1: pwm control register1	77
13.6.3 CHX_CMPH: pwm high port compare register	78
13.6.4 CHX_CMPL: pwm low port compare register	78
13.6.5 FPIN_CON: input filter control register	78
13.6.6 MCPWM_CON: mcpwm control register	79
14 PULSE COUNTER	80
14.1 概述	80
14.2 寄存器说明	80
14.2.1 PL_CNT_CON	80
14.2.2 PL_CNT_VAL	81
14.3 示例代码	81
15 PWM LED	82
15.1 概述	82
15.2 特性	82
15.3 特殊注意点	84
15.4 寄存器并接	85
15.5 时钟控制	85
15.6 二级亮灭控制	86
15.7 呼吸灯控制	87
15.8 控制寄存器说明	88
15.8.1 CON0: PWM control register 0	88
15.8.2 CON1: PWM control register 1	92
15.8.3 BRI_PRD: PWM0 brightness period register	93
15.8.4 BRI_DUTY0: PWM0 low state brightness duty register	93
15.8.5 BRI_DUTY1: PWM0 high state brightness duty register	93
15.8.6 PRD_DIV: PWM1 period divider register	93
15.8.7 DUTY0: PWM duty0 register	94
15.8.8 DUTY1: PWM duty1 register	94

15.8.9 DUTY2: PWM duty2 register	94
15.8.10 DUTY3: PWM duty3 register	94
15.8.11 SCON0: pwm_sync_controller0	95
15.8.12 ANA_CON0: pwm_ana_controller0	96
15.8.13 SCON1: pwm_sync_controller1	96
15.8.14 OUT1_DIV	96
15.8.15 OUT0_DIV	97
15.8.16 CNT_CON : counter_cnt read control	97
15.8.17 CNT_DATA : counter_cnt read data	98
15.8.18 CNT_DEC : counter_cnt decrease	99
15.8.19 CNT_SYNC : counter_cnt sync	99
16 QDEC	102
16.1 概述	102
16.2 特殊注意点	102
16.3 数字模块控制寄存器	102
16.3.1 QDECx_CON: QDEC control register	102
16.3.2 QDECx_SMP: QDEC sample register	103
16.3.3 QDECx_DAT: QDEC control register	103
16.3.4 QDECx_DBE: QDEC double transition error	103
16.4 DEMO CODE	103
17 RAND64	105
17.1 概述	105
17.2 控制寄存器	105
18 SDC	106
18.1 概述	106
18.2 数字模块控制寄存器	106
18.2.1 JL_SD0->CON0: sd0 control register 0	106
18.2.2 JL_SD0->CON1: sd0 control register 1	107
18.2.3 JL_SD0->CON2: sd0 control register 2	108
18.2.4 JL_SD0-> CTU_CON: sd0 continue control register	108
18.2.5 JL_SD0-> CTU_CNT: sd0 continual transfer counter register	109
18.2.6 JL_SD0-> CPTR: sd0 command pointer register	109
18.2.7 JL_SD0-> DPTR: sd0 data pointer register	110
18.3 配置流程示例	110
18.3.1 发送命令、接收响应流程	110

18.3.2 发送数据流程（单块写CMD24）	110
18.3.3 接收数据流程（单块读CMD17）	110
18.3.4 发送数据流程（多块写CMD25）	111
18.3.5 接收数据流程（多块写CMD18）	111
18.4 DEMO CODE	111
19 SPI	113
19.1 概述	113
19.2 特殊注意点	113
19.3 数字模块控制寄存器	114
19.3.1 SPIx_CON: SPIx control register 0	114
19.3.2 SPIx_BAUD: SPIx baudrate setting register	115
19.3.3 SPIx_BUF: SPIx buffer register	115
19.3.4 SPIx_ADR: SPIx DMA address register	115
19.3.5 SPIx_CNT: SPIx DMA counter register	116
19.3.6 SPIx_CON1: SPIx control1 register	116
19.4 传输波形	116
20 32位 GPTIMER	118
20.1 概述	118
20.2 控制寄存器	118
20.3 寄存器说明	118
20.3.1 Tx_CON: timer x control register	118
20.3.2 Tx_CNT: timer x counter register	119
20.3.3 Tx_PR: timer x period register	119
20.3.4 Tx_PWM: timer x PWM register	120
20.3.5 Tx_IRFLT: timer x PWM register	120
21 UART	122
21.1 概述	122
21.2 特殊注意点	122
21.3 数字模块控制寄存器	122
21.3.1 TX_CON0: uart tx control register 0	122
21.3.2 RX_CON0: uart rx control register 0	122
21.3.3 TX_CON1: uart tx control register 1	123
21.3.4 RX_CON1: uart rx control register 1	124
21.3.5 UTx_CON2: uart x control register 2	125
21.3.6 UTx_BAUD: uart x baudrate register	125

21.3.7 UTx_BUF: uart x data buffer register	126
21.3.8 UTx_TXADR: uart x TX DMA buffer register	126
21.3.9 UTx_TXCNT: uart x TX DMA counter register	126
21.3.10 UTx_RXCNT: uart x RX DMA counter register	126
21.3.11 UTx_RXSADR: uart x RX DMA start address register	127
21.3.12 UTx_RXEADR: uart x RX DMA end address register	127
21.3.13 UTx_HRXCNT: uart x have RX DMA counter register	127
21.3.14 UTx_OTCNT: uart x Over Timer counter register	127
21.3.15 UTx_ERR_CNT: uart x error byte counter register	127
22 WWDG(Window watch dog)	129
22.1 概述	129
22.2 主要特性	129
22.3 框架图	129
22.4 寄存器说明	130
22.4.1 KEY	130
22.4.2 CR	130
22.4.3 CFG	130
22.4.4 SR	131
22.5 窗口时间计算	131
22.6 示例代码	132
23 AUDIO LINK	134
23.1 概述	134
23.2 特殊注意点	134
23.3 ALNK采样率配置说明	134
23.3.1 ALNK模式说明	134
23.4 数字模块控制寄存器	136
23.4.1 ALNK_CON0: control register 0	136
23.4.2 ALNK_CON1: control register 1	137
23.4.3 ALNK_CON2: control register 2	137
23.4.4 ALNK_CON3: control register 3	138
23.4.5 ALNK_CON4: control register 4	139
23.4.6 ALNK_ADRX: alnk dma start address register	139
23.4.7 ALNK_LEN: alnk dma sample length register	139
23.4.8 ALNK_PNS: alnk ring buff pnding set number	140
23.4.9 ALNK_BLOCK: alnk_block number set	140

23.4.10 ALNK_HWPTRX: alnk hardware rd/wr pointer	140
23.4.11 ALNK_SWPTRX: alnk software rd/wr pointer	140
23.4.12 ALNK_SHNX: alnk software handle data number	141
23.5 SW_MUTE参考代码	141
23.6 数据通路	141
23.6.1 乒乓缓存	141
23.6.2 循环缓存	142

杰理科技授权潮汐专用

1 序言

本芯片是内置高性能32位双发射DSP的系统级SOC，支持高性能低功耗的蓝牙音频应用。DSP集成数学运算加速的处理器，整合多种硬件加速模块，有效的降低了无线音频场景的运行功耗；内置蓝牙调制解调器、基带及模拟RF模块，支持蓝牙V5.4版本及超低功耗蓝牙连接待机；集成高性能音频ADC/DAC，支持免电容mic，支持通话回声消除；PMU支持低功耗关机触摸唤醒，内置DCDC模块，给系统长时间的蓝牙音频续航。

本芯片集成了一套的蓝牙解决方案，为用户开发提供了方便的环境，出色的电池使用寿命使其适合功耗敏感的应用，为用户提供立体声蓝牙耳机全功能单芯片方案。

杰理科技授权潮汐专用

2 总体介绍

2.1 概述

SYSTEM

- 32bit Single-core DSP 192MHz
- On-chip SRAM 280kbyte(share Cache 40k)
- With IEEE754 Single precision FPU
- Support FFT/MATRIX/MATH
- I-cache and D-cache
- Support EMU
- Support MMU
- Support MPU
- Built-In Flash
- 24MHz crystal oscillator
- Internal RC oscillator,PLL

DSP Audio Processing

- SBC/AAC/LDAC/LHDC/LC3/CVSD/mSBC codec
- mSBC voice codec supported for BT phone
- PLC for voice processing
- Single/Dual MIC ENC
- Multi-band DRC
- Multi-band EQ
- Support spatial sound
- Support assistive listening
- Support Hi-Res Audio

ANC

- Wide band digital adaptive ANC
- Support hybrid/feedforward/feedback
- Support wind noise detection
- Support wide area tap
- Support Speak-to-Chat
- Support tip fit test & wear detection
- Input-to-output latency
 - 10us@SR=750kHz
 - 14us@SR=375kHz

Audio

- 2 x 24bit DAC
 - SNR 113dB
 - Noise 2.5uVrms
 - Support single mode
 - Support differential mode
 - Support VCMO mode
 - Support OWS dynamic power control
 - Sampling rate 8~384kHz
- 2 x 24bit ADC
 - SNR 100dB

Sampling rate 8~192kHz

- 1 x I2S AUDIO Master/Slave interface
Sampling rate 8~384kHz
Support TX and RX
Support multi-slot mode(TDM)
- 1 x PDM AUDIO Slave interface
Sampling rate 8~192kHz
4 x DMIC input

Bluetooth

- Dual-mode BT6.0 with LE Audio
- (DN Q332415)
- Support AoA
- Support LE audio BIS/CIS
- Maximum transmitting power 13dBm
- Receiver sensitivity
 - 94dBm @BR
 - 95dBm @EDR $\Pi/4$ DQPSK
 - 87dBm @EDR 8DPSK

Peripherals

- 1 x Full speed USB
- 1 x SD host controller
- 4 x Multi-function 32bit timer
- 2 x UART interface
- 1 x I2C Master/Slave interface
- 2 x SPI Master/Slave interface
- 1 x QDEC
- 1 x 10bit ADC(16 Channels)
- 4 x MCPWM
- 5 x LP_Touch with low power wake up
- 4 x IR RX
- 24 x GPIO Support function remapping

PMU

- Integrated battery charger up to 225mA
- 1 x Buck DC-DC converter
- Support temperature sensor
- Support VBAT input current OCP
- Support IOVDD domain LVD
- VPWR range 4.5V to 5.5V
- VBAT range 2.7V to 4.5V
- IOVDD range 2.7V to 3.6V
- TWS Power < 4.4mA
@AAC No-Load VBAT=3.7V

Packages

- QFN32(4mm*4mm)

Temperature

- Operating temperature
TC = -20°C to +85°C(standard range)
TC = -40°C to +105°C(extended range)
- Storage temperature -65°C to +150°C

Applications

- ANC TWS/OWS
- Bluetooth audio device

2.2 系统框图

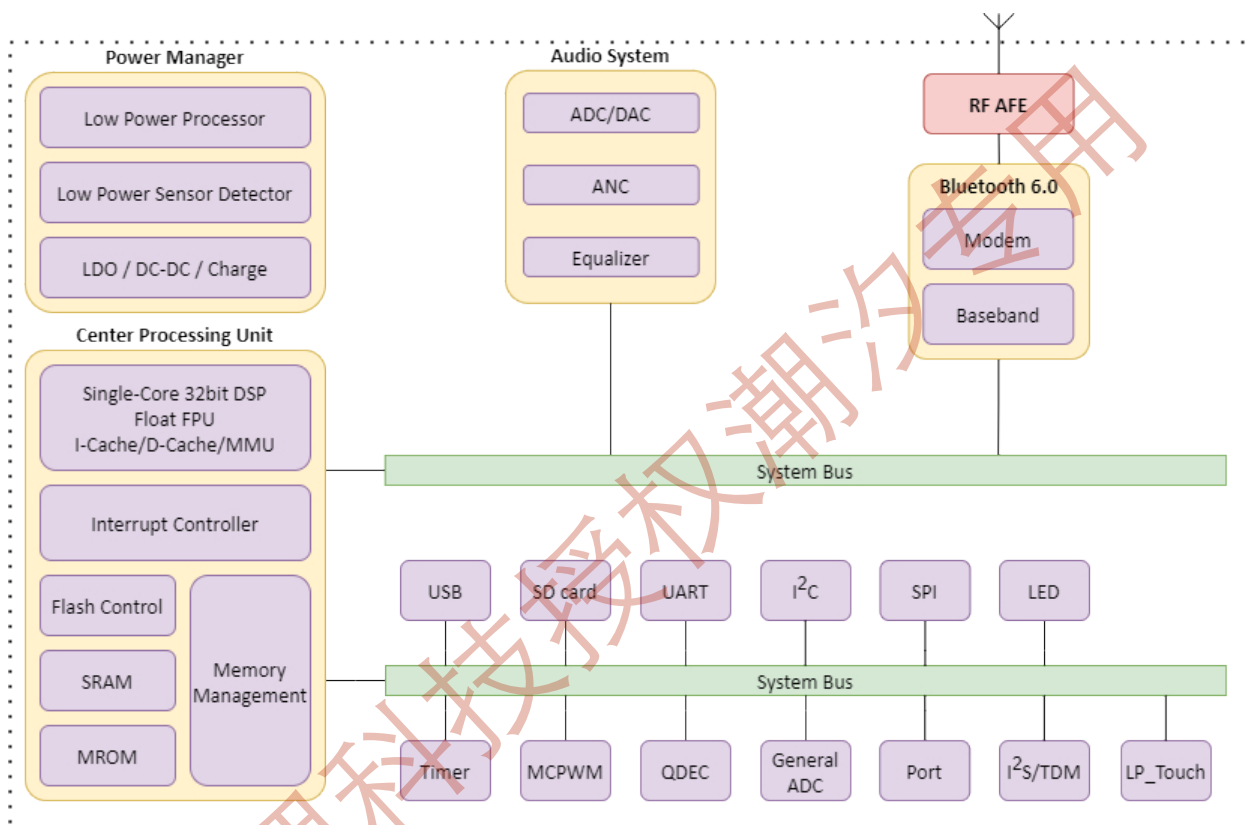


图1 AC710N总体框图

3 IO_CONTROL

3.1 概述

IO状态控制寄存器

3.2 数字模块控制寄存器

3.2.1 PORTX_IN: portx input control register:

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	0x0	-	预留
15-0	PORTX_IN	r	-	-	获取portx组的输入值，只有在对应的GPIO设置为输入使能（DIE） 有效时数值才正确 0: 输入为0 1: 输入为1

3.2.2 PORTX_OUT: portx output control register:

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	0x0	-	预留
15-0	PORTX_OUT	RW	0x0	-	控制portx组的输出值，只有在对应的GPIO设置为输出方向（DIR） 有效时输出数值才正确 0: 输出为0 1: 输出为1

3.2.3 PORTX_DIR: portx direction control register:

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	0x0	-	预留
15-0	PORTX_DIR	RW	0x1	-	控制portx组的输入输出方向 0: 设置为输出方向 1: 设置为输入方向

3.2.4 PORTX_DIE: portx input enable control register:

Bit	Name	RW	Default	RV	Description
-----	------	----	---------	----	-------------

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	0x0	-	预留
15-0	PORTX_DIE	RW	0x1	-	控制portx组的输入使能 0: 禁止输入使能 1: 允许输入使能

3.2.5 PORTX_DIEH: portx input enable p33 control register:

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	0x0	-	预留
15-0	PORTX_DIEH	RW	0x1	-	控制portx组的P33输入使能 0: 禁止输入使能 1: 允许输入使能

3.2.6 PORTX_PU: portx pull up control register:

Bit	Name	RW	Default	RV	Description
31-30	PORTX_PU15	RW	0x0	-	控制portx组的上拉电阻值, 控制PX15
29-28	PORTX_PU14	RW	0x0	-	控制portx组的上拉电阻值, 控制PX14
27-26	PORTX_PU13	RW	0x0	-	控制portx组的上拉电阻值, 控制PX13
25-24	PORTX_PU12	RW	0x0	-	控制portx组的上拉电阻值, 控制PX12
23-22	PORTX_PU11	RW	0x0	-	控制portx组的上拉电阻值, 控制PX11
21-20	PORTX_PU10	RW	0x0	-	控制portx组的上拉电阻值, 控制PX10
19-18	PORTX_PU9	RW	0x0	-	控制portx组的上拉电阻值, 控制PX9
17-16	PORTX_PU8	RW	0x0	-	控制portx组的上拉电阻值, 控制PX8
15-14	PORTX_PU7	RW	0x0	-	控制portx组的上拉电阻值, 控制PX7
13-12	PORTX_PU6	RW	0x0	-	控制portx组的上拉电阻值, 控制PX6
11-10	PORTX_PU5	RW	0x0	-	控制portx组的上拉电阻值, 控制PX5
9-8	PORTX_PU4	RW	0x0	-	控制portx组的上拉电阻值, 控制PX4
7-6	PORTX_PU3	RW	0x0	-	控制portx组的上拉电阻值, 控制PX3
5-4	PORTX_PU2	RW	0x0	-	控制portx组的上拉电阻值, 控制PX2
3-2	PORTX_PU1	RW	0x0	-	控制portx组的上拉电阻值, 控制PX1

Bit	Name	RW	Default	RV	Description
1-0	PORTX_PU0	RW	0x0	-	控制portx组的上拉电阻值，控制PX0，共四个档位 00: 无上拉电阻 01: 10KΩ 10: 100KΩ 11: 1MΩ

注意：（mclr上拉电阻不受该位控制，由p33控制）

3.2.7 PORTX_PD: portx pull down0 control register:

Bit	Name	RW	Default	RV	Description
31-30	PORTX_PD15	RW	0x0	-	控制portx组的下拉电阻值，控制PX15
...	...	...	...	...	
9-8	PORTX_PD4	RW	0x0	-	控制portx组的下拉电阻值，控制PX4
7-6	PORTX_PD3	RW	0x0	-	控制portx组的下拉电阻值，控制PX3
5-4	PORTX_PD2	RW	0x0	-	控制portx组的下拉电阻值，控制PX2
3-2	PORTX_PD1	RW	0x0	-	控制portx组的下拉电阻值，控制PX1
1-0	PORTX_PD0	RW	0x0	-	控制portx组的下拉电阻值，控制PX0，共四个档位 00: 无下拉电阻 01: 10KΩ 10: 100KΩ 11: 1MΩ

3.2.8 PORTX_HD: portx high driver control register:

Bit	Name	RW	Default	RV	Description
31-30	PORTX_HD15	RW	0x1	-	控制portx组的强驱动电流档位，控制PX15
...	...	...	...	...	
9-8	PORTX_HD4	RW	0x1	-	控制portx组的强驱动电流档位，控制PX4
7-6	PORTX_HD3	RW	0x1	-	控制portx组的强驱动电流档位，控制PX3
5-4	PORTX_HD2	RW	0x1	-	控制portx组的强驱动电流档位，控制PX2
3-2	PORTX_HD1	RW	0x1	-	控制portx组的强驱动电流档位，控制PX1

Bit	Name	RW	Default	RV	Description
1-0	PORTX_HD0	RW	0x1	-	控制portx组的强驱动电流档位，控制PX0，共四个档位

注意：PORTP 组 IO 的HD档位默认值为0，部分PIN使用HD做特殊功能

3.2.9 PORTX_SPL: portx pull-down resistor enable in output control register:

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	0x0	-	预留
15-0	SPL	RW	0x0	-	在IO为输出状态时保持上下拉电阻功能，置1使能

3.2.10 PORTX_OUT_BSR: portx bit set resistor in output control register:

Bit	Name	RW	Default	RV	Description
31-16	OUT_BSR	W	0x0	-	设置portx组的输出0，只有在对应的GPIO设置为输出方向（DIR）有效时输出数值才正确 0: 无效 1: 有效
15-0	OUT_BSR	W	0x0	-	设置portx组的输出1，只有在对应的GPIO设置为输出方向（DIR）有效时输出数值才正确 0: 无效 1: 有效

3.2.11 PORTX_DIR_BSR: portx bit set resistor in dir control register:

Bit	Name	RW	Default	RV	Description
31-16	DIR_CLR	W	0x0	-	设置portx组对应GPIO的方向为输出 0: 无效 1: 有效
15-0	DIR_SET	W	0x0	-	设置portx组对应GPIO的方向为输入 0: 无效 1: 有效

3.2.12 PORTX_DIE_BSR: portx bit set resistor in die control register:

Bit	Name	RW	Default	RV	Description
31-16	DIE_CLR	W	0x0	-	设置portx组对应GPIO的die为0 0: 无效 1: 有效

Bit	Name	RW	Default	RV	Description
15-0	DIE_SET	W	0x0	-	设置portx组对应GPIO的die为1 0: 无效 1: 有效

3.2.13 PORTX_DIEH_BSR: portx bit set resistor in dieh control register:

Bit	Name	RW	Default	RV	Description
31-16	DIEH_CLR	W	0x0	-	设置portx组对应GPIO的dieh为0 0: 无效 1: 有效
15-0	DIEH_SET	W	0x0	-	设置portx组对应GPIO的dieh为1 0: 无效 1: 有效

3.2.14 PORTX_PUL_BSR: portx bit set resistor in pu control register of Low group:

Bit	Name	RW	Default	RV	Description
31-16	PUL_CLR	W	0x0	-	设置portx组对应GPIO的PU[15:0]为0 0: 无效 1: 有效
15-0	PUL_SET	W	0x0	-	设置portx组对应GPIO的PU[15:0]为1 0: 无效 1: 有效

3.2.15 PORTX_PU1_BSR: portx bit set resistor in pu control register of High group:

Bit	Name	RW	Default	RV	Description
31-16	PUH_CLR	W	0x0	-	设置portx组对应PU[31:16]为0 0: 无效 1: 有效
15-0	PUH_SET	W	0x0	-	设置portx组对应PU[31:16]为1 0: 无效 1: 有效

3.2.16 PORTX_PDL_BSR: portx bit set resistor in pd control register of Low group:

Bit	Name	RW	Default	RV	Description
31-16	PDL_CLR	W	0x0	-	设置portx组对应PD[15:0]为0 0: 无效 1: 有效

Bit	Name	RW	Default	RV	Description
15-0	PDL_SET	W	0x0	-	设置portx组对应PD[15:0]为1 0: 无效 1: 有效

3.2.17 PORTX_PDH_BSR: portx bit set resistor in pd1 control register of High group:

Bit	Name	RW	Default	RV	Description
31-16	PDL_CLR	W	0x0	-	设置portx组对应PD[31:16]为0 0: 无效 1: 有效
15-0	PDL_SET	W	0x0	-	设置portx组对应PD[31:16]为1 0: 无效 1: 有效

3.2.18 PORTX_HDL_BSR: portx bit set resistor in hd control register of Low group:

Bit	Name	RW	Default	RV	Description
31-16	HDL_CLR	W	0x0	-	设置portx组对应HD[15:0]为0 0: 无效 1: 有效
15-0	HDL_SET	W	0x0	-	设置portx组对应HD[15:0]为1 0: 无效 1: 有效

3.2.19 PORTX_HDH_BSR: portx bit set resistor in hd control register of High group:

Bit	Name	RW	Default	RV	Description
31-16	HDH_CLR	W	0x0	-	设置portx组对应HD[31:16]为0 0: 无效 1: 有效
15-0	HDH_SET	W	0x0	-	设置portx组对应HD[31:16]为1 0: 无效 1: 有效

3.2.20 PORTX_SPL_BSR: portx bit set resistor in spl control register:

Bit	Name	RW	Default	RV	Description
31-16	SPL_CLR	W	0x0	-	设置portx组对应GPIO的spl为0 0: 无效 1: 有效

Bit	Name	RW	Default	RV	Description
15-0	SPL_SET	W	0x0	-	设置portx组对应GPIO的spl为1 0: 无效 1: 有效

3.3 USB_IO CONTROL

1. JL_PORTUSB->DIE:

2. JL_PORTUSB->DIEH:

3. JL_PORTUSB->DIR:

4. JL_PORTUSB->PU: 通用上拉电阻设置，一共三档，分别对应10k/200k/1M 欧姆上拉电阻

[注意]: 该设置用于PORTUSB在普通IO模式下设置上拉电阻档位，当PORTUSB用作USB通信功能时，需要关闭所有上拉电阻设置

5. JL_PORTUSB->PD: 通用下拉电阻设置，一共三档，分别对应10k/200k/1M 欧姆上拉电阻

[注意]: 该设置用于PORTUSB在普通IO模式下设置上拉电阻档位，当PORTUSB用作USB通信功能时，需要关闭所有上拉电阻设置

6. JL_PORTUSB->IN:

7. JL_PORTUSB->OUT:

8. JL_PORTUSB->SPL:

上述USB IO寄存器跟普通IO寄存器功能一样，不过USB 只有两个IO,BIT(0)设置DP，BIT(1)设置DM

9. JL_PORTUSB->CON: usb io control register 0

Bit	Name	RW	Default	RV	Description
31-13	RESERVED	-	-	-	预留
19	DM_FUSB_PD	rw	1	-	DM 满足USB协议的下拉电阻设置 0:关断下拉电阻 1: 15k电阻下拉
18	DP_FUSB_PD	rw	1	-	DP 满足USB协议的下拉电阻设置 0:关断下拉电阻 1: 15k电阻下拉
17	DM_FUSB_PU	rw	0	-	DM 满足USB协议的上拉电阻设置 0:关断上拉电阻 1: 180k电阻上拉

Bit	Name	RW	Default	RV	Description
16	DP_FUSB_PU	rw	0	-	DP 满足USB协议的上拉电阻设置 0:关断上拉电阻 1: 1.5k电阻上拉
15-13	RESERVED	-	-	-	预留
12-11	DBG_SEL	rw	0x0	-	测试信号选择: 00: 1'b0 0 1: usb_diff 10: usb_dp 11: usb_dm
10	USB_CP_MODE	rw	-	-	测试信号使能
9	USB_DF_MODE	rw	-	-	测试信号使能
8	IO_MODE	rw	0x1	-	IO模式使能 0: USB模式 1: 普通IO模式
7	CHKDPO	r	0x1	-	CHKDPO经系统时钟采样后的输入
6	DIDF	r	0	-	差分输入DIDF经lsb_clk采样后的输入
5	HDEN	-	-	-	驱动能力选择 0:low drive mode 1:high drive mode
4	PDCHKDP	rw	0	-	DP 外接下拉检查使能 0: disable 1: enable
3	DM_ADCEN	-	-	-	dm to adc en
2	SR0	rw	0x0	-	输出驱动能力选择
1	DP_ADCEN	rw	0	-	dp to adc en
0	RCVEN	rw	0	-	USB_PHY使能（差分输入使能，使用USB功能时需打开这一BIT）

3.4 IO模拟状态设置

当IO用于模拟功能时，需要正确设置IO寄存器，确保模拟功能不受IO影响。正确配置如下：

如将PA0设置为模拟功能。

```
JL_PORTA->DIR |= BIT(0);
JL_PORTA->DIE &= ~BIT(0);
JL_PORTA->DIEH &= ~BIT(0);
JL_PORTA->PU0 &= ~BIT(0);
JL_PORTA->PU1 &= ~BIT(0);
```

```
JL_PORTA->PD0 &= ~BIT(0);  
JL_PORTA->PD1 &= ~BIT(0);
```

杰理科技授权潮汐专用

4 IO_WAKEUP

4.1 概述

IO Wakeup是一个异步事件唤醒模块，它可以捕获IO 的上升沿/下降沿，将处于standby/sleep状态下的系统唤醒，进入正常工作模式。当系统处于正常模式时，Wakeup唤醒源可作为中断触发源，最多可有32个唤醒来源(具体数目以下述Wakeup唤醒源为准)；

4.2 Wakeup唤醒源

- 事件0: PA0
- 事件1: PA1
- 事件2: PA2
- 事件3: PA3
- 事件4: PA4
- 事件5: PA5
- 事件6: PA6
- 事件7: PB0
- 事件8: PB1
- 事件9: PB2
- 事件10: PB3
- 事件11: PB4
- 事件12: PB5
- 事件13: PB6
- 事件14: PB7
- 事件15: PB8
- 事件16: PB9
- 事件17: PB10
- 事件18: PB11
- 事件19: PC0
- 事件20: PC1
- 事件21: USB_DP
- 事件22: USB_DM
- 事件23: PP0 (VPWR)

- 事件24: PP1 (FSPG)
- 事件25: PP2 (SDPG)
- 事件26: gp_inch0
- 事件27: gp_inch1
- 事件28: gp_inch2
- 事件29: gp_inch3
- 事件30: gp_inch4
- 事件31: gp_inch5

唤醒源数量: 32

4.3 寄存器

4.3.1 WKUP_CON0: wakeup enable control register

偏移地址: 0x00

Bit	Name	RW	Default	Description
31-n	RESERVED	-	-	-
n-0	WKUP_EN	rw	0x0	对应唤醒源使能 0: 关闭唤醒功能 1: 打开唤醒功能

n 为唤醒源数量

4.3.2 WKUP_CON1: wakeup edge control register

偏移地址: 0x04

Bit	Name	RW	Default	Description
31-n	RESERVED	-	-	-
n-0	WKUP_EDGE	rw	0x0	对应唤醒源边沿选择 0: 上升沿唤醒; 1: 下降沿唤醒

n 为唤醒源数量

4.3.3 WKUP_CON2: wakeup pending clear control

偏移地址: 0x08

Bit	Name	RW	Default	Description
-----	------	----	---------	-------------

Bit	Name	RW	Default	Description
31-n	RESERVED	-	-	-
n-0	WKUP_CPND	w	0x0	写1清除对应唤醒源 pending

n 为唤醒源数量

4.3.4 WKUP_CON3: wakeup pending register

偏移地址: 0x0c

Bit	Name	RW	Default	Description
31-n	RESERVED	-	-	-
n-0	WKUP_PND	r	0x0	对应唤醒源 pending

n 为唤醒源数量

杰理科技授权潮汐专用

5 IOMC

5.1 概述

IOMC (IO remapping control) 控制寄存器主要用于控制IO除crossbar之外的一些拓展功能的配置，每个外设类型都具有相应的独立IOMC控制寄存器；

项目存在两个基地址不同的IOMC，分别为JL_IOMC和JL_SFC_IOMC，前者用于处理通用外设的拓展功能，后者只用于SFC/spi0处于flash相关的功能。

5.2 JL_IOMC控制寄存器

5.2.1 SPI_IOMC0: spi iomc control register 0

偏移地址: 0x14

Bit	Name	RW	Default	Description
31	Reserved	-	-	Reserved
30-27	Reserved	-	-	Reserved
26-24	Reserved	-	-	Reserved
23	Reserved	-	-	Reserved
22-20	Reserved	-	-	Reserved
18-16	Reserved	-	-	Reserved
15-13	Reserved	-	-	Reserved
12	Reserved	-	-	Reserved
11-8	Reserved	-	-	Reserved
7	Reserved	-	-	Reserved
6-3	Reserved	-	-	Reserved
2-0	Reserved	-	-	Reserved

5.2.2 SDC_IOMC0: sdc iomc control register 0

偏移地址: 0x18

Bit	Name	RW	Default	Description
-----	------	----	---------	-------------

Bit	Name	RW	Default	Description
31	Reserved	-	-	Reserved
30-27	Reserved	-	-	Reserved
26-24	Reserved	-	-	Reserved
23	Reserved	-	-	Reserved
22-19	Reserved	-	-	Reserved
18-16	Reserved	-	-	Reserved
15	Reserved	-	-	Reserved
14-10	Reserved	-	-	Reserved
11	Reserved	-	-	Reserved
10-8	Reserved	-	-	Reserved
7-2	Reserved	-	-	Reserved
3	Reserved	-	-	Reserved
2-0	Reserved	-	-	Reserved

5.2.3 sys iomc control register 0

偏移地址: 0x1c

Bit	Name	RW	Default	Description
31-12	Reserved	-	-	Reserved
11	Reserved	-	-	Reserved
10	Reserved	-	-	Reserved
9	Reserved	-	-	Reserved
8	Reserved	-	-	Reserved
7	Reserved	-	-	Reserved
6	Reserved	-	-	Reserved
5	Reserved	-	-	Reserved
4	Reserved	-	-	Reserved
3	Reserved	-	-	Reserved
2	Reserved	-	-	Reserved

Bit	Name	RW	Default	Description
1-0	Reserved	-	-	Reserved

5.2.4 sys iomc control register 1

偏移地址: 0x20

Bit	Name	RW	Default	Description
31	Reserved	-	-	Reserved
30-27	Reserved	-	-	Reserved
26	Reserved	-	-	Reserved
25-24	Reserved	-	-	Reserved
23-0	Reserved	-	-	Reserved

5.2.5 ASS_IOMC0: audio iomc control register 0

偏移地址: 0x38

Bit	Name	RW	Default	Description
31-26	ASS_OCH1_sSEL	rw	0	
25	ASS_OCH1_REG	rw	0	
24	ASS_OCH1_EN	rw	0	
23-18	ASS_OCH0_sSEL	rw	0	
17	ASS_OCH0_REG	rw	0	
16	ASS_OCH0_EN	rw	0	
15	Reserved	-	-	Reserved
14-11	Reserved	-	-	Reserved
10-8	Reserved	-	-	Reserved
7	Reserved	-	-	Reserved
6-3	Reserved	-	-	Reserved
2-0	Reserved	-	-	Reserved

5.2.6 WL_IOMC0: wireless iomc control register 0

偏移地址: 0x3c

Bit	Name	RW	Default	Description
31	Reserved	-	-	Reserved
30-27	Reserved	-	-	Reserved
26-24	Reserved	-	-	Reserved
23	Reserved	-	-	Reserved
22-19	Reserved	-	-	Reserved
18-16	Reserved	-	-	Reserved
15	Reserved	-	-	Reserved
14-11	Reserved	-	-	Reserved
10-8	Reserved	-	-	Reserved
7-6	Reserved	-	-	Reserved
5	Reserved	-	-	Reserved
4	Reserved	-	-	Reserved
3	Reserved	-	-	Reserved
2-0	Reserved	-	-	Reserved

5.2.7 ICH_IOMC0: input channel control register0

偏移地址: 0x40

Bit	Name	RW	Default	Description
31-30	Reserved	-	-	
29-25	ICH5_SEL (TMR1_CAP)	rw	0	同ICH0选项
24-20	ICH4_SEL (TMR0_CAP)	rw	0	同ICH0选项
19-15	ICH3_SEL (TMR3_CIN)	rw	0	同ICH0选项
14-10	ICH2_SEL (TMR2_CIN)	rw	0	同ICH0选项
9-5	ICH1_SEL (TMR1_CIN)	rw	0	同ICH0选项

Bit	Name	RW	Default	Description
4-0	ICH0_SEL (TMR0_CIN)	rw	0	功能输入通道选择 0-5: gp_ich0-gp_ich5 6: tmr2_pwm 7: tmr3_pwm 8: wl_ampe 9: wl_lnae

5.2.8 ICH_IOMC1: input channel control register1

偏移地址: 0x44

Bit	Name	RW	Default	Description
31-30	Reserved	-	-	
29-25	ICH11_SEL (MC_PWM1_FP)	rw	0	同ICH0选项
24-20	ICH10_SEL (MC_PWM0_FP)	rw	0	同ICH0选项
19-15	ICH9_SEL (MC_PWM1_CK)	rw	0	同ICH0选项
14-10	ICH8_SEL (MC_PWM0_CK)	rw	0	同ICH0选项
9-5	ICH7_SEL (TMR3_CAP)	rw	0	同ICH0选项
4-0	ICH6_SEL (TMR2_CAP)	rw	0	同ICH0选项

5.2.9 ICH_IOMC2: input channel control register2

偏移地址: 0x48

Bit	Name	RW	Default	Description
31-30	Reserved	-	-	
29-25	ICH17_SEL (EXT_CLK_P)	rw	0	同ICH0选项
24-20	ICH16_SEL (CLK_MUX_IN)	rw	0	同ICH0选项
19-15	ICH15_SEL (CAP_MUX_OUT)	rw	0	同ICH0选项
14-10	ICH14_SEL (PLNK_IDAT1)	rw	0	同ICH0选项
9-5	ICH13_SEL (PLNK_IDAT0)	rw	0	同ICH0选项
4-0	ICH12_SEL (UART1_CTS)	rw	0	同ICH0选项

5.2.10 ICH_IOMC3: input channel control register3

偏移地址: 0x4c

Bit	Name	RW	Default	Description
31-30	Reserved	-	-	
29-25	ICH23_SEL (Reserve)	-	0	同ICH0选项
24-20	ICH22_SEL (Reserve)	-	0	同ICH0选项
19-15	ICH21_SEL (Reserve)	-	0	同ICH0选项
14-10	ICH20_SEL (AUD_DBG_DATI)	rw	0	同ICH0选项
9-5	ICH19_SEL (WLC_EXT_ACTIVE)	rw	0	同ICH0选项
4-0	ICH18_SEL (SPI1_CS)	rw	0	同ICH0选项

5.2.11 ICH_IOMC4: input channel control register4

偏移地址: 0x50

Bit	Name	RW	Default	Description
31-30	Reserve	-	-	
29-25	ICH29_SEL (Reserve)	rw	0	同ICH0选项
24-20	ICH28_SEL (Reserve)	rw	0	同ICH0选项
19-15	ICH27_SEL (Reserve)	rw	0	同ICH0选项
14-10	ICH26_SEL (Reserve)	rw	0	同ICH0选项
9-5	ICH25_SEL (Reserve)	rw	0	同ICH0选项
4-0	ICH24_SEL (Reserve)	rw	0	同ICH0选项

5.2.12 ICH_IOMC5: input channel control register5

偏移地址: 0x54

Bit	Name	RW	Default	Description
31-10	Reserved	-	-	
19-15	ICH33_SEL (Reserved)	rw	0	同ICH0选项
14-10	ICH32_SEL (Reserved)	rw	0	同ICH0选项
9-5	ICH31_SEL (Reserved)	rw	0	同ICH0选项
4-0	ICH30_SEL (Reserved)	rw	0	同ICH0选项

5.2.13 OCH_IOMC0: output channel control register0

偏移地址: 0x58

Bit	Name	RW	Default	Description
31-24	OCH3_SEL	rw	0	同OCH0选项 27: wl_dbg_port[3]
23-16	OCH2_SEL	rw	0	同OCH0选项 27: wl_dbg_port[2]
15-8	OCH1_SEL	rw	0	同OCH0选项 27: wl_dbg_port[1]
7-0	OCH0_SEL	rw	0	通用输出通道选择 0: tmr0_pwm 1: tmr1_pwm 2: tmr2_pwm 3: tmr3_pwm 4: gp_ich0 5: gp_ich1 6: uart1_rts 7: plnk_clk 8: wl_ampe 9: wl_lnae 10: wlc_int_active 11: wlc_int_status 12: wlc_int_freq 13: aud_dbg_clko 14: aud_dbg_dato[0] 15: aud_dbg_dato[1] 16: aud_dbg_dato[2] 17: aud_dbg_dato[3] 18: aud_dbg_dato[4] 19: clk_out2io[0] 20: clk_out2io[1] 21: clk_out2io[2] 22: p33_clk_dbg 23: p33_sig_dbg[0] 24: p33_sig_dbg[1] 25: usb_dbg_out 26: p11_dbg_out 27: wl_dbg_port[0]

5.2.14 OCH_IOMC1: output channel control register1

偏移地址: 0x5c

Bit	Name	RW	Default	Description
31-24	OCH7_SEL	rw	0	同OCH0选项 27: wl_dbg_port[7]

Bit	Name	RW	Default	Description
23-16	OCH6_SEL	rw	0	同OCH0选项 27: wl_dbg_port[6]
15-8	OCH5_SEL	rw	0	同OCH0选项 27: wl_dbg_port[5]
7-0	OCH4_SEL	rw	0	同OCH0选项 27: wl_dbg_port[4]

5.2.15 OCH_IOMC2/3/4/5: output channel control register2/3/4/5

偏移地址: 0x60/0x64/0x68/0x6c

Bit	Name	RW	Default	Description
31-24	OCH11_SEL/OCH15_SEL/OCH19_SEL/OCH23_SEL	-	-	Reserved
23-16	OCH10_SEL/OCH14_SEL/OCH18_SEL/OCH22_SEL	-	-	Reserved
15-8	OCH9_SEL/OCH13_SEL/OCH17_SEL/OCH21_SEL	-	-	Reserved
15-8	OCH8_SEL/OCH12_SEL/OCH16_SEL/OCH20_SEL	-	-	Reserved

5.2.16 SDPG_CON: SDPG control register

偏移地址: 0x68

Bit	Name	RW	Default	Description
31-1	Reserved	-	-	Reserved
0	SDPG_SOFT	rw	0	SDPG 驱动强度选择, 为SDCard供电时, 上下电速度的控制 0: 快上/下电 1: 慢上/下电

注意: 当SDPG($PP2$)的HD(JL_PORTP->HD的对应位置)为0时, SDPG处于IO Mode, 此时由SDPG_SOFT控制输出压摆率, 当SDPG($PP2$)的HD为1时, SDPG处于PG Mode, 此时由SDPG_SOFT控制输出上/掉电模式

处于IO Mode时($HD=0$)

SDPG_SOFT为0, SDPG输出信号最高频率为50MHz ($tr/tf < 5ns$)

SDPG_SOFT为1, SDPG输出信号最高输出频率为 5KHz ($tr/tf > 20us$)

处于PG Mode时($HD=1$)

SDPG_SOFT为0, SDPG处于正常供电的状态, 输出稳定可靠的电压

SDPG_SOFT为1, SDPG处于缓上/掉电的状态, 输出缓慢变化的电压($tr \geq 200us, tf \geq 20us @$
load=1uF ~ 10uF)

处于PG mode时, SDPG_SOFT的控制时序如下:

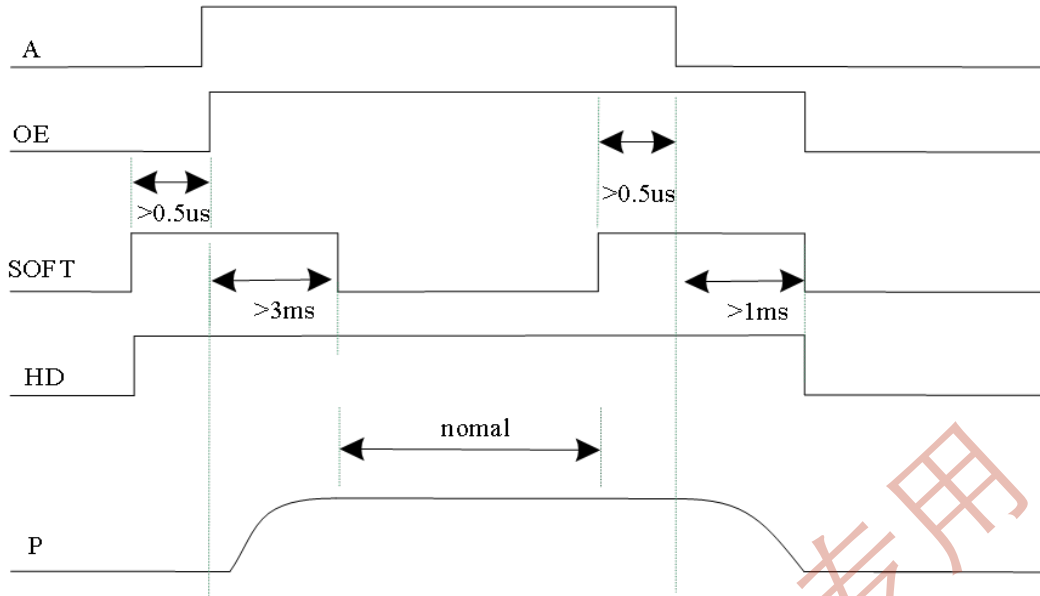


图1. SDPG_SOFT的控制时序

注意：设置SDPG(P2)为工作状态（即配置DIR或OUT）之前，需提前配置SDPG_SOFT=1、HD=1并至少保持0.5us的时间

缓上电流程，提前配置SDPG_SOFT=1、HD=1并至少保持0.5us的时间，设置SDPG为输出（即配置OUT为1，DIR为0），并保持SDPG_SOFT为1至少3ms的时间

缓掉电流程，保持SDPG输出电平不变，提前配置SDPG_SOFT=1、HD=1并至少保持0.5us的时间，然后切换SDPG的输出电平（即配置OUT为0），之后保持SDPG_SOFT为1至少1ms的时间

5.3 JL_SFC_IOMC控制寄存器

5.3.1 FSPG_CON: FSPG control register

偏移地址: 0x00

Bit	Name	RW	Default	Description
31-3	Reserved	-	-	Reserved
2	SOFT	rw	0	FSPG 驱动强度选择，为flash 供电时，上/掉电速度的控制 0: 快上/下电 (tr/tf≤2us) 1: 慢上/下电 (tr/tf≥20us)
1	CS1_EN	rw	0	是否短接CS1到FSPG 该功能用于FLASH上/掉电时避免电流通过CS PIN倒灌
0	CS0_EN	rw	0	是否短接CS0到FSPG 该功能用于FLASH上/掉电时避免电流通过CS PIN倒灌

SOFT功能用于控制FSPG(PP1)的上电和掉电速度

当SOFT为0时，FSPG处于IO mode，输出信号最高频率为50MHz

当SOFT为1时，FSPG处于PG mode，输出缓上电缓掉电的电压

处于PG mode时，SOFT的控制时序如下：

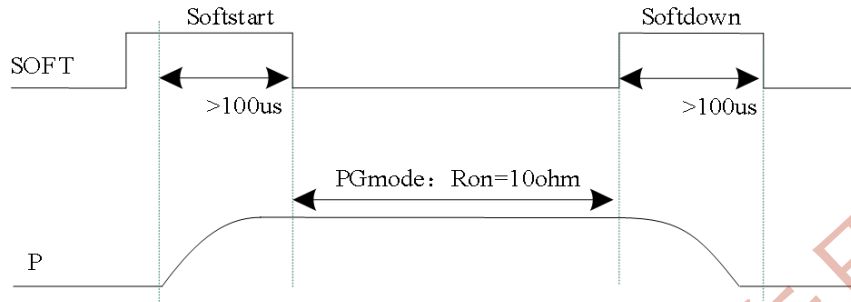


图2. FSPG SOFT的控制时序

5.3.2 IOMC0: IOMC0 register

偏移地址: 0x04

Bit	Name	RW	Default	Description
31-2	Reserved	-	-	Reserved
6-4	SPI0_IOS	rw	0	SPI0 固定IO mux 选择，具体IO参考IO Mapping 0: A组IO 1: B组IO
3-2	Reserved	-	-	Reserved
1	SFC_IOS	rw	0	SFC 固定IO mux 选择，具体IO参考IO Mapping 0: A组IO 1: B组IO
0	SFC_1P8_EN	-	-	Reserved (本项目不存在)

5.3.3 IOMC1: IOMC1 register

偏移地址: 0x08

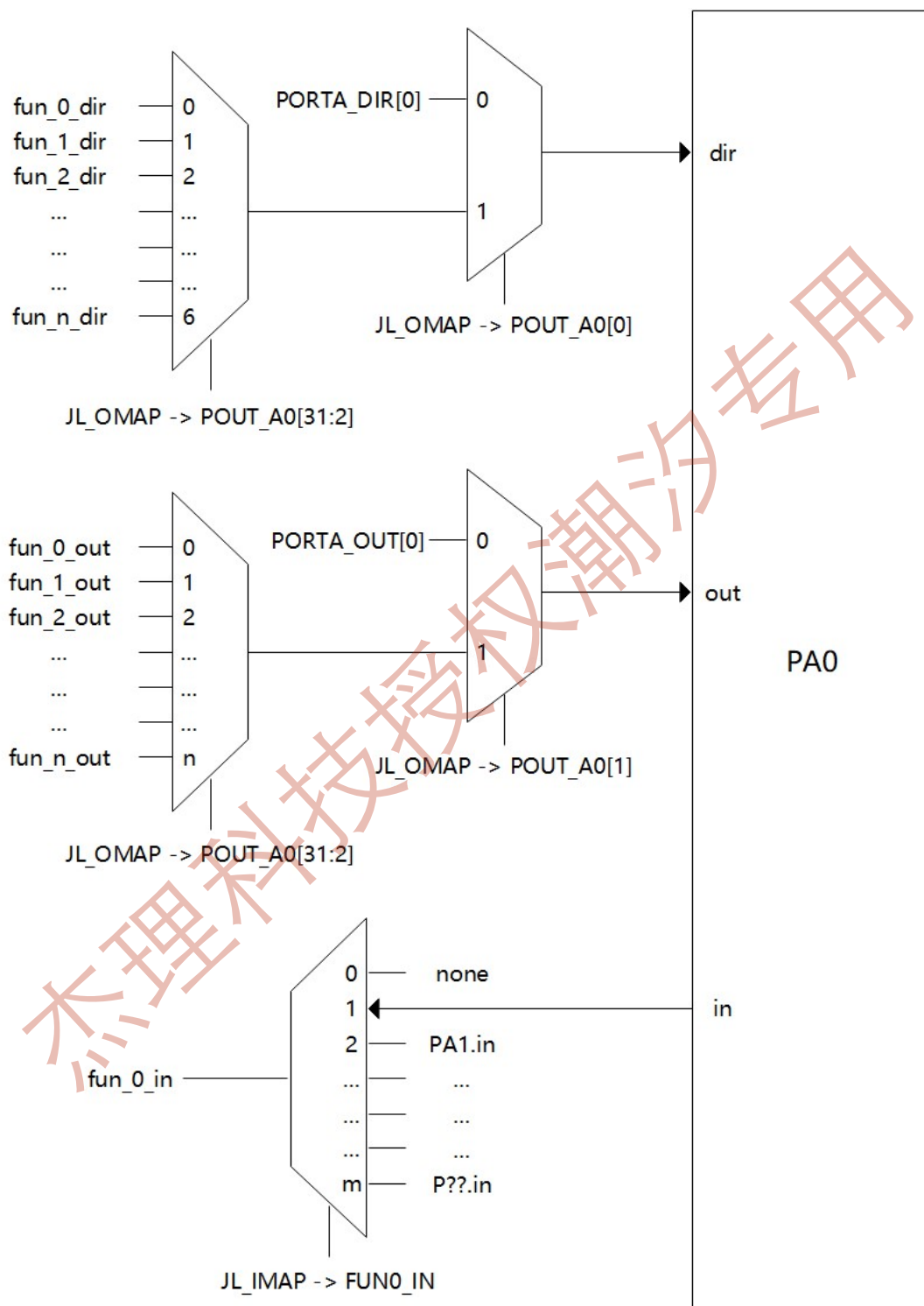
Bit	Name	RW	Default	Description
-----	------	----	---------	-------------

Bit	Name	RW	Default	Description
31-0	Reserved	-	-	Reserved

杰理科技授权潮汐专用

6 PORT Control Crossbar

6.1 Crossbar功能结构图



6.2 寄存器功能

6.2.1 JL_OMAP-> POUT

Bit	Name	RW	Description
[0]	FUN_DIR_EN	rw	Crossbar 方向控制使能: 0: CPU 控制 1: 外设控制 (注: 若所选外设无方向控制功能, 则都由 CPU 控制)
[1]	FUN_OUT_EN	rw	Crossbar 数据控制使能: 0: CPU 数据 1: 外设数据
[31:2]	FUN_SELECT	rw	Crossbar 输出功能选择: (见功能表或头文件)
[31:0]	INPUT_SELECT	rw	Crossbar 输入引脚选择: (见功能表或头文件)

6.2.2 JL_IMAP-> FUN_IN

Bit	Name	RW	Description
[31:0]	INPUT_SELECT	rw	Crossbar 输出引脚选择: (见功能表或头文件)

7 CLOCK_SYSTEM

7.1 原生时钟源

本芯片具备的原生时钟源如下表所示：

时钟	概述
rc_250k	内置RC振荡器，用于复位系统和watch dog功能时钟
rc_16M	内置RC振荡器，用于系统启动的初始时钟
lrc_200k	内置低温度电压漂移RC振荡器，用于低功耗计数和PLL参考时钟
xosc_24M	外挂晶振

7.2 内部PLL时钟源

PLL时钟	概述
SYSPLL	中心频率192MHz，具备D1P0，D1P5，D2P0，D2P5，D3P5 分频输出

7.3 标准时钟

除上述的原生时钟外，芯片内还有频率固定的标准时钟，分别是wclk、pll_96m、std_48m、std_24m和std_12m，其时钟源头选择如下：

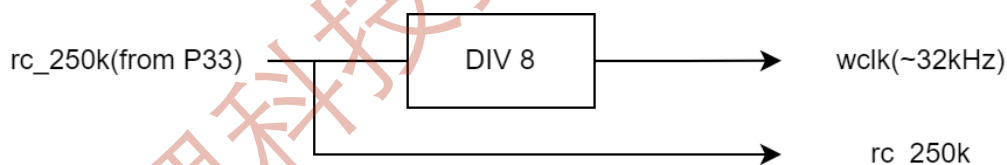


图1 系统 rc_250k 和 wclk 时钟结构

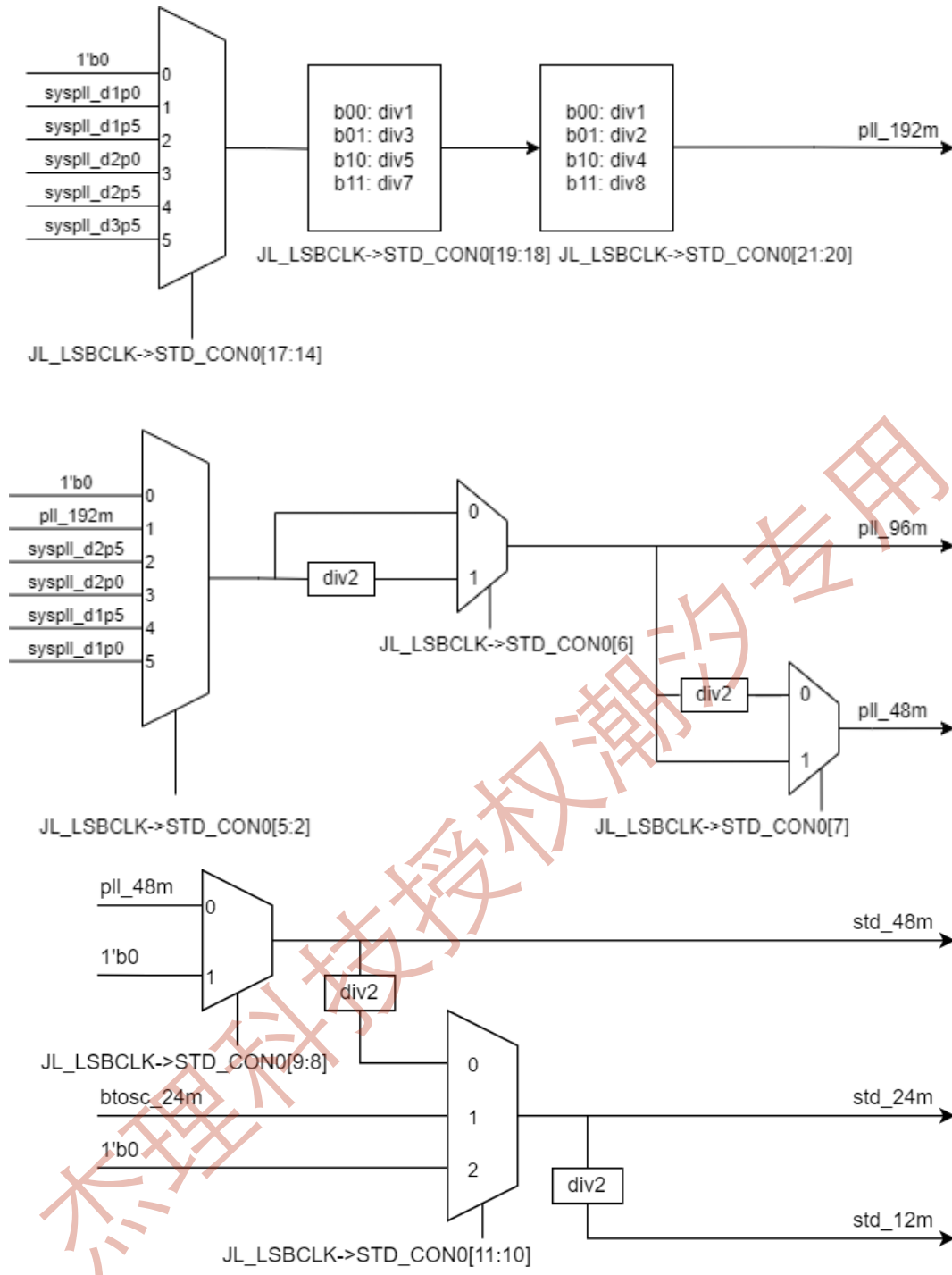


图2 STD_CLK时钟结构图

7.4 系统时钟

系统时钟部分电路框图如下，系统运行过程中可随时更改hsb_clk，lsb_clk的分频值，但需确保在任何时刻，各分频时钟都不会超过其允许的最高运行频率。

7.4.1 HSB时钟源

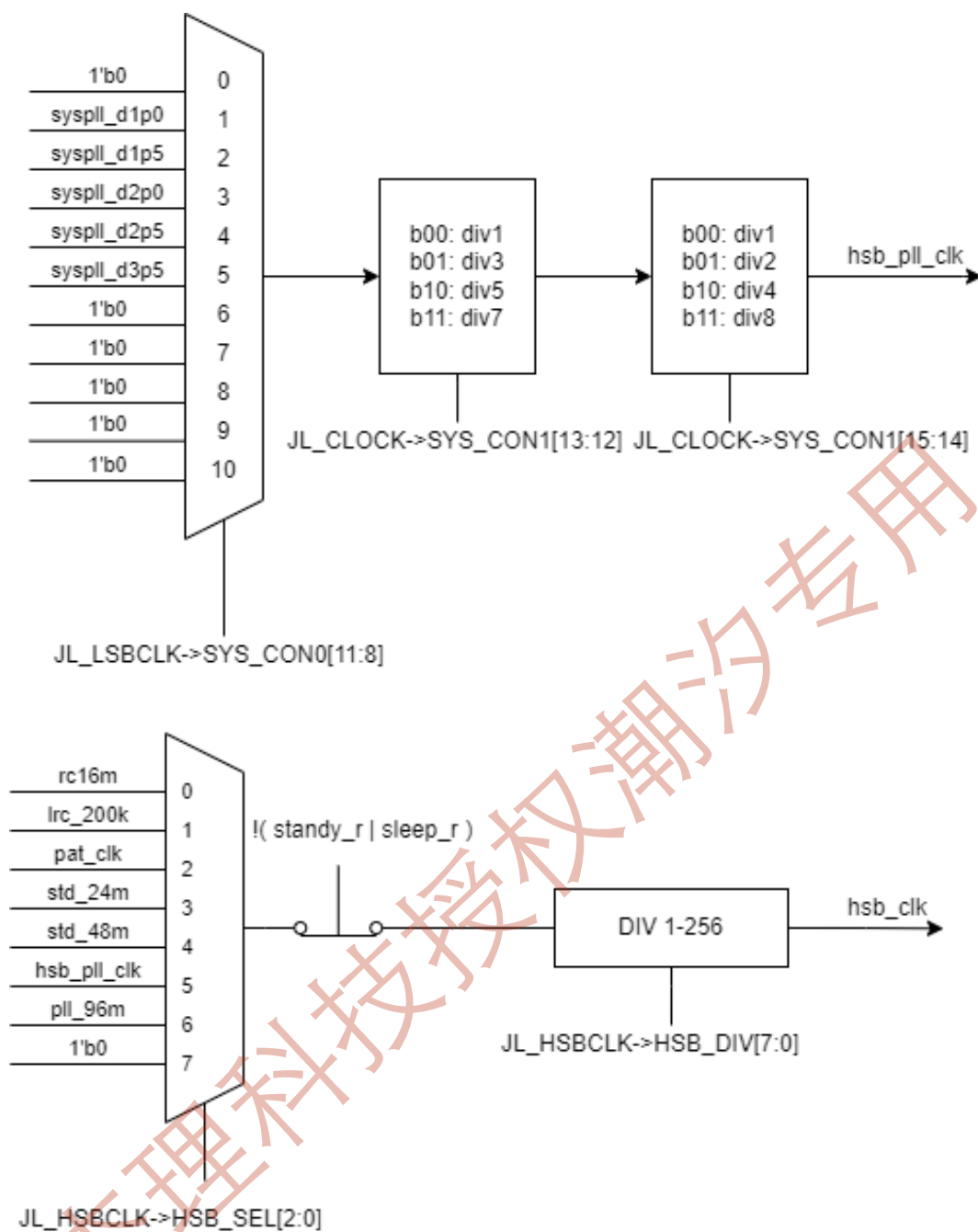


图3 hsb_clk时钟结构图

7.4.2 LSB时钟源

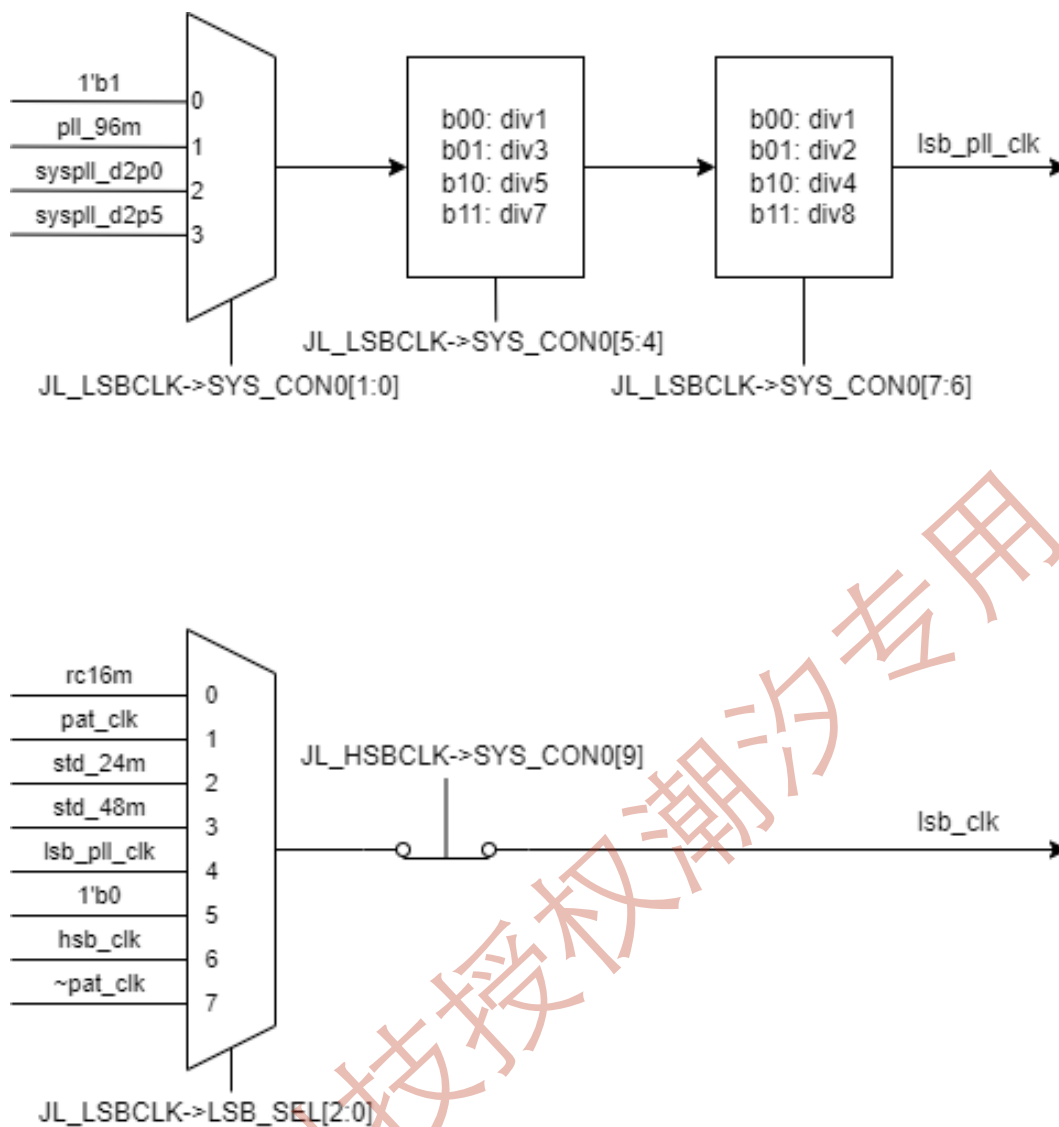


图4 lsb_clk时钟电路

【注意】：LSB_SEL 不能连续切换，两次切换中间需要等待5个LSB_CLK;

7.5 外设时钟

7.5.1 SFC 时钟源

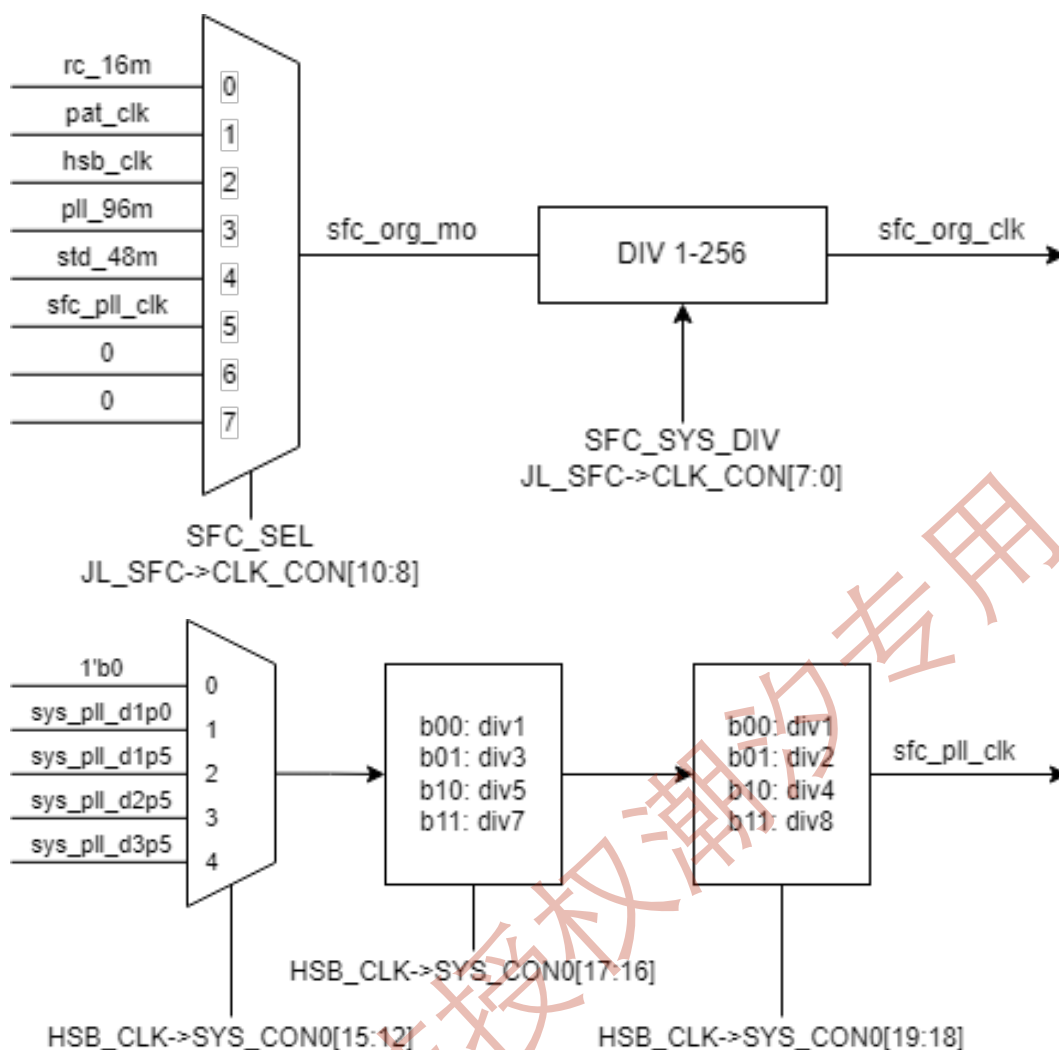
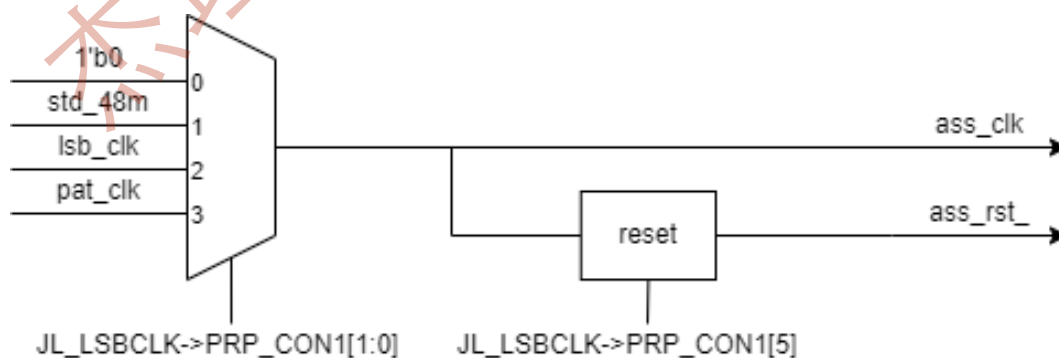


图5 SFC时钟结构图

【注意】：SFC_CLK可动态切换，但不能连续切换，两次切换中间需要等待大于5个SFC_CLK；
【注意2】：sfc_pll_clk的时钟与sfc_org_clk的控制寄存器的基地址不同

7.5.2 音频时钟源



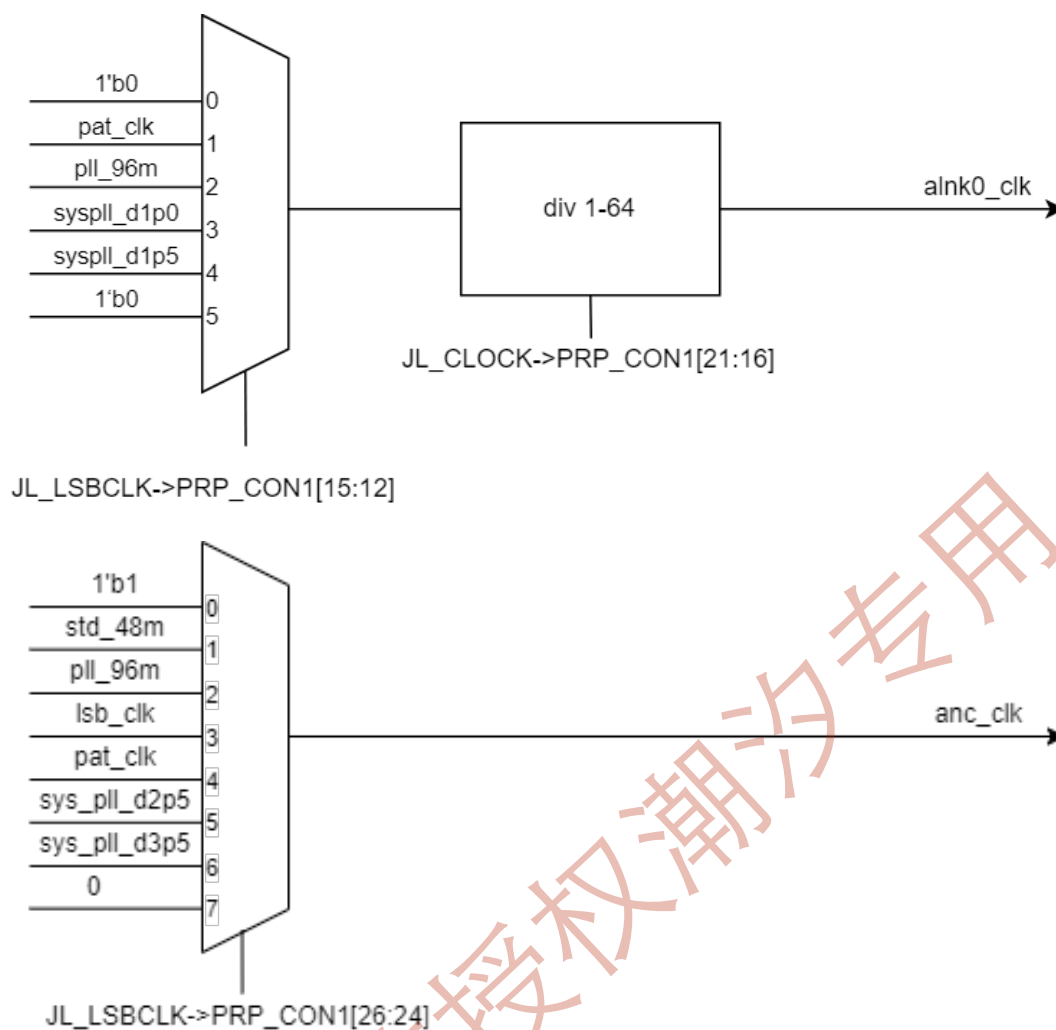
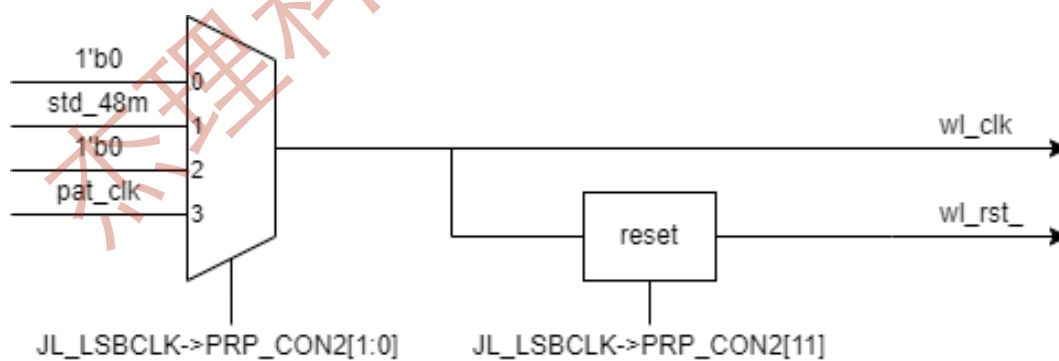


图6 音频时钟结构图

7.5.3 WL时钟源



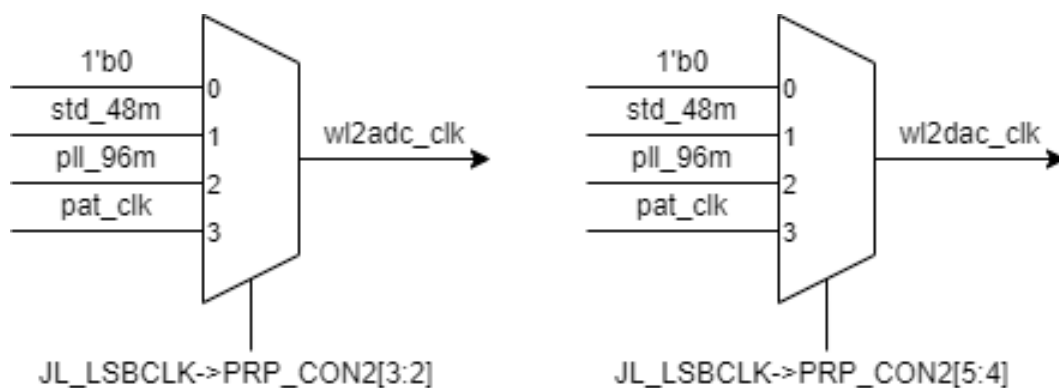
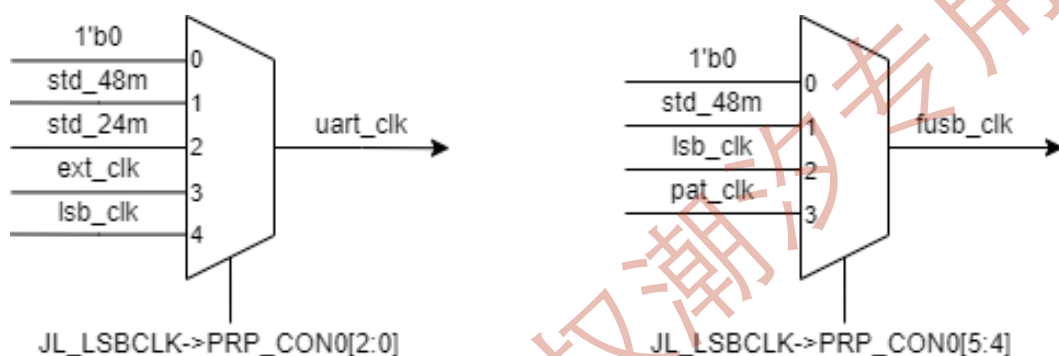
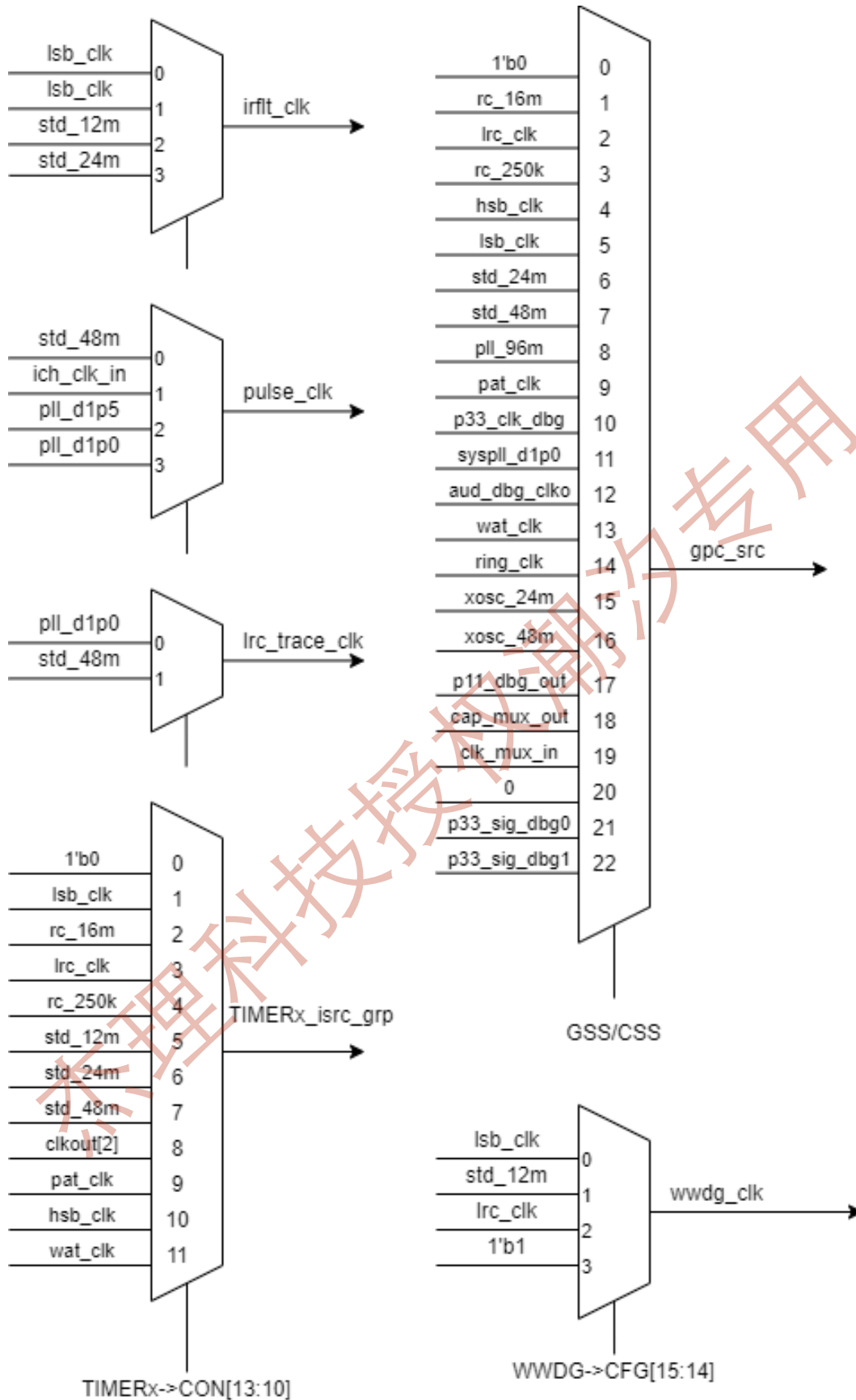


图7 无线模块时钟结构图

7.5.4 外设时钟源





- 该芯片有2组时钟输出，可以通过crossbar将内部的时钟信号驱动至片外，用于测试或特殊用途。

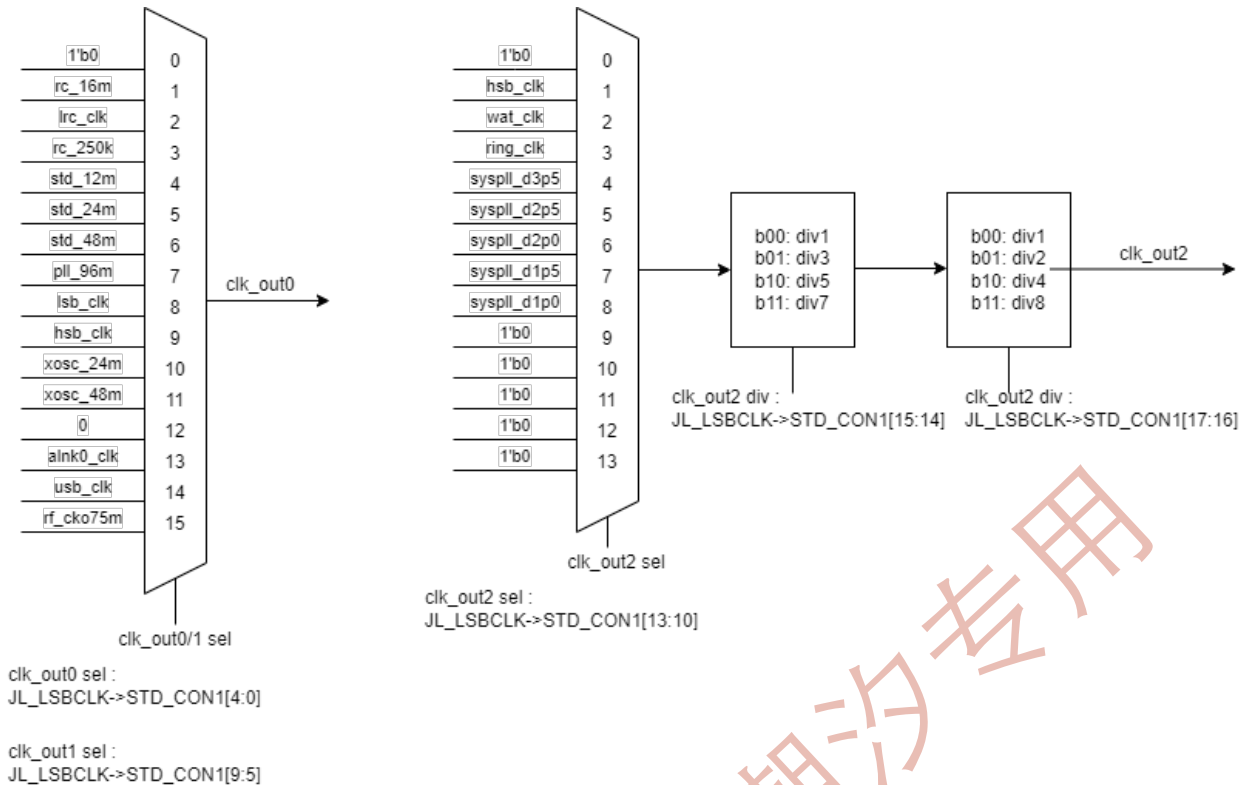


图9 IO输出时钟结构图

7.6 数字模块控制寄存器

7.6.1 JL_LSBCK->SYS_CON0: lsback system config register 0

Bit	Name	RW	Default	RV	Description
31-11	RESERVED	-	0	-	预留
15-12	HSB_PLL_DIV	rw	0	-	HSB_PLL时钟分频, 见图3
11-8	HSB_PLL_SEL	rw	0	-	HSB_PLL时钟选择, 见图3
7-4	LSB_PLL_DIV	rw	0	-	LSB_PLL时钟分频, 见图4
3-0	LSB_PLL_SEL	rw	0	-	LSB_PLL时钟选择, 见图4

7.6.2 LSB_SEL: lsb clock sel register

Bit	Name	RW	Default	RV	Description
31-20	RESERVED	-	0	-	预留
2-0	LSB_SEL	rw	0	-	LSB时钟选择, 见图4

7.6.3 JL_LSBCK->STD_CON0: standard clock register 0

Bit	Name	RW	Default	RV	Description
31-9	RESERVED	-	0	-	预留
21-18	PLL_192M_DIV	rw	0	-	PLL_192M分频系数，见图2
17-14	PLL_192M_SEL	rw	0	-	PLL_192M输入时钟源选择，见图2
13-12	RESERVED	-	0	-	预留
11-10	STD_24M_SEL	rw	0	-	STD_24M来源选择，见图2
9-8	STD_48M_SEL	rw	0	-	STD_48M来源选择，见图2
7	PLL_48M_DS	rw	0	-	PLL_48M DIV2使能选择，见图2
6	PLL_96M_DS	rw	0	-	PLL_96M DIV2使能选择，见图2
5-2	PLL_96M_SEL	rw	0	-	PLL_96M输入时钟源选择，见图2
1	RC_250K_EN	rw	1	-	片内RC（250k）振荡器使能（PMU低功耗模式无效） 0: 关闭片内RC振荡器 1: 打开片内RC振荡器
0	RC_16M_EN	rw	1	-	片内RC（16M）振荡器使能（PMU低功耗模式无效） 0: 关闭片内RC振荡器 1: 打开片内RC振荡器

7.6.4 JL_LSBCK->STD_CON1: standard clock register 1

Bit	Name	RW	Default	RV	Description
31-20	RESERVED	-	-	-	预留
17-14	CLKOUT2_DIV	rw	1	-	CLK_OUT2输出时钟分频选择，见图9
13-10	CLKOUT2_SEL	rw	0	-	CLK_OUT2输出时钟源选择，见图9
9-5	RESERVED	-	-	-	预留
4-0	CLKOUT0_SEL	rw	0	-	CLK_OUT0输出时钟选择，见图9

7.6.5 JL_LSBCK->STD_CON2: standard clock register 2

Bit	Name	RW	Default	RV	Description
-----	------	----	---------	----	-------------

Bit	Name	RW	Default	RV	Description
31-8	RESERVED	-	-	-	预留
7-4	XOSC_MDXT_CKSEL	rw	0	0	晶振单双脚检测时钟选择 0:0 1:STD_24M 2:RC_16M 3:0
3-0	XOSC_EXT_CKSEL	rw	0	0	晶振牵引时钟选择 0:0 1:STD_24M 2:RC_16M 3:0

7.6.6 JL_LSBCK->PRP_CON0: peripheral clock config register 0

Bit	Name	RW	Default	RV	Description
31-10	RESERVED	-	0	-	预留
9	RINGOSC_CKEN	rw	0	-	ring_clk使能
8-7	MBIST_CKSEL	rw	0	-	IC内部测试功能，MBIST时钟源
6	USB_CKPOL	rw	0	0	IC内部测试功能，fusb时钟反相
5-4	USB_CKSEL	rw	0	-	fusb时钟源选择，见图8
3	UART_CKPOL	rw	0	0	IC内部测试功能，uart 时钟反相
2-0	UART_CKSEL	rw	0	-	uart 时钟源选择，见图8

7.6.7 JL_LSBCK->PRP_CON1: peripheral clock config register 1

Bit	Name	RW	Default	RV	Description
31-27	RESERVED	-	0	-	预留
26-24	anc_sel	rw	0	-	anc时钟选择
23-22	RESERVED	-	0	-	预留
21-16	ALNK0_DIV	rw	0	-	alnk0_clk分频选择，见图8
15-12	ALNK0_SEL	rw	0	-	alnk0_clk来源选择，见图8
11-6	RESERVED	-	0	-	预留
5	ASS_REG_RSTN	rw	0	-	ass复位

Bit	Name	RW	Default	RV	Description
4-2	RESERVED	-	0	-	预留
1-0	ASS_CKSEL	rw	0	-	ass_clk来源选择, 见图8

7.6.8 JL_LSBCK->PRP_CON2: peripheral clock config register 2

Bit	Name	RW	Default	RV	Description
31-12	RESERVED	-	0	-	预留
11	WL_REG_RSTN	rw	0	-	wl_aud复位
10-6	RESERVED	-	0	-	预留
5-4	WL_DAC_CKSEL	rw	0	-	wl_dac_clk来源选择, 见图9
3-2	WL_ADC_CKSEL	rw	0	-	wl_adc_clk来源选择, 见图9
1-0	WL_CKSEL	rw	0	-	wl_clk来源选择, 见图9

7.6.9 JL_HSBCK->SYS_CON0: hsbck system config register 0

Bit	Name	RW	Default	RV	Description
31-11	RESERVED	-	-	-	预留
19-16	SFC_PLL_DIV	rw	0	-	sfc_pll_mo的时钟分频
15-12	SFC_PLL_SEL	rw	0	-	sfc_pll_mo的来源选择: 0: 空 1: sys_pll_d1p0 2: sys_pll_d1p5 3: sys_pll_d2p5 4: sys_pll_d3p5
11	RESERVED	-	-	-	预留
10	LSB_SFR_CKMD	rw	0	-	lsb_sfr_clk时钟模式选择: 0: 有SFR写的时候, lsb_sfr_clk才使能 1: 无论有无SFR写, lsb_sfr_clk都使能
9	LSB_CKEN	rw	1	-	LSB时钟使能, 默认开
8	AXI_EN	rw	1	-	axi_clk使能: 0: 不使能 1: 使能
7-1	RESERVED	-	0	-	预留

Bit	Name	RW	Default	RV	Description
0	HSB_SFR_CKMD	rw	0	-	hsb_sfr_clk时钟模式选择: 0: 有SFR写的时候, hsb_sfr_clk才使能 1: 无论有无SFR写, hsb_sfr_clk都使能

7.6.10 JL_HSBCLK->HSB_DIV: hsb clock div register

Bit	Name	RW	Default	RV	Description
31-20	RESERVED	-	0	-	预留
7-0	HSB_DIV	rw	0	-	HSB时钟分频, 见图5

7.6.11 JL_HSBCLK->HSB_SEL: hsb clock sel register

Bit	Name	RW	Default	RV	Description
31-20	RESERVED	-	0	-	预留
2-0	HSB_SEL	rw	0	-	HSB时钟选择, 见图5

8 ADC

8.1 概述

10Bit ADC(A/D转换器)，其时钟最大不可超过500KHz。

8.2 寄存器说明

8.2.1 ADC_CON: ADC control register

Bit	Name	RW	Default	RV	Description
31	DONE_PND	r	0	-	只读，采样完成pnd; kst后完成一次该位置1
30	DONE_PND_CLR	w	-	-	只写，写‘1’清除DONE_PND位，写‘0’无效
29	DONE_PND_IE	rw	0	-	DONE_PND中断允许
28	RESERVED	-	-	-	预留
27-24	ADC_ASEL	rw	0x0	-	内部模拟通道选择: 0: BT_L 1: AUDIO 2: PMU 3: LRC200K 4: OCP 5: SYSPLL 6: LPCTM 7: WAT 8: XOSC 9: PADC0 10: BT_H
23-21	ADC_MUX_SEL	rw	0x0	0	ADC模式选择: 001: ADC采集IO通道电压 010: ADC采集内部模拟通道电压 111: 将选通的内部模拟电压通过IO口输出
20-18	RESERVED	-	-	-	预留
17	ADC_CLKEN	rw	0	0	ADC 时钟使能
16	ADCISEL	rw	0		ADC CMP power select
15-12	WAIT_TIME	rw	x	-	启动延时控制，实际启动延时为此数值乘8个ADC时钟

Bit	Name	RW	Default	RV	Description
11-8	CH_SEL	rw	x	-	IO通道选择: 0: 选择PC1 1: 选择PC2 2: 选择PC3 3: 选择PC4 4: 选择PA3 5: 选择PA4 6: 选择PA5 7: 选择PB0 8: 选择PB1 9: 选择PB2 10: 选择PB3 11: 选择PB7 12: 选择PB8 13: 选择PP2 14: 选择USBDP 15: 选择USBDM
7	PND	r	1	-	PND: 只读, 中断请求位, 当ADC完成一次转换后, 此位会被设置为‘1’, 需由软件清‘0’
6	CPND	w	x	-	CPND: 只写, 写‘1’清除中断请求位, 写‘0’无效
5	ADC_IE	rw	0	-	ADC_IE: ADC中断允许
4	ADC_EN	rw	0	-	ADC_EN: ADC控制器使能
3	ADC_AE	rw	0	-	ADC_AE: ADC 模拟模块常使能
2-0	ADC_BAUD	rw	0x0	-	ADC_BAUD: ADC时钟频率选择 000: LSB时钟1分频 001: LSB时钟6分频 010: LSB时钟12分频 011: LSB时钟24分频 100: LSB时钟48分频 101: LSB时钟72分频 110: LSB时钟96分频 111: LSB时钟128分频

8.2.2 ADC_RES: ADC result register

Bit	Name	RW	Default	RV	Description
31-10	RESERVED	-	-	-	预留
9-0	RES	r	x	-	ADC结果

9 CRC

9.1 概述

CRC16计算器，每次运算8bit，多项式为： $X^{16} + X^{12} + X^5 + X^1$ 。

CRC模块的原理框图如下所示：

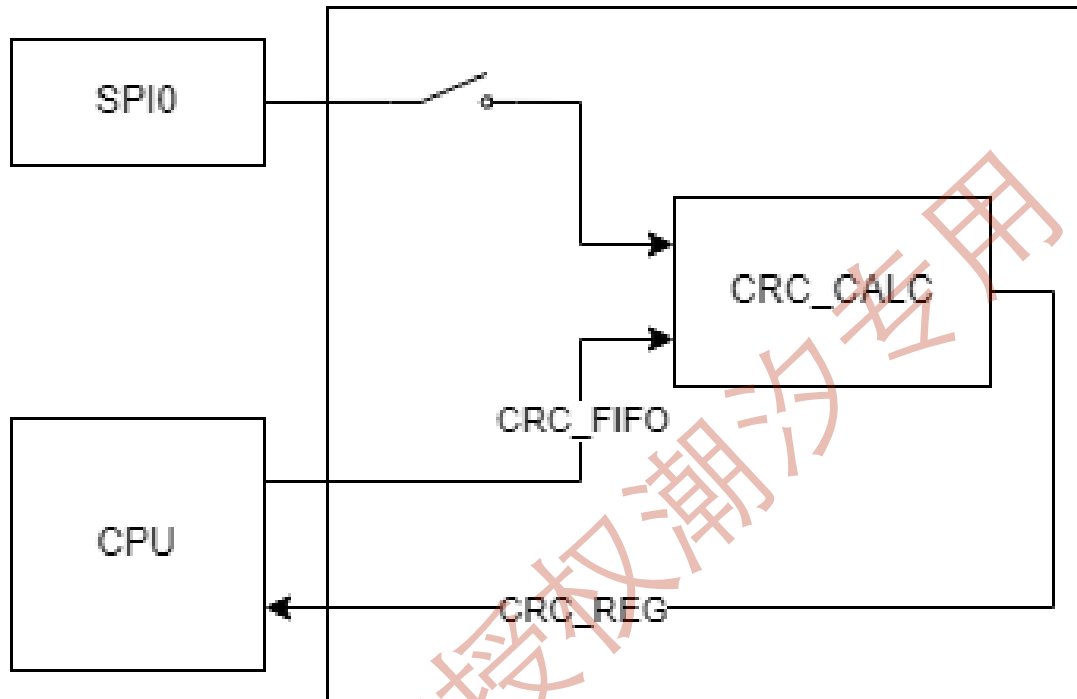


图1、CRC原理框图

当SPI0在DMA状态下计算CRC时，应避免使用CPU操作CRC_FIFO

注意：该项目CRC模块被集成到SPI0中，与SPI0共用基地址

9.2 寄存器说明

9.2.1 REG: CRC16 校验码

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	-	-	预留
15-0	CRC_REG	rw	x	-	写入初始值，CRC计算完毕，读取校验码

9.2.2 FIFO: 运算数据输入

Bit	Name	RW	Default	RV	Description
31-8	RESERVED	-	-	-	预留

Bit	Name	RW	Default	RV	Description
7-0	CRC_FIFO	w	x	-	运算数据输入，HSB先运算，LSB后运算

杰理科技授权潮汐专用

10 GPCNT

10.1 概述

GPCNT为时钟脉冲计数器，用于计算两个时钟周期的比例，即用一个已知时钟（主时钟）计算另一个时钟（次时钟）的周期。

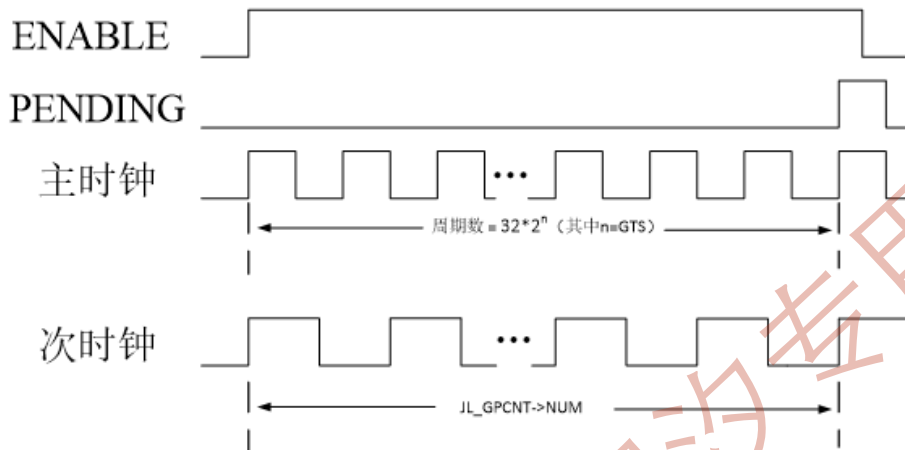


图 1 GPCNT时钟计算示意图

10.2 寄存器说明

10.2.1 JL_GPCNT->CON: GPCNT configuration register

Bit	Name	RW	Default	RV	Description
31	PND	r	0	-	中断请求标志（当JL_GPCNT->CON[0]置1后，主时钟达到JL_GPCNT->CON[11:8]设定的周期数后，PENDING置1，并请求中断）： 0: 无PENDING; 1: 有PENDING
30	CLR_PND	w	x	-	清除中断请求标志位： 0: 无效; 1: 清PENDING
29-20	RESERVED	-	-	-	预留
19-16	GTS	rw	0x0	-	主时钟周期数选择：主时钟周期数 = $32 * 2^n$ （其中n=GTS）
15-13	RESERVED	-	-	-	预留
12-8	GSS	rw	0x0	-	主时钟选择（计数时钟）：参见Clock_System文档
7-6	RESERVED	-	-	-	预留

Bit	Name	RW	Default	RV	Description
5-1	CSS	rw	0x0	-	次时钟选择（被计数时钟）：参见Clock_System 文档
0	ENABLE	rw	0	-	GPCNT模块使能位： 0: 不使能； 1: 使能

【注意】：需配置好其他位，才将ENABLE置1。

10.2.2 JL_GPCNT->NUM: The number of GPCNT clock cycle register

Bit	Name	RW	Default	RV	Description
31-0	NUM	r	0x0	-	在JL_GPCNT->CON[0]置1到 PENDING置1（中断到来）之间，次时钟跑的周期数。

杰理科技授权潮汐专用

11 GPCRC

11.1 概述

GPCRC（General Purpose Cyclic Redundancy Calculation Unit，即通用循环冗余检验计算单元）使用多项式计算一个8位/16位/32位的数据字产生的CRC码。

支持多项式位数：7位、8位、16位、32位。

支持多项式系数编程，或使用默认多项式（CRC-32/MPEG-2）。

支持输入数据位反转、输出数据位反转、输入数据端序翻转。

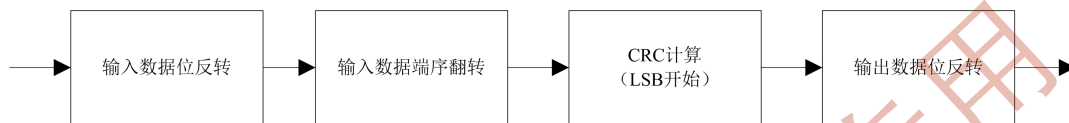


图1-1 GPCRC原理图

11.2 功能描述

11.2.1 数据处理

输入数据端序翻转：

指大小端变换，如数据为{Byte3, Byte2, Byte1, Byte0}，半字翻转为{Byte2, Byte3, Byte0, Byte1}，字翻转为{Byte0, Byte1, Byte 2, Byte3}。

输入数据反转：

指字节按位反转，如数据为{0x01,0x02,0x03,0x04}，反转为{0x80,0x40,0xc0,0x20}。

输出数据反转：

指对应7位、8位、16位、32位多项式按位反转。

11.2.2 多项式配置

多项式名称	多项式公式	REV_IN	REV_OUT	POL	INIT
CRC-7/MMC	x^7+x^3+1	不反转	不反转	0x00000009	0x00000000
CRC-8	$x^8+x^2+x^1+1$	不反转	不反转	0x00000007	0x00000000
CRC-8/ITU	$x^8+x^2+x^1+1$	不反转	不反转	0x00000007	0x00000000
CRC-8/ROHC	$x^8+x^2+x^1+1$	反转	反转	0x00000007	0x000000ff
CRC-8/MAXIM	$x^8+x^5+x^4+1$	反转	反转	0x00000031	0x00000000
CRC-16/IBM	$x^{16}+x^{15}+x^2+1$	反转	反转	0x00008005	0x00000000

多项式名称	多项式公式	REV_IN	REV_OUT	POL	INIT
CRC-16/MAXIM	$x^{16}+x^{15}+x^2+1$	反转	反转	0x00008005	0x00000000
CRC-16/USB	$x^{16}+x^{15}+x^2+1$	反转	反转	0x00008005	0x0000ffff
CRC-16/MODBUS	$x^{16}+x^{15}+x^2+1$	反转	反转	0x00008005	0x0000ffff
CRC-16/CCITT	$x^{16}+x^{12}+x^5+1$	反转	反转	0x00001021	0x00000000
CRC-16/CCITT-FALSE	$x^{16}+x^{12}+x^5+1$	不反转	不反转	0x00001021	0x0000ffff
CRC-16/X25	$x^{16}+x^{12}+x^5+1$	反转	反转	0x00001021	0x0000ffff
CRC-16/XMODEM	$x^{16}+x^{12}+x^5+1$	不反转	不反转	0x00001021	0x00000000
CRC-16/DNP	$x^{16}+x^{13}+x^{12}+x^{11}+x^{10}+x^8+x^6+x^5+x^2+1$	反转	反转	0x00003d65	0x00000000
CRC-32	$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$	反转	反转	0x04c11db7	0xffffffff
CRC-32/MPEG-32	$x^{32}+x^{26}+x^{23}+x^{22}+x^{16}+x^{12}+x^{11}+x^{10}+x^8+x^7+x^5+x^4+x^2+x+1$	不反转	不反转	0x04c11db7	0xffffffff

11.3 数字模块控制寄存器

11.3.1 GPCRC_CON: gpcrc control register

Bit	Name	RW	Default	RV	Description
31:9	RESERVED	-	-	-	预留
8	REV_IN	rw	1	-	输入数据位反转 0: 反转 1: 不反转
7	REV_OUT	rw	0	-	输出数据位反转 0: 不反转 1: 反转

Bit	Name	RW	Default	RV	Description
6:5	SWP_IN	rw	0x0	-	输入数据端序翻转 0/3: 不翻转 1: 半字翻转 2: 字翻转
4:3	POL_SIZE	rw	0x0	-	多项式位数 0: 32位 1: 16位 2: 8位 3: 7位
2:1	DAT_SIZE	rw	0x0	-	输入数据类型 0/3: 32位 1: 16位 2: 8位
0	RESERVED	-	-	-	预留

11.3.2 GPCRC_POL: gpcrc poly register

Bit	Name	RW	Default	RV	Description
31:0	POL	rw	0x04c11db7	-	CRC多项式系数，第0位恒为1

11.3.3 GPCRC_INIT: gpcrc initial register

Bit	Name	RW	Default	RV	Description
31:0	INIT	w	-	-	写入CRC初始数据

11.3.4 GPCRC_REG: gpcrc data register

Bit	Name	RW	Default	RV	Description
31:0	REG	rw	0xffffffff	-	写入CRC数据或读出CRC结果

11.4 DEMO CODE

```
#define gpcrc_con_set(swp_in, rev_out, rev_in, pol_size, dat_size) \
( \
    (rev_in & 0x1) << 8 | \
    (rev_out & 0x1) << 7 | \
    (swp_in & 0x3) << 5 | \
    (pol_in & 0x3) << 3 | \
    (dat_size & 0x3) << 1 | \
)

typedef enum {
    NO_SWP_IN, HWAD_SWP_IN, WORD_SWP_IN
```

```
} SWP_IN_CFG;

typedef enum {
    NO_REV_OUT, REV_OUT
} REV_OUT_CFG;

typedef enum {
    REV_IN, NO_REV_IN
} REV_IN_CFG;

typedef enum {
    POL32, POL16, POL8, POL7
} POL_SIZE;

typedef enum {
    U32, U16, U8
} DAT_SIZE;

u8 crc8_rohc_u8(u8 *data, u32 length)
{
    JL_GPCRC->INIT = 0x000000ff;
    JL_GPCRC->CON = gpcrc_con_set(NO_SWP_IN, REV_OUT, REV_IN, POL8, U8);
    JL_GPCRC->POL = 0x00000007;
    while(length--)
        JL_GPCRC->REG = *data++;
    return JLGPCRC->REG;
}

u8 crc8_rohc_u16(u16 *data, u32 length)
{
    JL_GPCRC->INIT = 0x000000ff;
    JL_GPCRC->CON = gpcrc_con_set(NO_SWP_IN, REV_OUT, REV_IN, POL8, U16);
    JL_GPCRC->POL = 0x00000007;
    while(length--)
        JL_GPCRC->REG = *data++;
    return JLGPCRC->REG;
}

u8 crc8_rohc_u32(u32 *data, u32 length)
{
    JL_GPCRC->INIT = 0x000000ff;
    JL_GPCRC->CON = gpcrc_con_set(NO_SWP_IN, REV_OUT, REV_IN, POL8, U32);
    JL_GPCRC->POL = 0x00000007;
    while(length--)
        JL_GPCRC->REG = *data++;
    return JLGPCRC->REG;
}
```

12 IIC

12.1 概述

IIC接口是一个可兼容大部分IIC总线标准的串行通讯接口。在上面传输的数据以Byte（8bit）为最小单位，且永远是MSB在前。

- IIC接口主要支持特性：(1). 主模式和从模式，不支持多机功能；(2). 支持标准速度模式（Standard-mode, Sm, 高达100kHz），快速模式（Fast-mode, Fm, 高达400kHz），快速增强模式（Fast-mode Plus, Fm+, 高达1MHz）；(3). 7bit主从机寻址模式，可配置从机地址应答，广播地址响应；(4). 总线时钟延展功能可配置；(5). 总线数据建立时间和保持时间可配置；(6). 基于便捷的事务操作驱动总线；(7). 数据接收发送各独立具有1字节缓冲；(8). 可配置总线信号数字滤波器；
- 主机：(1). IIC接口SCL时钟由本机产生，提供给片外IIC设备使用；(2). IIC接口SCL时钟可配置，SCL时钟周期为：

$$TIIC_BUS_SCL = TIIC_clk / (2 * BUAD_CNT) + T_{电阻上拉}$$

- 从机：(1). IIC接口SCL时钟由片外IIC设备产生，经内部IIC_clk滤波采样后给本机使用；(2). IIC总线上每一个start/restart位后接从机地址，此时当外部IIC设备所发的地址与我们匹配时（开启广播地址响应，在接收到广播地址时也会响应），可配置硬件自动回应（ack位为“0”）；

12.2 特殊注意点

时钟要求：作为主机时，BAUD、TSU、THD、FLT_SEL值设置应当满足以下关系

$$BAUD_CNT > (SETUP_CNT + HOLD_CNT + FLT_SEL + 5)$$

作为从机时，SCL时钟低电平时间与TSU、THD、FLT_SEL值设置应当满足以下关系

$$TSCL_L > (SETUP_CNT + HOLD_CNT + FLT_SEL + 5) * TI2C_clk$$

IO设置：在选择IO作为IIC接口的SCL、SDA输入输出时，必须配置该两个IO的SPL寄存器位为1；

事务寄存器：写入事务后需等待硬件将写入的事务加载后（tx_task_load/rx_task_load置位即表示对应事务被硬件加载，task_done pnd置位即表示事务已被硬件加载且执行完成）才可写入新的事务；

【注意】：IIC的通信频率不仅与所配置的波特率有关，还会受到IIC总线上拉时间的影响，请尽可能符合IIC总线硬件电气要求。

12.3 数字模块控制寄存器

12.3.1 IIC_CON0: iic control register 0

Bit	Name	RW	Default	RV	Description
31-10	RESERVED	-	-	-	预留
10	BADDR_RESP_EN	rw	0	-	广播地址响应使能： 1：响应总线上的广播地址； 0：不响应总线上的广播地址

Bit	Name	RW	Default	RV	Description
9	AUTO_SLV_TX	rw	0	-	从机自动发送数据事务使能（主机模式下无效）： 0：不使能； 1：使能； 从机接收匹配的地址且为从机接收数据时，接下来硬件会自动填充SEND_DATA事务操作，使得硬件能不间断的发送数据； （作为从机与不支持时钟延展的主机通信时必须使能，其他工作场景依据需求使能）
8	AUTO_SLV_RX	rw	0	-	从机自动接收数据事务使能（主机模式下无效）： 0：不使能； 1：使能 从机接收匹配的地址且为从机接收数据时，接下来硬件会自动填充RECV_DATA事务操作，使得在软件读取数据时，硬件接口能继续不间断接收数据； （作为从机与不支持时钟延展的主机通信时必须使能，其他工作场景依据需求使能）
7	AUTO_ADR_RESP	rw	0	-	从机接收地址硬件自动响应使能： 作为从机时，在接收到地址数据后（接收的第一字节数据） 0：不自动进行响应，即不对接收到的地址数据进行NACK/ACK，需cpu读取接收数据后，再执行NACK/ACK事务； 1：自动进行响应，接收到匹配的地址（含广播地址）自动响应ACK；接收到不匹配的地址，自动响应NACK （从机与不支持时钟延展的主机通信时必须使能，其他工作场景依据需求使能）
6	IGNORE_NACK	rw	0	-	NACK调试模式使能 0：在发送数据中遇到NACK后会停止接收发送； 1：在发送数据中遇到NACK后不会中断接收发送事务（仅用于调试）；
5	NO_STRETCH	rw	0	-	IIC主从时钟延展功能选择 0：支持时钟延展； 1：不支持时钟延展
4-3	FLT_SEL	rw	0	-	IIC接口数字滤波器选择， 无特殊情况软件配置值为2'b10 2'b11： 滤除3 * Tiic_baud_clk 以下尖峰脉宽； 2'b10： 滤除2 * Tiic_baud_clk 以下尖峰脉宽； 2'b01：滤除Tiic_baud_clk 以下尖峰脉宽； 2'b00：关闭数字滤波器
2	SLAVE MODE	rw	0	-	IIC接口主从机模式： 0：主机模式； 1：从机模式

Bit	Name	RW	Default	RV	Description
1	I2C_RST	rw	0	-	IIC接口复位（使能后再释放复位） 0：IIC接口复位； 1：IIC接口释放
0	EN	rw	0	-	IIC接口使能。 0：关闭IIC接口 1：打开IIC接口

12.3.2 IIC_TASK: iic task register

Bit	Name	RW	Default	RV	Description
31-4	RESERVED	-	-	-	预留
12	SLAVE_CALL	r	-	-	从机地址匹配（仅debug，用户正常流程不可使用）
11	BC_CALL	r	-	-	广播地址匹配（仅debug，用户正常流程不可使用）
10	SLAVE_RW	r	-	-	从机读写状态（仅debug，用户正常流程不可使用）
9-6	RUNNING_TASK	r	-	-	正在运行事务（仅debug，用户正常流程不可使用）
5	TASK_RUN_RDY	r	-	-	事务运行状态（仅debug，用户正常流程不可使用）
4	TASK_LOAD_RDY	r	-	-	事务加载状态（仅debug，用户正常流程不可使用）
3-0	RUN_TASK	w	0x0	-	事务操作：以下9种事务操作涵盖了主从机接收发送时的总线行为序列（具体解析见下节） 0x0: SEND_RESET 0x1: SEND_ADDR 0x2: SEND_DATA 0x3: SEND_ACK 0x4: SEND_NACK 0x5: SEND_STOP 0x6: SEND_NACK_STOP 0x7: RECV_DATA 0x8: RECV_DATA_WITH_ACK 0x9: RECV_DATA_WITH_NACK

12.3.3 IIC_PND: iic pending register

Bit	Name	RW	Default	RV	Description
31-29	RESERVED	-	-	-	预留
28	RXTASK_LOAD_PND	r	0x0	-	硬件将当前的接收相关的task事务加载完成
27	TXTASK_LOAD_PND	r	0x0	-	硬件将当前的发送相关的task事务加载完成

Bit	Name	RW	Default	RV	Description
26	RXBYTE_DONE_PND	r	0x0	-	接收到1byte的数据
25	ADR_MATCH_PND	r	0x0	-	作为从机接收到与ADDR相符的地址 (开始广播地址响应使能后, 匹配广播地址也会起该PND)
24	RXNACK_PND	r	0x0	-	发送地址/数据后接收到NACK
23	RXACK_PND	r	0x0	-	发送地址/数据后接收到ACK
22	STOP_PND	r	0x0	-	接收到总线STOP信号序列
21	RESTART_PND	r	0x0	-	接收到总线RESTART信号序列
20	TASK_PND	r	0x0	-	事务执行完成PND
19	RESERVED	-	-	-	预留
18	RXTASK_LOAD_CLR	w	-	-	写“1”清除对应的PND
17	TXTASK_LOAD_CLR	w	-	-	写“1”清除对应的PND
16	RXDATA_DONE_CLR	w	-	-	写“1”清除对应的PND
15	ADR_MATCH_CLR	w	-	-	写“1”清除对应的PND
14	RXNACK_CLR	w	-	-	写“1”清除对应的PND
13	RXACK_CLR	w	-	-	写“1”清除对应的PND
12	STOP_CLR	w	-	-	写“1”清除对应的PND
11	RESTART_CLR	w	-	-	写“1”清除对应的PND
10	TASK_CLR	w	-	-	写“1”清除对应的PND
9	RESERVED	-	-	-	预留
8	RXTASK_LOAD_IE	rw	0		对应PND的中断使能
7	TXTASK_LOAD_IE	rw	0		对应PND的中断使能
6	RXDATA_DONE_IE	rw	0		对应PND的中断使能
5	ADR_MATCH_IE	rw	0		对应PND的中断使能
4	RXNACK_IE	rw	0		对应PND的中断使能
3	RXACK_IE	rw	0	-	对应PND的中断使能
2	STOP_IE	rw	0	-	对应PND的中断使能
1	RESTART_IE	rw	0	-	对应PND的中断使能

Bit	Name	RW	Default	RV	Description
0	TASK_IE	rw	0	-	对应PND的中断使能

12.3.4 IIC_TXBUF: iic tx buff register

Bit	Name	RW	Default	RV	Description
31-8	RESERVED	-	-	-	-
7-0	TX_BUFF	rw	-	-	待发送数据

12.3.5 IIC_RXBUF: iic rx buff register

Bit	Name	RW	Default	RV	Description
31-8	RESERVED	-	-	-	-
7-0	RX_BUFF	r	-	-	已接收数据

12.3.6 IIC_ADDR: iic device address register

Bit	Name	RW	Default	RV	Description
31-8	RESERVED	-	-	-	-
7-1	IIC DEVICE ADDR	rw	0x00	-	IIC 设备地址（仅支持7bit模式）
0	W/R OPTION	rw	0x0	-	作为主机发送地址时的读写标志： 0：发送数据到从机； 1：读取从机的数据；

12.3.7 IIC_BAUD: iic baud clk register

Bit	Name	RW	Default	RV	Description
31-12	RESERVED	-	-	-	-
11-0	BAUD_CNT	rw	-	-	IIC 总线传输时钟SCL速率配置： $FIIC_BUS_SCL = FIIC_clk / BUAD_CNT$ 注意：不可设置为0

12.3.8 IIC_TSU: iic setup time register

Bit	Name	RW	Default	RV	Description
31-7	RESERVED	-	-	-	-

Bit	Name	RW	Default	RV	Description
5-0	SETUP_CNT	rw	-	-	IIC 总线SDA信号更新后到SCL时钟上升的最少时间配置： $T_{setup} = T_{IIC_clk} * SETUP_CNT$ 无特殊情况软件配置值为2；

12.3.9 IIC_THD: iic hold time register

Bit	Name	RW	Default	RV	Description
31-7	RESERVED	-	-	-	-
5-0	HOLD_CNT	rw	-	-	IIC 总线SCL时钟下降到SDA信号更新前的最少时间配置： $T_{hold} = T_{IIC_clk} * HOLD_CNT$ 无特殊情况软件配置值为2；

12.3.10 IIC_DBG: iic debug register

Bit	Name	RW	Default	RV	Description
31-16	SDA_DBG	r	-	-	SDA相关调试信号
15-0	SCL_DBG	r	-	-	SCL相关调试信号

12.4 基本事务操作

IIC_TASK 事务寄存器允许写入以下9种事务操作值来驱动IIC接口做出相应总线行为序列：

1. 0x0: SEND_RESET

工作在主机模式下，向IIC总线发送START 和 STOP；

2. 0x1: SEND_ADDR

工作在主机模式下，向IIC总线发送START 和 ADDR的数据，并接收从机回复的ACK/NACK；

3. 0x2: SEND_DATA

工作在主机模式下，向IIC总线发送TX BUFF的数据，并接收从机回复的ACK/NACK；

4. 0x3: SEND_ACK

工作在主/从机模式下，向IIC总线发送ACK；

5. 0x4: SEND_NACK

工作在主/从机模式下，向IIC总线发送NACK；

6. 0x5: SEND_STOP

工作在主机模式下，向IIC总线发送STOP；

7. 0x6: SEND_NACK_STOP

工作在主机模式下，向IIC总线发送NACK和STOP；

8. 0x7: RECV_DATA

工作在主机/从机模式下，从IIC总线上接收一个字节数据，但不进行ACK/NACK（持续拉低SCL线，总线上的设备需支持时钟延展），直到SEND_ACK/SEND_NACK的事务被填充，SCL才会撤销拉低；

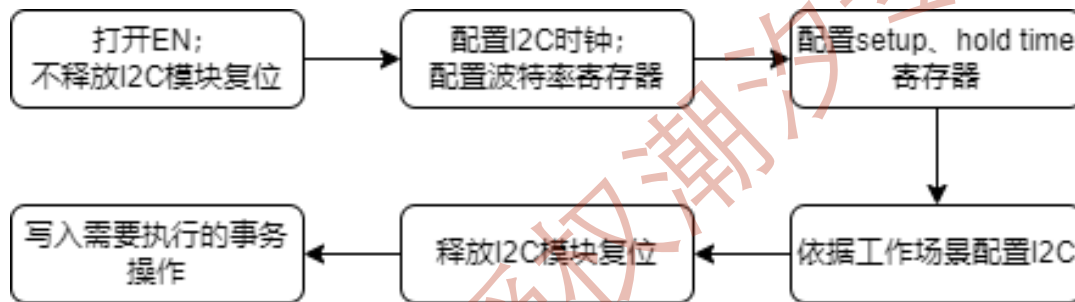
9. 0x8: RECV_DATA_WITH_ACK

工作在主机/从机模式下，从IIC总线上接收一个字节数据，且随后向IIC总线发送ACK；

10. 0x9: RECV_DATA_WITH_NACK

工作在主机/从机模式下，从IIC总线上接收一个字节数据，且随后向IIC总线发送NACK；

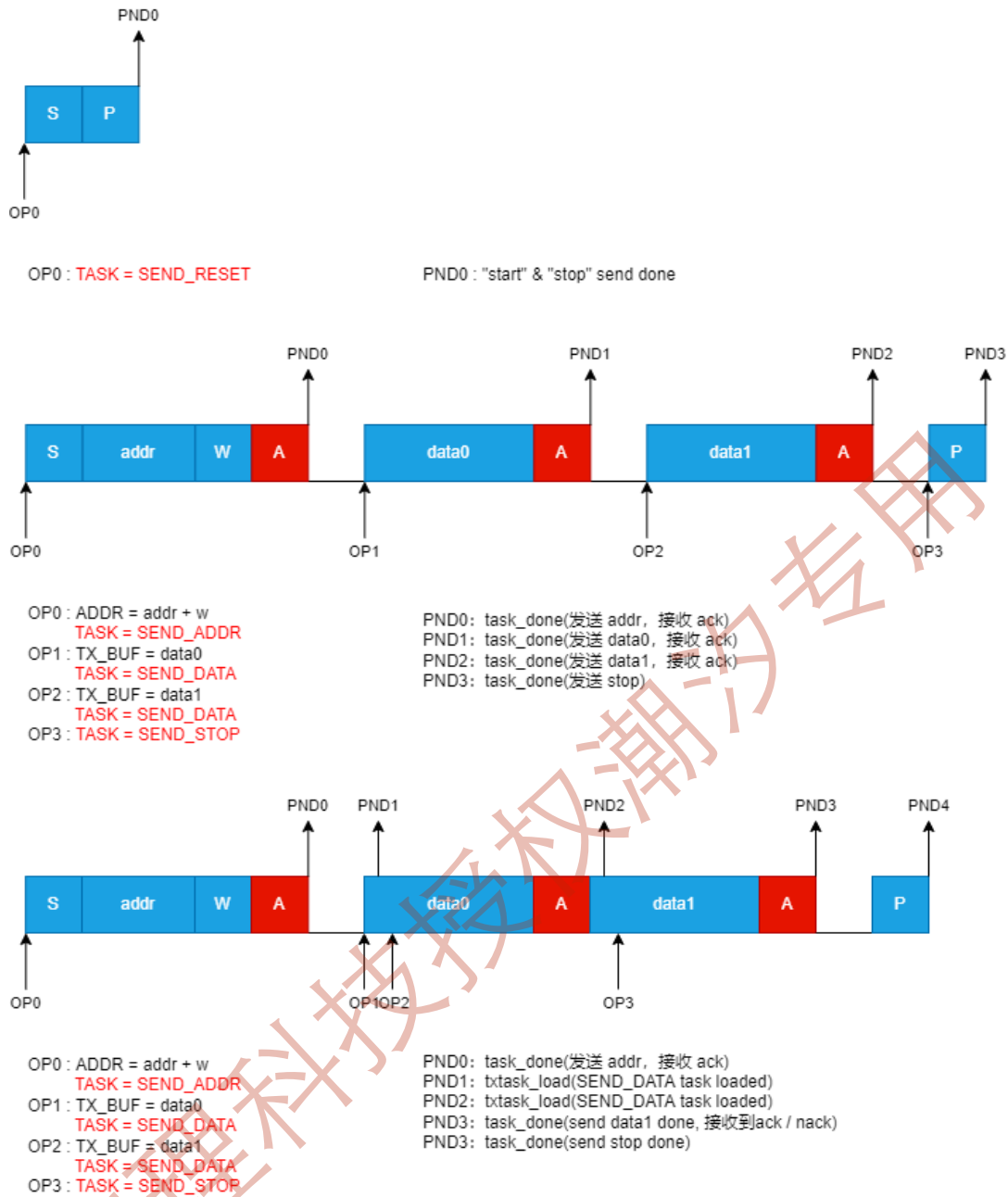
12.5 基本工作场景使用流程



1. 主机发送

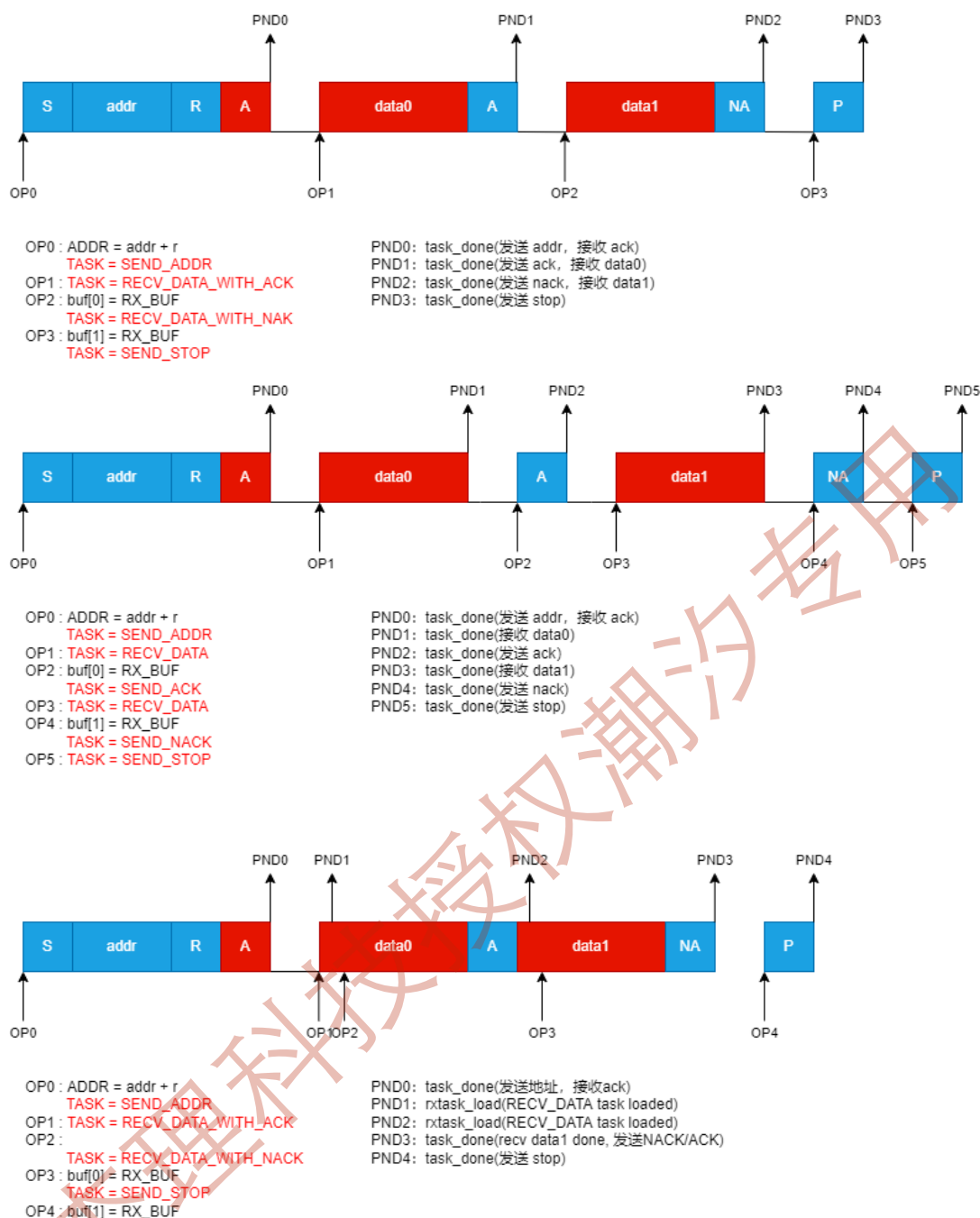
主机发送具有字节断续和字节连续发送两种工作场景。

字节连续发送：用于传输数据量较大的（中途无需进行数据处理）场景，主机在发送地址并且匹配后通过txtask_pnd来预先（提前约8个I2C_SCL时钟周期）填充下一事务达到连续发送。

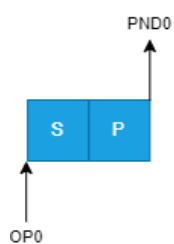


2. 主机接收

与主机发送相似，主机发送同样具有字节断续和字节连续发送两种工作场景；

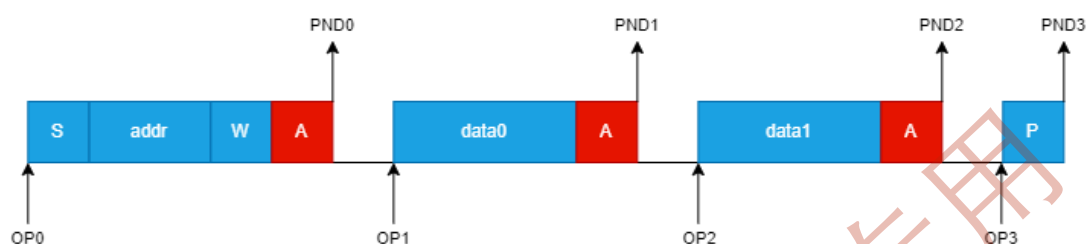


3. 主机其他操作（用于复位总线上其他从设备）



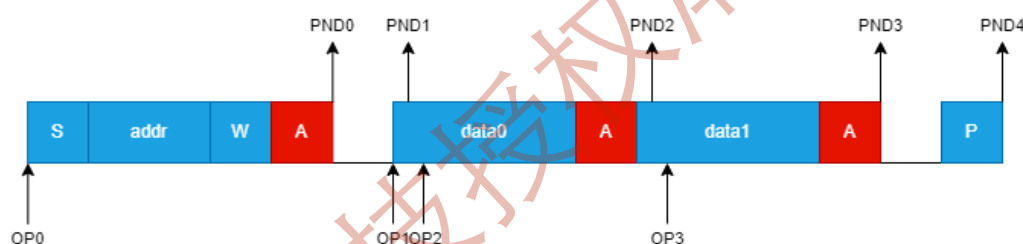
OP0: TASK = SEND_RESET

PND0: "start" & "stop" send done



OP0: ADDR = addr + w
TASK = SEND_ADDR
OP1: TX_BUF = data0
TASK = SEND_DATA
OP2: TX_BUF = data1
TASK = SEND_DATA
OP3: TASK = SEND_STOP

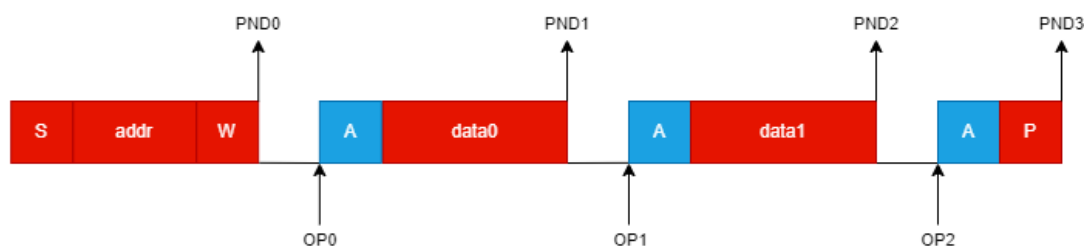
PND0: task_done(发送 addr, 接收 ack)
PND1: task_done(发送 data0, 接收 ack)
PND2: task_done(发送 data1, 接收 ack)
PND3: task_done(发送 stop)



OP0: ADDR = addr + w
TASK = SEND_ADDR
OP1: TX_BUF = data0
TASK = SEND_DATA
OP2: TX_BUF = data1
TASK = SEND_DATA
OP3: TASK = SEND_STOP

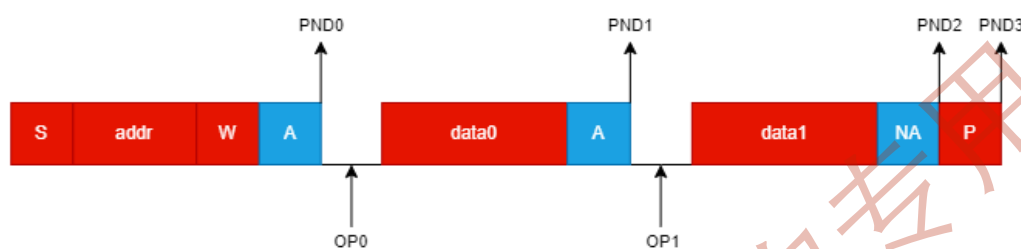
PND0: task_done(发送 addr, 接收 ack)
PND1: txtask_load(SEND_DATA task loaded)
PND2: txtask_load(SEND_DATA task loaded)
PND3: task_done(send data1 done, 接收到ack / nack)
PND3: task_done(send stop done)

4. 从机接收



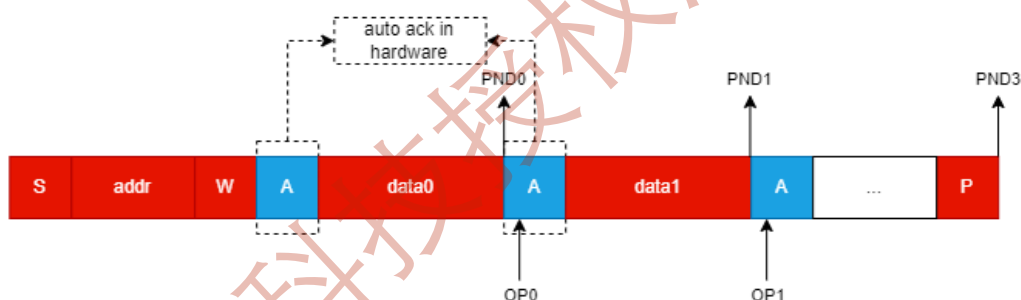
STRETCH_EN = 1
AUTO_ADR_RESP = 0

OP0 : TASK = RECV_DATA_WITH_ACK
OP1 : TASK = RECV_DATA_WITH_ACK
OP2 : TASK = RECV_DATA_WITH_ACK / SEND_NACK
PND0 : task_done("addr" match)
PND1 : task_done("data0" recieve done)
PND2 : task_done("data1" recieve done)
PND3 : task_done("stop" recieve)



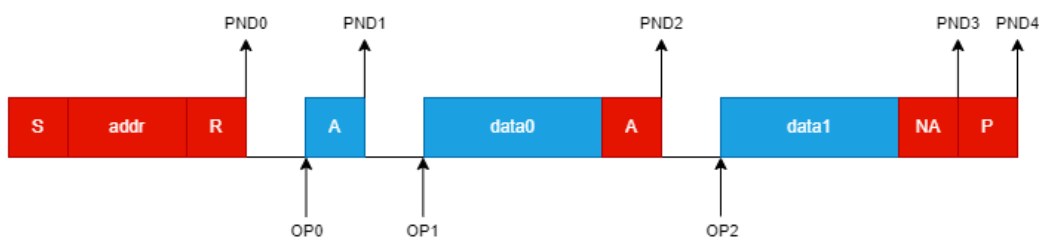
STRETCH_EN = 1
AUTO_ADR_RESP = 1

OP0 : TASK = RECV_DATA_WITH_ACK
OP1 : TASK = RECV_DATA_WITH_ACK / RECV_DATA_WITH_NACK
PND0 : task_done("addr" match)
PND1 : task_done("data0" recieve done)
PND2 : task_done("data1" recieve done)
PND3 : task_done("stop" recieve)



STRETCH_EN = 0
AUTO_ADR_RESP = 1
AUTO_SLV_RX = 1
OP0 : TASK = SEND_ACK(for data1)
OP1 : TASK = SEND_ACK(for data2)
...
PND0 : rxbyte_done("addr" match, "data0" recieve)
PND1 : rxdata_done("data1" recieve)
...
PND3 : stop("stop" recieve)

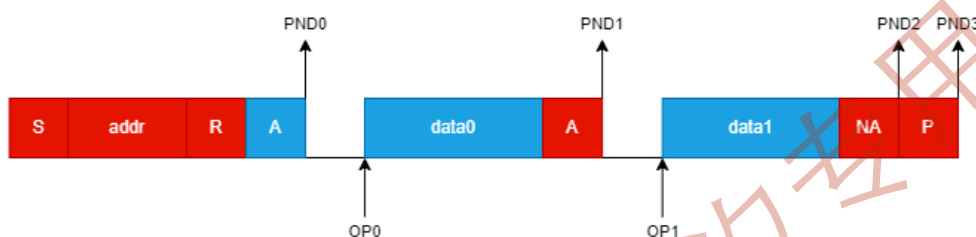
5. 从机发送



STRETCH_EN = 1
AUTO_ADR_RESP = 0

OP0 : TASK = SEND_ACK
OP1 : TX_BUF = data0
TASK = SEND_DATA
OP2 : TX_BUF = data1
TASK = SEND_DATA

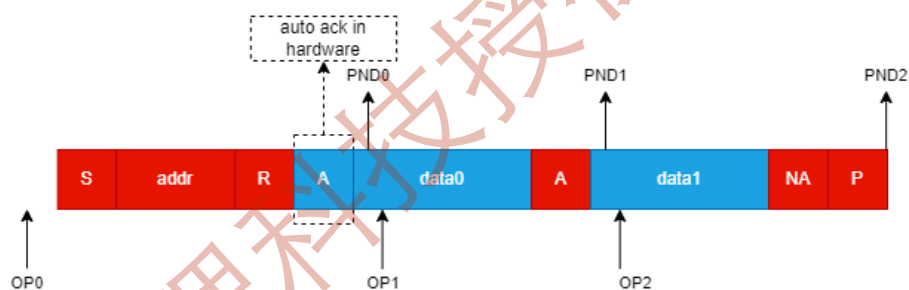
PND0 : task_done("addr" match)
PND1 : task_done("ack" send done)
PND2 : task_done("data0" send done, "ack" receive)
PND3 : task_done("data1" send done, "nack" receive)
PND4 : task_done("stop" receive)



STRETCH_EN = 1
AUTO_ADR_RESP = 1

OP0 : TX_BUF = data0
TASK = SEND_DATA
OP1 : TX_BUF = data1
TASK = SEND_DATA

PND0 : task_done("addr" match)
PND1 : task_done("data0" send done, "ack" receive)
PND2 : task_done("data1" send done, "nack" receive)
PND3 : task_done("stop" receive)



STRETCH_EN = 0
AUTO_ADR_RESP = 1
AUTO_SLV_TX = 1

OP0 : TX_BUF = data0
OP1 : TX_BUF = data1
OP2 : TX_BUF = data2

PND0 : tx load done("addr" match, "data0" tx load done)
PND1 : tx load done("data1" tx load done, "ack" receive)
PND2 : stop ("nack" receive, "stop" receive)

13 MCPWM

13.1 概述

MCPWM功能模块包括：n个timer时基模块，n对独立的PWM通道，n路故障保护输入。框图如下：

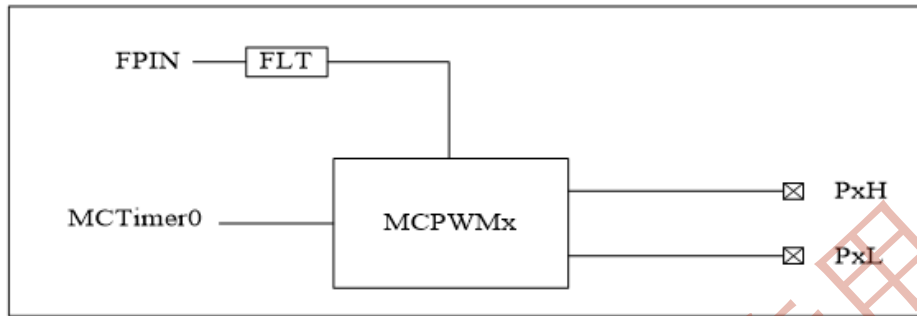


图1-1 MCPWM结构图

13.2 定时器 MCTIMER

1. 概述

mctimer是16位定时器，可作为pwm的时基控制

- 模块特性 (1). 16位定时功能 (2). 带缓冲的周期寄存器 (3). 支持多种工作模式：递增、递增-递减、外部引脚控制递增或递减 (4). 多种中断模式：上溢出中断、下溢出中断、上下溢出中断

3. 注意：MCPWM增加了一BIT (MCPWM_CON BIT (16))用于开关MCPWM时钟，防止MCPWM模块不用时还有漏电；使用MCPWM之前需要打开这一BIT。

13.3 TIMER模块寄存器

13.3.1 TMRx_CON: Timer contrl register

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	-	-	预留
15	INCF	rw	0	-	递增递减标志位 0: 递减 1: 递增
13	UFPND	r	0	-	递减借位标志 0: 没借位 1: 已发生借位
12	OFPN	r	0	-	计数溢出 (TMRXCNT == TMRXPR) 标志 0: 没溢出 1: 已发生溢出

Bit	Name	RW	Default	RV	Description
11	UFCLR	w	-	1	清除UFPND标志位 0: 无效 1: 清除UFPND
10	OFCLR	w	-	1	清除OFPND标志位 0: 无效 1: 清除OFPND
9	UFIE	rw	0	1	定时递减借位中断使能 0: 禁止 1: 允许
8	OFIE	rw	0	1	计数溢出中断使能 0: 禁止 1: 允许
7	CKSRC	rw	0	-	定时器时钟源 0: 内部时钟 1: 外部引脚时钟
6-3	CKPS	rw	0	-	时钟预分频设置, $TCK/(2^{CKPS})$
2	RESERVED	-	-	-	预留
1-0	MODE	rw	0	-	定时器工作模式 00: 保持计数值 01: 递增模式 10: 递增-递减循环模式 11: 递减模式

13.3.2 TMRx_CNT: counter initial value register

Bit	Name	RW	Default	RV	Description
15-0	TMR_CNT	rw	x	-	计数/定时初值

13.3.3 TMR_PR: counter target value register

Bit	Name	RW	Default	RV	Description
15-0	TMR_PR	rw	x	-	计数/定值目标值

13.4 模块引脚

PWM0: PWMCH0H、PWMCH0L

PWM1: PWMCH1H、PWMCH1L

PWM2: PWMCH2H、PWMCH2L

.....

故障输入引脚：FPIN

13.5 模块特性

1. 边沿对齐和中心对齐输出模式
2. 可运行过程中更改PWM频率、占空比
3. 多种更新频率/占空比模式：上溢出重载、下溢出重载、上下溢出重载
4. 灵活配置每对通道的有效电平状态
5. 可编程死区控制
6. 硬件故障输入引脚

13.6 PWM模块控制寄存器

13.6.1 CHx_CON0: pwm control register0

Bit	Name	RW	Default	RV	Description
15-12	DTCKPS	rw	0	-	死区时钟预分频, $T_{sys}/(2^{\wedge}DTCKPS)$
11-7	DTPR	rw	0	-	死区时间控制 死区时间: $T_{sys}/(2^{\wedge}DTCKPS) \times (DTPR+1)$
6	DTEN	rw	0	-	死区允许控制, 对应PWMCHxH、PWMCHxL 0: 禁止 1: 允许
5	L_INV	rw	0	-	对应PWMCHXL输出反向控制 0: 反向禁止 1: 反向允许
4	H_INV	rw	0	-	对应PWMCHXH输出反向控制 0: 反向禁止 1: 反向允许
3	L_EN	rw	0	-	对应PWMCHXL输出允许控制 0: 禁止PWMCHXL 1: 允许PWMCHXL
2	H_EN	rw	0	-	对应PWMCHxH输出允许控制 0: 禁止PWMCHxH 1: 允许PWMCHxH
1-0	CMP_LD	rw	0	-	CHx_CMP重新载入控制 00: 时基TMRX_CNT等于“0”载入 01: 时基TMRX_CNT等于“0”或者等于TMRX_PR时候载入 10: 时基TMRX_CNT等于TMRX_PR时候载入 11: 立即载入

13.6.2 CHX_CON1: pwm control register1

Bit	Name	RW	Default	RV	Description
15	FPND	r	0	-	故障保护输入标志，只读，写无效 读0：未发生保护 读1：已发生保护，模块的PWM引脚会变成高阻态
14	FCLR	w	-	-	清除FPND标志位，只写，读为“0” 写0：无效 写1：清除FPND
13-12	RESERVED	-	-	-	预留
11	INTEN	rw	0	1	FPND中断允许 0：禁止 1：允许
10-8	TMRSEL	rw	0	-	选择TMR0-7作为PWM时基 0-7：选择时基
7-5	RESERVED	-	-	-	预留
4	FPINEN	rw	0	1	故障保护输入允许控制 0：禁止保护 1：允许保护
3	FPINAUTO	rw	0	1	故障自动保护控制 0：禁止自动保护 1：允许自动保护， 当检测到故障引脚有效信号时， 自动把PWM引脚设成高阻态，直到软件清除FPND。
2-0	FPINSEL	rw	0	-	故障保护输入引脚选择 0-1：选择FPIN作为故障保护输入 (0:选择FPIN0 1:选择FPIN1)

13.6.3 CHX_CMPH: pwm high port compare register

Bit	Name	RW	Default	RV	Description
15-0	MMRX_PRH	rw	-	-	带缓冲的16位比较寄存器，对应PWMCHXH引脚的占空比控制

13.6.4 CHX_CMPL: pwm low port compare register

Bit	Name	RW	Default	RV	Description
15-0	TMRX_PRL	rw	-	-	带缓冲的16位比较寄存器，对应PWMCHXL引脚的占空比控制

13.6.5 FPIN_CON: input filter control register

Bit	Name	RW	Default	RV	Description
23-16	EDGE	rw	0	-	FPINx边沿选择 0: 下降沿 1: 上升沿
15-8	FLT_EN	rw	0	-	FLT_EN: FPINx滤波使能开关 0: 滤波关闭 1: 滤波开启
7-6	RESERVED	-	-	-	预留
5-0	FLT_PR	rw	0	-	FLT_PR: 滤波宽度选择 滤波宽度=16 × FLT_PR × lsb_clk

13.6.6 MCPWM_CON: mcpwm control register

Bit	Name	RW	Default	RV	Description
16	CLK_EN	rw	0	-	mcpwm 模块时钟使能， 配置mcpwm模块之前先配该位为1
15-8	TMR_EN	rw	0	-	定时器计数开关控制(tmr7-0) 0: 定时器计数关闭 1: 定时器计数打开
7-0	PWM_EN	rw	0	-	模块开关控制 (pwm7-0) 0: 模块关闭 1: 模块开启

14 PULSE COUNTER

14.1 概述

以下为pulse_counter/触摸键的寄存器说明及使用方法。其工作原理如下所示：

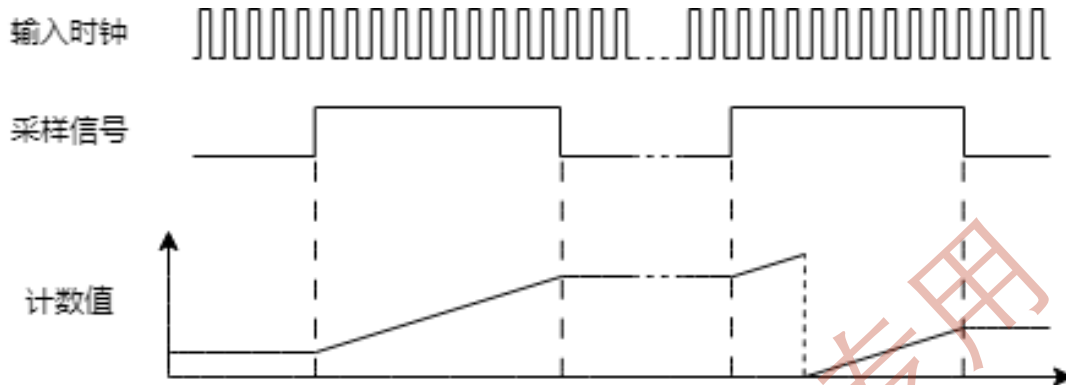


图1、pulse counter计数原理

14.2 寄存器说明

14.2.1 PL_CNT_CON

Bit	Name	RW	Default	RV	Description
31-5	RESERVED	-	-	-	预留
4	CAP_MUX_EDGE	rw	0	-	输入脉冲极性反转使能 0: 输入脉冲不反转 1: 输入脉冲极性反转
3-2	CLK_SEL	rw	0x0	-	pulse counter 时钟选择 00: 选择std48m作为计数时钟; 01: 选择ich_clk_pin作为计数时钟, 具体可参考IOMC; 10: 选择pll_d1p5_mo作为计数时钟; 11: 选择pll_d1p0_mo作为计数时钟; 备注: 时钟选择的频率越大, pl_cnt_value计数值会越大, 分辨率会越高, 触摸键的灵敏度也越高
1	EN	rw	0	-	触摸键使能, 当cap_mux_in为1时, PLCNTVL累加计数, 计满后归0再重新累加 0: 触摸键禁止; 1: 触摸键使能
0	TEST_EN	rw	0	0	触摸键测试使能 (此位专用于测试) 0: 测试模式未使能; 1: 测试模式使能;

【注意】：当cap_mux_in选择TMRx_PWMOUT, pulse counter时钟选择clk_mux_in时, 可以用内部固

定周期的PWM，计算芯片IO上不确定的时钟频率。(具体cap_mux选择参考IOMC文档)

14.2.2 PL_CNT_VAL

Bit	Name	RW	Default	RV	Description
31-0	VALUE	r	-	-	32位counter递增计数值，默认值为随机数

【注意】：芯片上电时Pluse Counter的PL_CNT_VAL都会产生一个随机的初始值，因此每次启动Pluse Counter前都需要读取一次

14.3 示例代码

以下以PA1为例：

1. 变量定义：

```
int Touchkey_value_old, Touchkey_value_new, Touchkey_value_delta;    //32bit
```

2. 选择检测管脚

```
JL_IMAP->FI_GP_ICH1 = PA1_IN; // PA1选通为ICH1
JL_IOMC->ICH_CONx = (ICH1)<<8; //PLUSE COUNTER 选择使用ICH1作为输入
```

3. 初始化计数器配置

```
JL_PCNT->CON = 0;    //
JL_PCNT->CON |= (PCCON_CLOCK_3)<<2;    //选择触摸键时钟
JL_PCNT->CON |= BIT(1);    //使能计数器
```

4. 检测电容

```
Touchkey_value_old = JL_PCNT->VAL;    //读取旧值
JL_PORTA->DIR |= BIT(1);    //对应管脚输入使能
While(PORTA_IN & BIT(1));
Touchkey_value_new = JL_PCNT->VAL;
// 判断是否溢出
if(Touchkey_value_old > Touchkey_value_new)
Touchkey_value_new += 0x10000;
Touchkey_value_delta = Touchkey_value_new - Touchkey_value_old;
```

15 PWM LED

15.1 概述

PWM LED是通过脉冲宽度调节控制LED的亮度与亮灭的一个模块。放在DVDD电源域，不可以关在软关机下工作。

PWM LED模块可以配置两个周期大小不同的PWM0、PWM1，分别控制LED的亮度（PWM0）与亮灭（PWM1）。

15.2 特性

1. 驱动档位特性：

1.1 PWM推高电平: 高阻，上拉，输出1强驱0，输出1强驱1，输出1强驱2，输出1强驱3，保持，输出1强驱2，输出1强驱1，输出1强驱0，上拉，高阻，保持（循环以上步骤）

1.2 PWM推低电平: 高阻，下拉，输出0强驱0，输出0强驱1，输出0强驱2，输出0强驱3，保持，输出0强驱2，输出0强驱1，输出0强驱0，下拉，高阻，保持（循环以上步骤）

1.3 最高驱动档位可设，强驱及高阻保持时间可设，驱动切换等待时间可设（所有等待时间为同样的n个PWM0_CLK）

2. 推灯功能：

2.1 固定亮度

A. （单LED）单闪；双闪（双闪的两个时间长度可调，两个间隔可调）；

B. （双LED）单闪 变色 单闪；

C. 双闪 变色 双闪（双闪的两个时间长度可调，两个间隔可调）；

D. n闪 变色 n闪（n<7）；

2.2 呼吸灯

A. （单LED）灭灯时间长度可调，亮灯时间长度可调，亮灯快慢可调，灭灯快慢可调；

B. （双LED）（灭灯时间长度可调，亮灯快慢可调，灭灯快慢可调）呼吸 变色 呼吸；

无灯—A灯逐渐变到最亮—最亮变色—B灯由最亮逐渐变到最暗—变色（无灯）—A灯逐渐变到最亮...；

无灯—A灯呼吸—无灯—A灯逐渐变到最亮—最亮变色—B灯由最亮逐渐变到最暗—无灯—B灯呼吸—变色（无灯）—A灯呼吸...；

3. 推灯模式

3.1 单IO双LED模式：

单IO双LED模式的典型原理图如下所示。在GPIO为高阻时，IOVDD的电压，不满足两个串联LED的压降，A、B灯均不点亮；IO输出高电平（IOVDD电平），A灯不亮，B灯被点亮；IO输出高电平（IOVDD电平），A灯被点亮，B灯不亮；当A、B灯交替点亮的频率大于人眼分辨极限，即可被认为两灯同时被点亮。

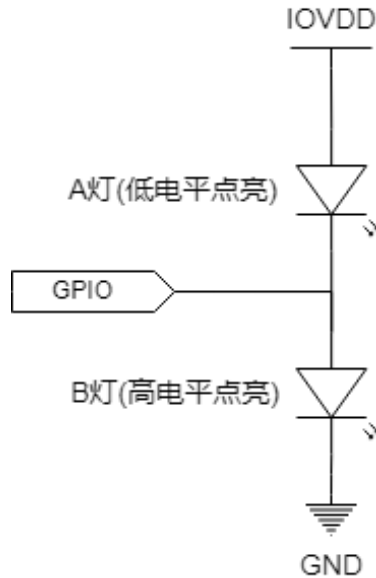


图 1-1 单IO控制双LED原理图

3.2 双IO双LED模式：

双IO双LED模式的典型原理图如下图所示。A、B灯可以以任意一种方式接入到SoC的IO中，包括但不限于图中所展示的类型。GPIO_0和GPIO_1都允许根据用户需要单独设置其驱动电平类型（高电平驱动或者低电平驱动）。

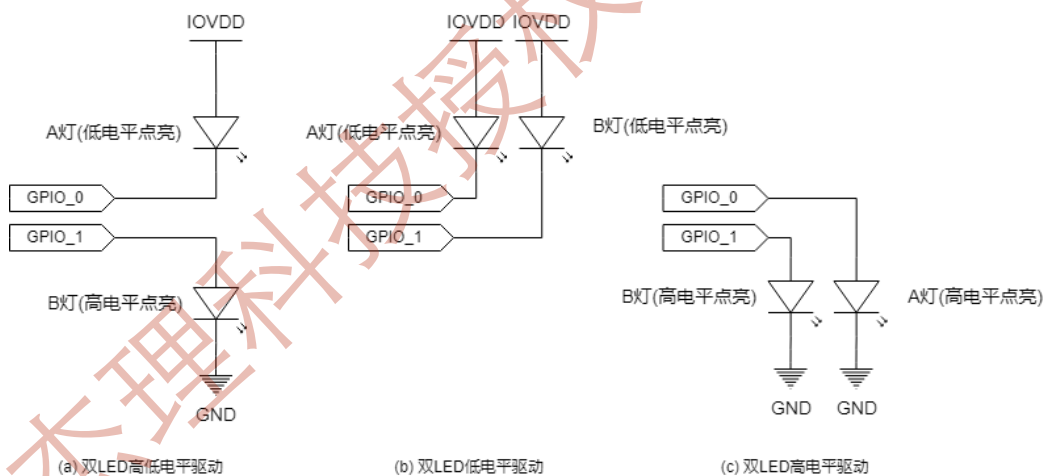


图 1-2 双IO控制双LED原理图

3.3 模拟推灯模式：

本项目不支持模拟推灯模式，因此SCON0和ANA_CON0寄存器配置无效

4. 工作状态说明

PWM LED模块可以配置两个周期和占空比不同的PWM0、PWM1实现对LED的亮度（PWM0）与亮灭（PWM1）控制，其关系如下图所示。PWM0的占空比将决定LED的亮度，PWM0的占空比越大，IO工作时输出的时间约长，流过LED灯的电流越大，表现为亮度越亮；PWM1的周期和占空比决定LED的闪烁模式，当处于亮灭模式时(BREATHE_EN=0)，PWM1的波形类似占空比固定PWM波，当处于呼吸灯模式时(BREATHE_EN=1)，PWM1的波形类似为三角波调制的PWM波，即该PWM波的占空比随时间逐渐增加或减少。

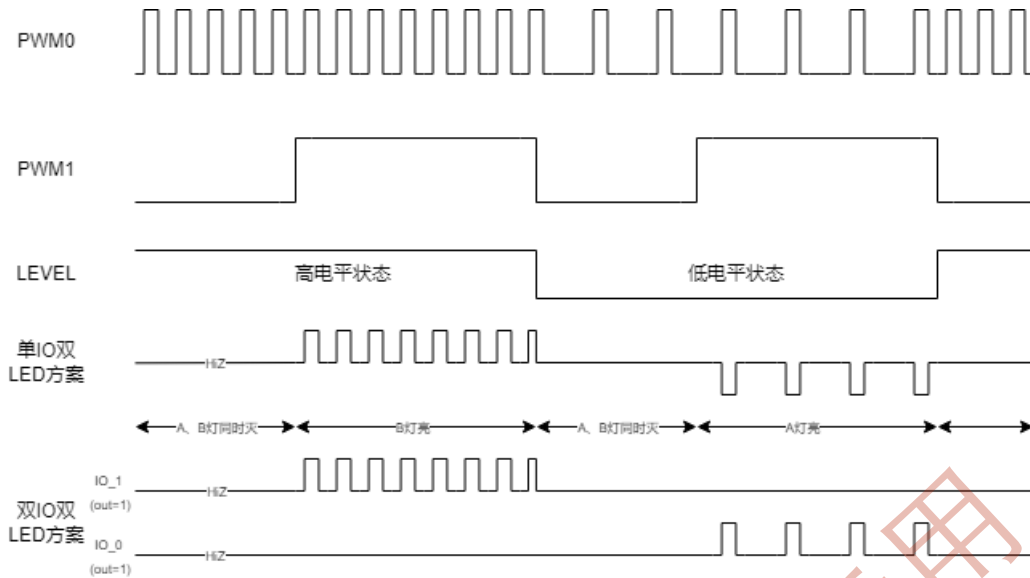


图 1-3 PWM_LED模式工作状态说明图

当PWM1的从0跳变1时，此时PWM LED模块控制IO进入输出周期，此时挂起一个开始信号，软件可根据该信号切换IO为PWM LED模式。当IO处于PWM LED模式下，当PWM0也输出为1时，此时IO进入输出状态，输出电平根据配置决定；当PWM1从1跳变为0时，此时PWM LED模块退出IO输出周期，此时也会挂起一个退出信号，软件可根据该信号切换IO为GPIO模式。当IO在其他模式下，PWM0不会控制GPIO的输出

可以通过开始信号和退出信号的配合，软件上实现GPIO的分时复用功能

15.3 特殊注意点

1. 所有GPIO可用（除PF口），不推荐在PP口和USB口上使用。其中，在USB DP/DM上使用时，其驱动强度与GPIO存在区别，PWMLLED无法更改USB的驱动强度档位；在PP口上使用时，其驱动行为与GPIO存在区别，PP口的HD会作为IO模式和PG模式切换控制信号。
2. PDOWN（蓝牙sniff）可用，SOFF不可用。
3. 目前只放了一个模块，只支持单IO单LED、单IO双LED、双IO双LED。
4. IO在普通IO和LED模块IO间切换时，会有一个时钟的波形突变，可能会造成LED灯闪亮，IO配置需依照以下顺序，来降低该突变影响：

4.2 普通IO→LED模块输出IO ``C xxxx //LED模块配置

```
JL_PORTX->DIE = 0x00;
JL_PORTX->DIEH = 0x00; // 关闭所使用IO的DIE和DIEH JL_PORTX->DIR = 0xff; // 所使用IO 设置为输入
JL_PORTX->PU |= (1<<(BIT(x)<<1)); // 打开所使用IO的上拉 JL_PORTX->PD |= (1<<(BIT(x)<<1)); // 打开所使用IO的下拉
JL_PORTX->DIR &= ~(1<<BIT(x)); // 所使用IO 设置为输出 JL_PLED->CON0 |= BIT(x); // 使能LED模块，开始驱动LED ``
```

4.2 LED模块输出IO→普通IO

```
JL_PORTX->DIR = 0xff; // 所使用IO 设置为输入
JL_PORTX->PU = 0x00; // 恢复所使用IO的上拉设置
JL_PORTX->PD = 0x00; // 恢复所使用IO的下拉
JL_PLED->CON0 &= ~BIT(0); // 关闭LED模块
```

5. PWM_LED模块的控制寄存器有操作顺序要求，当CON0的PWM_EN（即BIT(0)位）写入1后，当前配置就会被锁定，运行过程中写入的值会导致不可预测的效果。因此，需要先配置其他的设置，最后再使能PWM_EN启动整个PWM_LED模块。部分寄存器采用了特殊设计，能够在PWM_LED运行时（即PWM_EN使能后）动态配置，该部分寄存器需要等待PWM_EN使能且至少一个PWM0_CLK周期后才可以配置，特殊设计的寄存器如下所示：

5.1 OUT1_DIV / OUT0_DIV

5.2 CNT_CON / CNT_DATA / CNT_DEC / CNT_SYNC

15.4 寄存器并接

1. 呼吸灯，灭灯延时计数器，16bit

{PWM_DUTY3[7:0], PWM_DUTY2[7:0]};

15.5 时钟控制

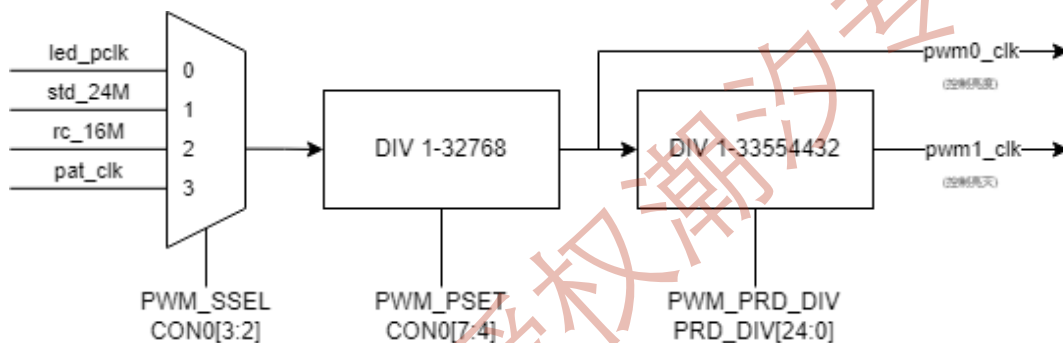


图 1-2 时钟控制框图

其中PWM0由pwm0_clk驱动，周期PRD0由PWM_BRI_PRD 10bit决定，有一个循环计数bright_cnt决定显示亮度。

其中PWM1由pwm1_clk驱动，周期PRD1 最多256个时钟，有一个循环计数counter_cnt。

特别地，当呼吸灯功能打开，PWM0、PWM1分别控制LED的亮度级数变化及最亮和最暗保持的时间。

led_pclk的时钟来自P33的模块，用于低功耗模式时保持推灯模块工作。

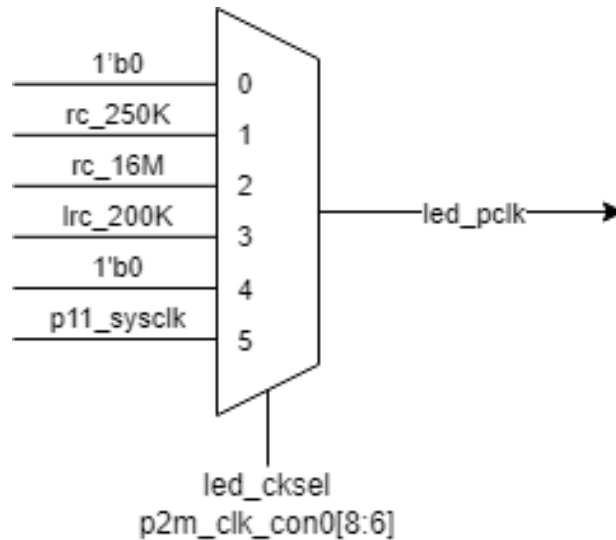


图 1-3 led_pclk时钟控制框图

15.6 二级亮灭控制

PWM1可以控制LED在一个PRD1周期亮灭一次或两次。

下图以一个LED灯PWM电平举例：

输出高电平亮

单周期双闪

单灯

非呼吸灯模式

配置：

```
PWM_DUTY3_EN = 0;
PWM_DUTY2_EN = 1;
PWM_DUTY1_EN = 1;
PWM_DUTY0_EN = 1;
PWM1_PRD_SEL = 0; //PWM1 PRD 为256 pwm1_clk
PWM1_INV = 0; //固定为0不变，亮灭不取反，如下图，周期开始时是从灯灭开始
PWM0_INV = 0; //固定为0不变，灯亮度控制
OUT_LOGIC = 1; //固定为1不变，PWM输出逻辑控制
BREATHE_EN = 0;
SHIFT_DUTY = 0;
```

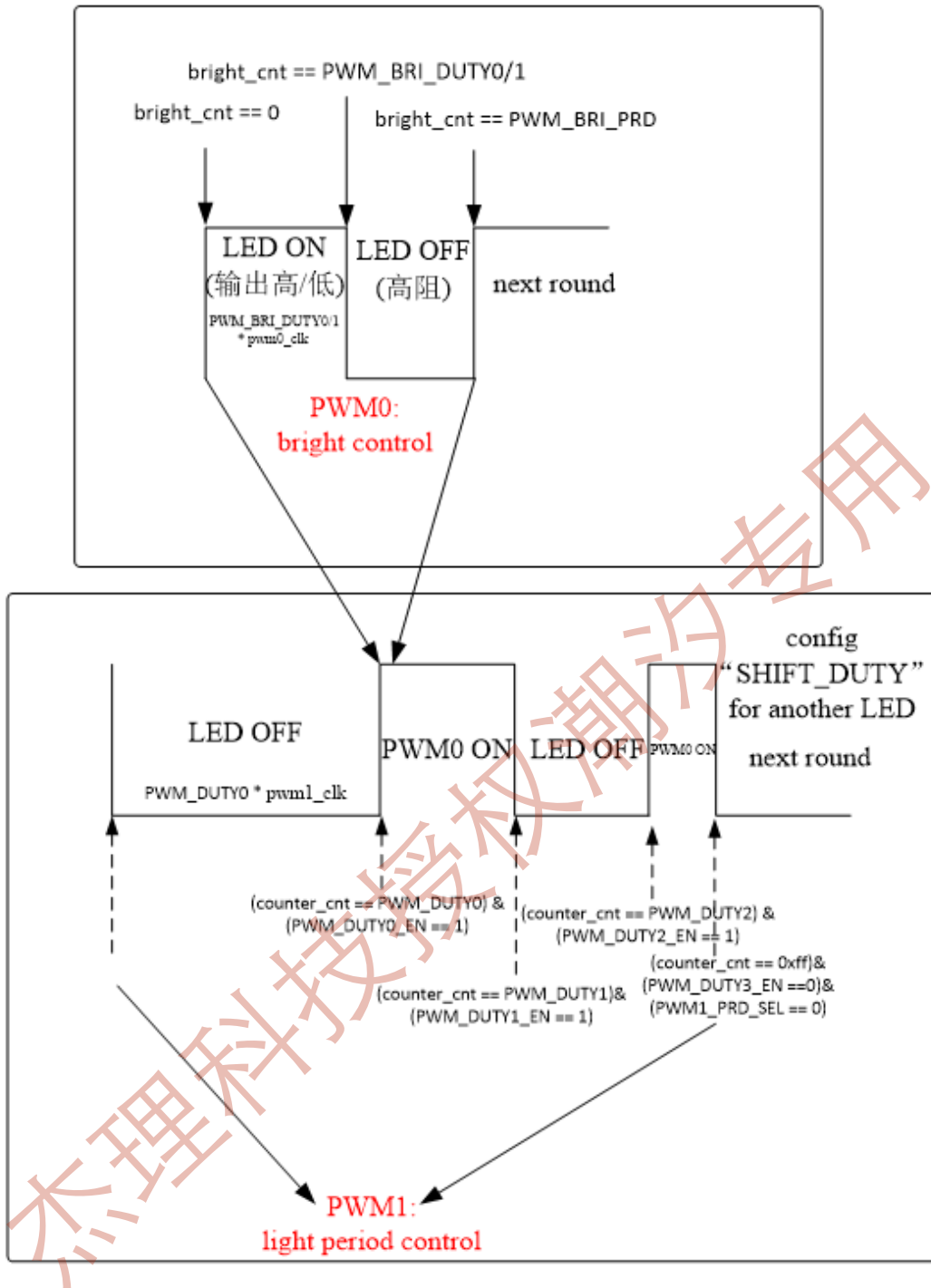


图 1-3 二级亮灭控制框图

【注意】：需要同时推两个颜色的灯时，请把PWM1的周期设置得尽可能短，小于人眼分辨极限即可。

15.7 呼吸灯控制

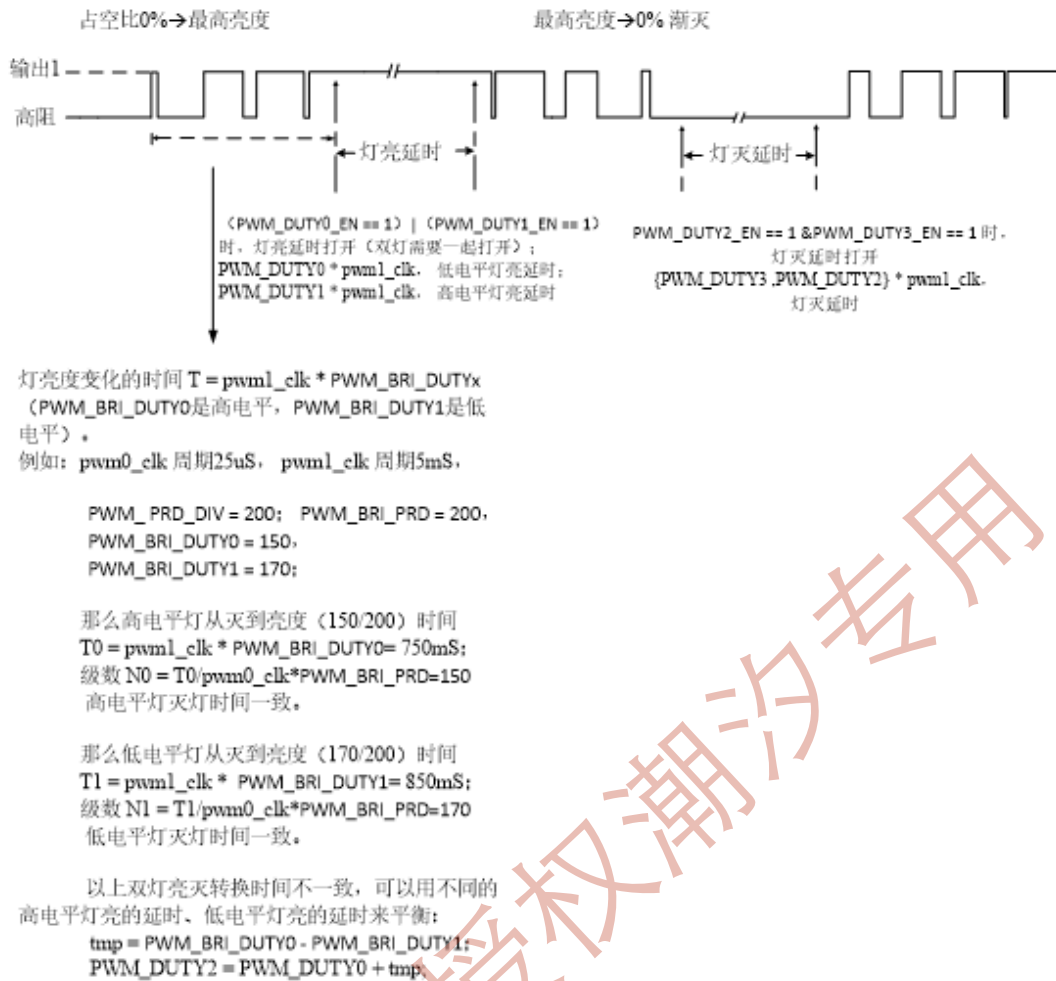


图 1-4 呼吸灯控制示意图

15.8 控制寄存器说明

15.8.1 CON0: PWM control register 0

Bit	Name	RW	Default	RV	Description
31-28	RESERVED	-	-	-	预留

Bit	Name	RW	Default	RV	Description
27-24	LEVEL_SHIFT_CYCLE	rw	0x0	-	LED周期变色（LEVEL信号切换周期），非呼吸灯 0000：不变色 0001：1个PWM1周期变色 ... 1111：15个PWM1周期变色 LED周期变色，SHIFT_DUTY[0]常设为0 0000：不变色 0010：1个呼吸灯周期变色（1个呼吸灯周期为：最暗到最亮，再到最暗） 0100：2个呼吸灯周期变色 ... 1110：7个呼吸灯周期变色
23	LEVEL_INV	rw	0	-	LED输出状态（控制LEVEL信号初始状态） 0：保持在低电平状态 1：保持为高电平状态 注 意在LEVEL_SHIFT_CYCLE为0时可用
22	RESERVED	-	-	-	预留
21-20	IO_DRV_SHIFT_CLK	rw	0x0	-	输出驱动切换时钟个数（PWM0_CLK） 00: 1 01: 2 10: 4 11: 8（干扰少，但输出会延时）
19-18	IO_MAX_DRV_1	rw	0x0	-	IO 高电平状态 最大输出强度： 00: 2.4mA（8mA mos + 120Ω res） 01: 8mA（8mA mos） 10: 18.4mA（24mA mos + 120Ω res） 11: 24mA（24mA mos）
17-16	IO_MAX_DRV_0	rw	0x0	-	IO 低电平状态 最大输出强度： 00: 2.4mA（8mA mos + 120Ω res） 01: 8mA（8mA mos） 10: 18.4mA（24mA mos + 120Ω res） 11: 24mA（24mA mos）

Bit	Name	RW	Default	RV	Description
15	DUTY3_EN	rw	0	-	亮灭DUTY控制3开关/呼吸灯空周期延时（高8bit）开关 0：亮灭DUTY控制3关闭（BREATHE_EN = 0 & PWM1_PRD_SEL = 0） 呼吸灯空周期延时关闭（BREATHE_EN = 1） 1：亮灭DUTY控制3开启（BREATHE_EN = 0 & PWM1_PRD_SEL = 0） 呼吸灯空周期延时开启（BREATHE_EN = 1）
14	DUTY2_EN	rw	0	-	亮灭DUTY控制2开关/呼吸灯空周期延时（低8bit）开关 0：亮灭DUTY控制2关闭（BREATHE_EN = 0） 呼吸灯空周期延时关闭（BREATHE_EN = 1） 1：亮灭DUTY控制2开启（BREATHE_EN = 0） 呼吸灯空周期延时开启（BREATHE_EN = 1）
13	DUTY1_EN	rw	0	-	亮灭DUTY控制1开关/呼吸灯最高亮度（高电平状态）延时开关 0：亮灭DUTY控制1关闭（BREATHE_EN = 0） 呼吸灯最高亮度延时（高电平状态）关闭（BREATHE_EN = 1） 1：亮灭DUTY控制1开启（BREATHE_EN = 0） 呼吸灯最高亮度延时（高电平状态）开启（BREATHE_EN = 1）
12	DUTY0_EN	rw	0	-	亮灭DUTY控制0开关/呼吸灯最高亮度（低电平状态）延时开关 0：亮灭DUTY控制0关闭（BREATHE_EN = 0） 呼吸灯最高亮度延时（低电平状态）关闭（BREATHE_EN = 1） 1：亮灭DUTY控制0开启（BREATHE_EN = 0） 呼吸灯最高亮度延时（低电平状态）开启（BREATHE_EN = 1）

Bit	Name	RW	Default	RV	Description
11	PWM1_PRD_SEL	rw	0	-	PWM1亮灭周期选择: 0: PWM1周期为0xff 1: PWM1周期为 PWM_DUTY3
10	OUT_LOGIC	rw	0	1	输出逻辑, 软件常设为1 0: LED在低电平时为亮 (PWM0 PWM1)。 1: LED在高电平时为亮 (PWM0 & PWM1)。
9	PWM1_INV	rw	0	0	PWM1输出的LED亮灭波形取 反(默认由0开始), 可以控制亮灭周期是从灯亮开 始还是灯灭开始, 软件常设为 0。 0: PWM1输出的LED亮度波形 不取反(灭灯开始的PWM1周 期) 1: PWM1输出的LED亮度波形 取反(亮灯开始的PWM1周 期)
8	PWM0_INV	rw	0	0	PWM0输出的LED亮度波形取 反(默认由1开始), 软件常设为0 0: PWM0输出的LED亮度波形 不取反 1: PWM0输出的LED亮度波形 取反
7-4	PWM_PSET	rw	0x0		PWM_PSET: PWM pre_scaler setting。设置PWM0的分 频器除法因子 [1:0]: 00:div1 01:div4 10:div16 11:div64 [2]: 0: x1 1: x2 [3]: 0: x1 1: x256 计算方式: DIV = [1:0]的divx 乘以 [2]倍数 乘以 [3]倍数。
3-2	PWM_SSEL	rw	0x0		PWM_SSEL: PWM时钟源的选 择。 00: led_clk 01: std_24M 10: rc_16M 11: pat_clk
1	BREATHE_EN	rw	0		呼吸灯使能。 0: 呼吸灯功能关闭 1: 呼吸灯功能打开

Bit	Name	RW	Default	RV	Description
0	PWM_EN	rw	0		PWM使能。 0: 模块总使能关闭 1: 模块总使能打开

15.8.2 CON1: PWM control register 1

Bit	Name	RW	Default	RV	Description
31-20	RESERVED	-	-	-	预留
11	OUT_END_PND	r	0	-	输出周期结束标志位，当模块即将进入输出周期（即，处于PWM1的下降沿）时，该标志位置1
10	OUT_END_PND_CLR	w	0	-	OUT_END_PND标志清除，写1进行清除操作
9	OUT_END_PND_IE	rw	0	-	OUT_END_PND中断使能，使能后允许相应的标志位产生中断请求
8	OUT_END_PND_WE	rw	0	-	OUT_END_PND唤醒使能，使能后允许相应的标志位产生低功耗唤醒请求
7	OUT_START_PND	r	0	-	输出周期开始标志位，当模块即将进入输出周期（即，处于PWM1的上升沿）时，该标志位置1
6	OUT_START_PND_CLR	w	0	-	OUT_START_PND标志清除，写1进行清除操作
5	OUT_START_PND_IE	rw	0	-	OUT_START_PND中断使能，使能后允许相应的标志位产生中断请求
4	OUT_START_PND_WE	rw	0	-	OUT_START_PND唤醒使能，使能后允许相应的标志位产生低功耗唤醒请求
3	PRD_PND	r	0	-	周期结束标志位，亮灭周期结束或呼吸灯周期结束时，该标志位置1
2	PRD_PND_CLR	w	0	-	PRD_PND标志清除，写1进行清除操作

Bit	Name	RW	Default	RV	Description
1	PRD_PND_IE	rw	0	-	PRD_PND中断使能，使能后允许相应的标志位产生中断请求
0	PRD_PND_WE	rw	0	-	PRD_PND唤醒使能，使能后允许相应的标志位产生低功耗唤醒请求

15.8.3 BRI_PRD: PWM0 brightness period register

Bit	Name	RW	Default	RV	Description
9-0	PWM_BRI_PRD	rw	0x3ff	-	亮度周期

15.8.4 BRI_DUTY0: PWM0 low state brightness duty register

Bit	Name	RW	Default	RV	Description
9-0	PWM_BRI_DUTY0	rw	0x0	-	亮度控制，低电平状态占空比，例： PWM_BRI_DUTY0 = 0; 灯最暗 PWM_BRI_DUTY0 = 1; 灯亮度增强一档 ... PWM_BRI_DUTY0 = PWM_BRI_PRD; 灯最亮

15.8.5 BRI_DUTY1: PWM0 high state brightness duty register

Bit	Name	RW	Default	RV	Description
9-0	PWM_BRI_DUTY1	rw	0x0	-	亮度控制，高电平状态占空比，例： PWM_BRI_DUTY1 = 0; 灯最暗 PWM_BRI_DUTY1 = 1; 灯亮度增强一档 ... PWM_BRI_DUTY1 = PWM_BRI_PRD; 灯最亮

15.8.6 PRD_DIV: PWM1 period divider register

Bit	Name	RW	Default	RV	Description
24-0	PWM_PRD_DIV	rw	0x0	-	PWM时钟的分频因子设置。

PWM_PRD_DIV : pwml_clk分频比控制器, 25bit

0x000: div = 1;

0x001: div = 2;

.....

0x1fff: div = 33554432;

15.8.7 DUTY0: PWM duty0 register

Bit	Name	RW	Default	RV	Description
7-0	PWM_DUTY0	rw	0x0	-	BREATHE_EN = 0 & PWM_DUTY0_EN = 1: 亮灭DUTY控制0 BREATHE_EN = 1 & PWM_DUTY0_EN = 1: 呼吸灯最高亮度延时(单IO模式下低电平状态; 双IO模式下IO_0工作状态)

15.8.8 DUTY1: PWM duty1 register

Bit	Name	RW	Default	RV	Description
7-0	PWM_DUTY1	rw	0x0		BREATHE_EN = 0 & PWM_DUTY1_EN = 1: 亮灭DUTY控制1 BREATHE_EN = 1 & PWM_DUTY1_EN = 1: 呼吸灯最高亮度延时(单IO模式下高电平状态; 双IO模式下IO_1工作状态)

15.8.9 DUTY2: PWM duty2 register

Bit	Name	RW	Default	RV	Description
7-0	PWM_DUTY2	rw	0x0		BREATHE_EN = 0 & PWM_DUTY2_EN = 1: 亮灭DUTY控制2 BREATHE_EN = 1 & PWM_DUTY2_EN = 1: 呼吸灯灭灯延时低8bit

15.8.10 DUTY3: PWM duty3 register

Bit	Name	RW	Default	RV	Description
-----	------	----	---------	----	-------------

Bit	Name	RW	Default	RV	Description
7-0	PWM_DUTY3	rw	0x0		BREATHE_EN = 0 & PWM1_PRD_SEL = 0 & PWM_DUTY3_EN = 1: 亮灭DUTY控制3 BREATHE_EN = 0 & PWM1_PRD_SEL = 1: 亮灭PWM1周期 PWM1_PRD BREATHE_EN = 1 & PWM_DUTY3_EN = 1: 呼吸灯灭灯延时高8bit

15.8.11 SCON0: pwm_sync_controller0

Bit	Name	RW	Default	RV	Description
31-9	RESERVED	-	-	-	预留
6-4	ANA_CLK_SEL	rw	0	-	选择模拟推灯模块的时钟源
3	ANA_DUTY_DIV2_EN	rw	0	-	模拟推灯模块时钟分频系数: 0: 不分频并保持占空比为50% 1: 2分频且占空比设置为25%
2	ANA_DEADTIME_EN	rw	0	0	模拟推灯模块在变色（自动切换红、蓝灯）时是否自动插入保护时间（保护时间内，模拟推灯模块输出为高阻，红、蓝灯都不点亮） 0: 不插入保护时间 1: 自动插入保护时间 注意：保护时间被硬件设置 1/pwm_clk。该值不可更改！
1	ANA_OUT_EN	rw	0	-	是否使能模拟推灯模块输出: 0: 高阻 1: 允许模拟推灯模块输出

Bit	Name	RW	Default	RV	Description
0	ANA_EN	rw	0	-	是否使能模拟推灯模块： 0: 关闭模拟推灯模块 1: 模拟推灯模块 注意：模拟推灯模块不影响双IO双LED模式

15.8.12 ANA_CON0: pwm_ana_controller0

Bit	Name	RW	Default	RV	Description
31-3	RESERVED	-	-	-	预留
2	ANA_RES	rw	0	-	模拟推灯模块保留功能
1-0	ANA_BLUE_IB	rw	0x0	-	模拟推灯模块，正电压驱动强度选择： 0: 约2.5mA 1: 约3.5mA 2: 约4.5mA 3: 约5.5mA

15.8.13 SCON1: pwm_sync_controller1

Bit	Name	RW	Default	RV	Description
4	OUT1_DIV_EN	rw	0	-	是否使能IO_1占空比系数功能
3	OUT0_DIV_EN	rw	0	-	是否使能IO_0占空比系数功能
2	IO1_OUT	rw	0	-	IO_1驱动LED电路类型： 0: 低电平驱动LED电路 1: 高电平驱动LED电路
1	IO0_OUT	rw	0	-	IO_0驱动LED电路类型： 0: 低电平驱动LED电路 1: 高电平驱动LED电路
0	DIO_EN	rw	0	1	是否使能双IO模块。 该功能不影响单IO模式

15.8.14 OUT1_DIV

Bit	Name	RW	Default	RV	Description
5-0	OUT1_DIV	rw	0x0	-	双IO模式的IO_1占空比系数

15.8.15 OUT0_DIV

Bit	Name	RW	Default	RV	Description
5-0	OUT0_DIV	rw	0x0	-	双IO模式的IO_0占空比系数

注意: OUTx_DIV寄存器需要PWM_LED模块运行后才允许配置, 即PWM_EN为1

IOx_DIV的作为占空比的系数影响最终得到的占空比, 即最终IO_x引脚的PWM0(亮度控制)的占空比如下公式所述:

$$IOx_DUTY_ALL = (OUTx_DIV + 1) / 64 * PWM0_DUTYx$$

上式中, IOx_DUTY_ALL是PWM0(亮度)控制最终输出的占空比结果, PWM0_DUTYx是通过寄存器设置的PWM0占空比, 其可以来自BRI_DUTY0/1的设置, 也可以来自呼吸灯模式下的自动占空比设置。该寄存器的工作原理如下图所示:

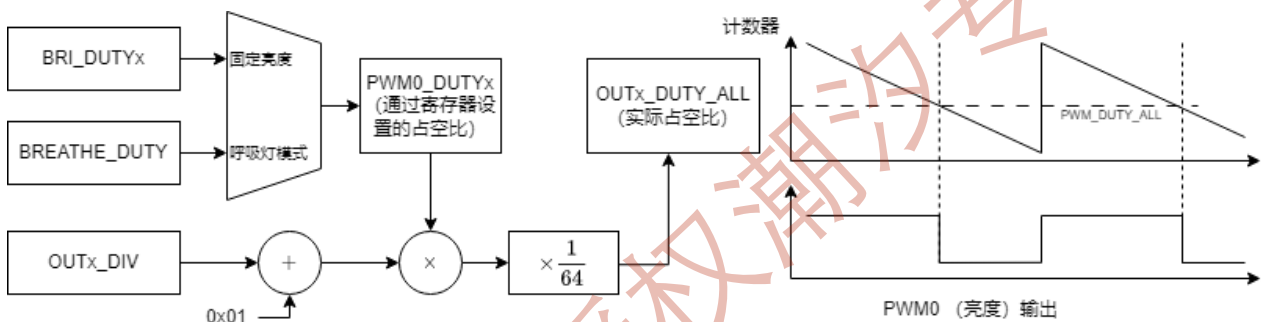


图 2-1 OUTx_DIV寄存器工作原理图

注意: 需要区分PWM0(亮度控制)与IO_0和IO_1输出的区别。PWM0是控制亮度的PWM生成器, 而IO_0与IO_1输出的LED亮度都由PWM0控制。当需要配置IO_0与IO_1的不同亮度区别时, 需要使用该寄存器。三者的关系如下图所示:

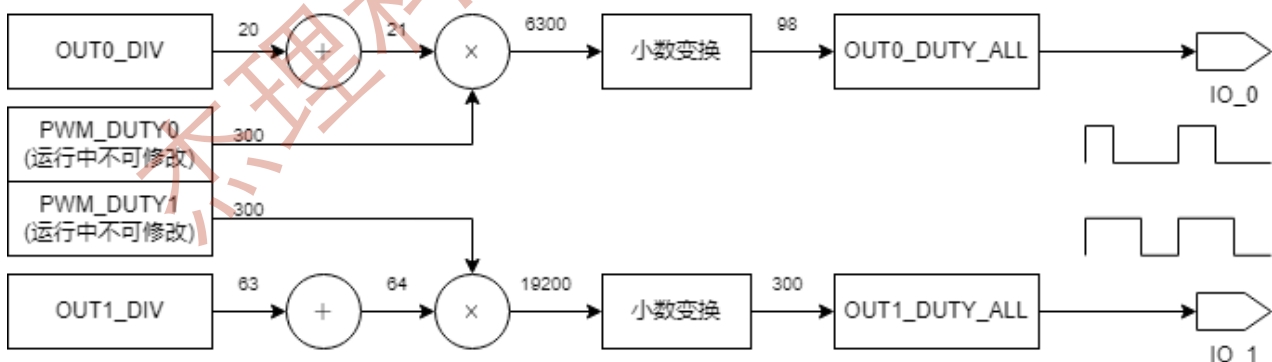


图 2-2 双IO模式动态更新亮度原理示意图

15.8.16 CNT_CON : counter_cnt read control

Bit	Name	RW	Default	RV	Description
7-6	RESERVED	-	-	-	预留

Bit	Name	RW	Default	RV	Description
5	RD_DONE	r	1		读取完成标记位 0: 寄存器未准备好 1: 寄存器更新完成, 可以进行读取操作
4	RD_KICK	w	0		读取rtc_dat
3-0	RESERVED	-	-	-	预留

写入RD_KICK后, CNT_DATA的值开始更新, 直到RD_DONE为1时更新完成, 在RD_DONE为0期间读取的CNT_DATA值为无效值

使用CNT_CON寄存器读取CNT_CON流程如下:

```

u16 pwm_cnt_read (void){
    u16 cnt;
    CNT_CON |= BIT(4);
    while(!(CNT_CON & BIT(5)));
    cnt = CNT_DATA;
    return cnt;
}

```

15.8.17 CNT_DATA : counter_cnt read data

Bit	Name	RW	Default	RV	Description
17	BREATHE BRIGHTNESS	r	x		LED breathe brightness: 0: LED breath brightness decrease 1: LED breath brightness increase
16	LED LEVEL	r	x		LED output level: 0: low level output state of pwmlcd 1: high level output state of pwmlcd
15:8	COUNTER_CNT	r	0x0		counter_cnt[15:8]:only use in breathe mode, meaningless in other case
7-0	COUNTER_CNT	r	0x0		counter_cnt[7:0]:use in all case

下图是呼吸灯状态下, CNT_RD的图解:

1. PWM_DUTY0_EN = 1; PWM_DUTY1_EN = 1; PWM_DUTY2_EN = 1;

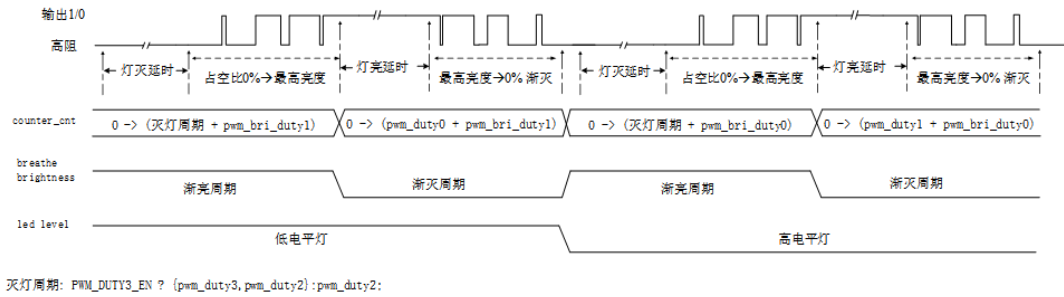


图 1-5 CNT_RD亮灭延时全开示意图

2. PWM_DUTY0_EN = 0; PWM_DUTY1_EN = 1; PWM_DUTY2_EN = 1;

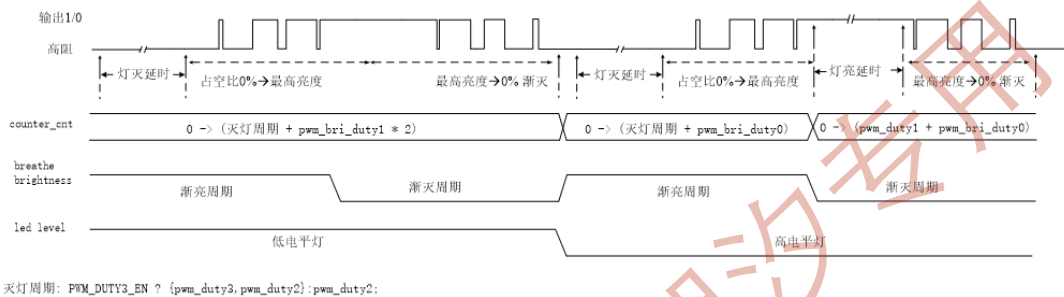


图 1-6 CNT_RD亮灭延时半开示意图

3. PWM_DUTY0_EN = 0; PWM_DUTY1_EN = 0; PWM_DUTY2_EN = 0;

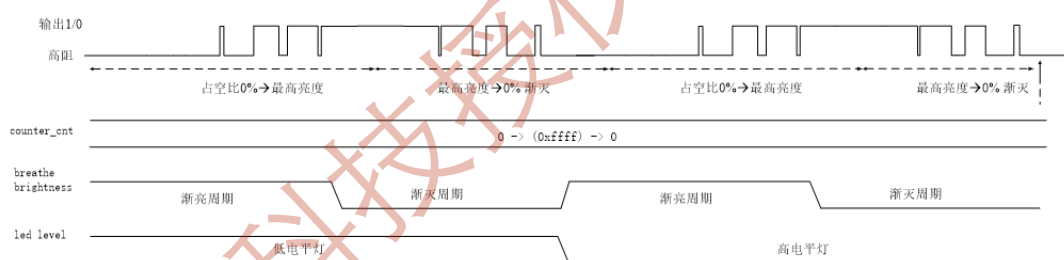


图 1-7 CNT_RD亮灭延时关闭示意图

15.8.18 CNT_DEC : counter_cnt decrease

Bit	Name	RW	Default	RV	Description
7-0	CNT DEC	rw	0x0		counter_cnt decrease in clocks below: 0: function disable 1: counter_cnt stop 1 clk every 2clk 2: counter_cnt stop 1 clk every 3clk 3: counter_cnt stop 1 clk every 4clk ...

15.8.19 CNT_SYNC : counter_cnt sync

Bit	Name	RW	Default	RV	Description
7-0	CNT SYNC	w	x		counter_cnt sync

use in more than one chip PWM LED period sync.

for example in TWS earphone:

L earphone clock frequency is 30k.

R earphone clock frequency is 32k.

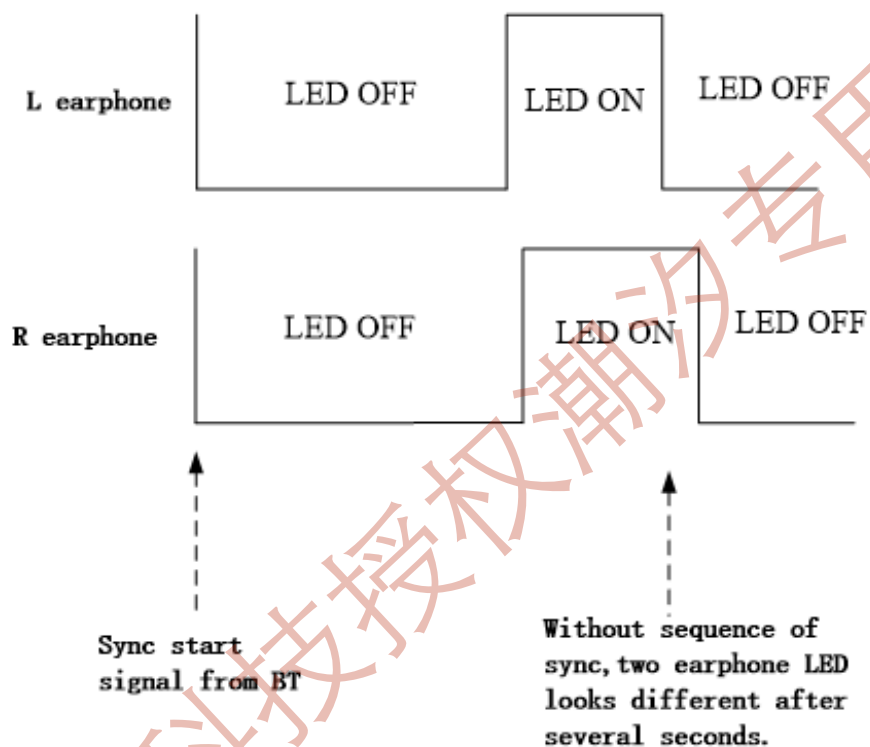


图 1-8 TWS LED without SYNC

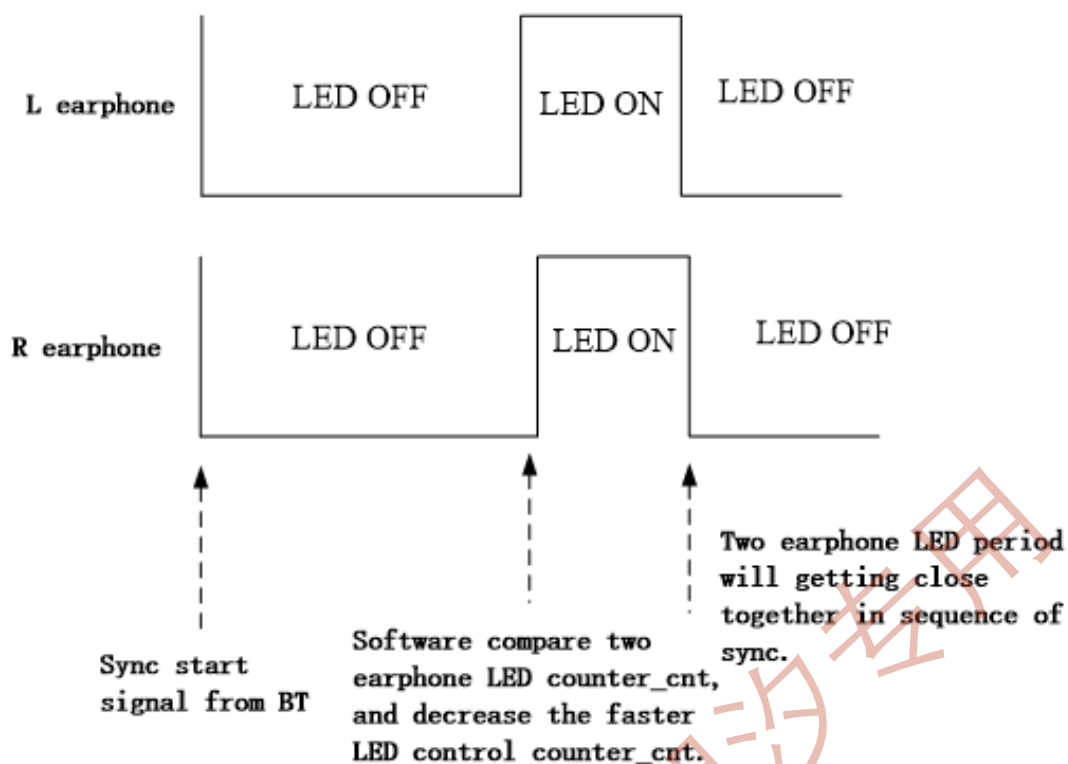


图 1-9 TWS LED SYNC start

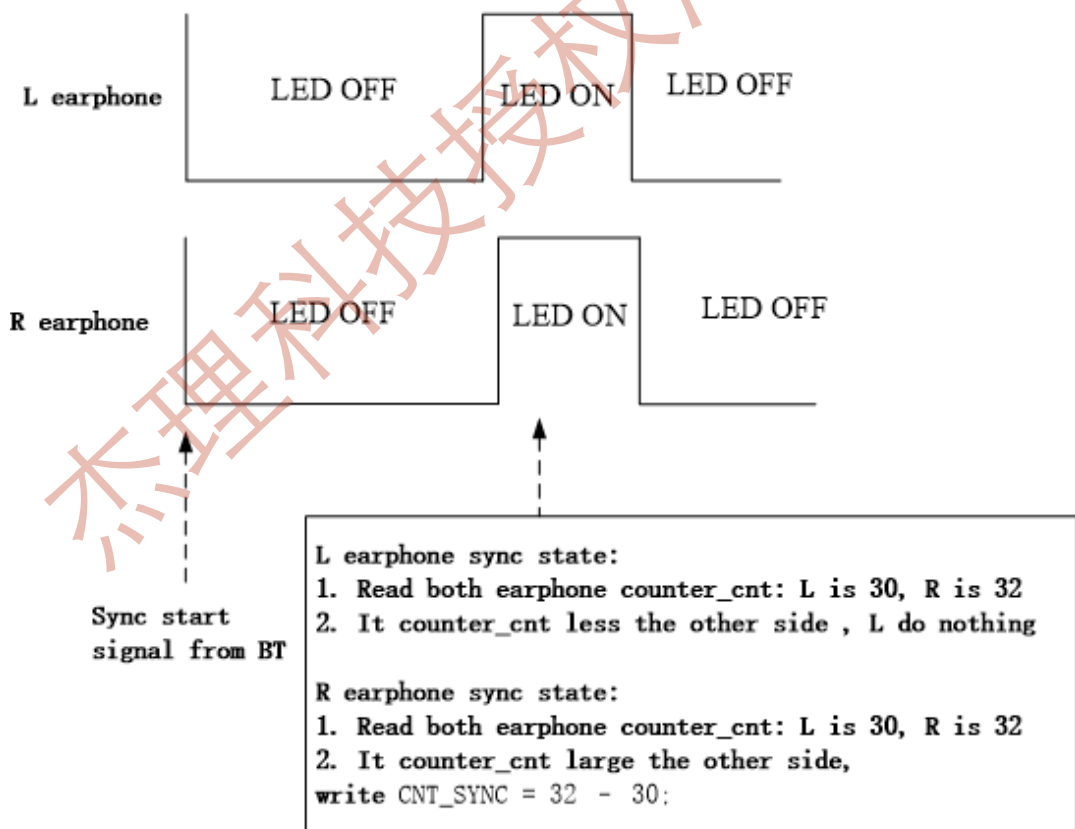


图 1-10 TWS LED SYNC strategy

16 QDEC

16.1 概述

QDEC (quadrature decoder) 是一个用于正交编码器检测的模块，支持两线输入。兼容4 phase、2 phase、1 phase 三种模式，通常旋转编码器使用 1phase 即可，鼠标滑轮等使用了 2 phase，分辨率要求更高的可使用 4 phase 模式；同时本模块自带定时器，可独立定时检测。

16.2 特殊注意点

16.3 数字模块控制寄存器

16.3.1 QDECx_CON: QDEC control register

Bit	Name	RW	Default	RV	Description
31-16	QDEC_SPND	rw	0x0	0x0	QDEC_SPD，采样时钟设置 $sr_clk = lsb_clk / (QDEC_SPND << 4)$ 采样时钟要求， 输入脉冲频率的16倍以上，一般 16 - 32 倍比较合适； 常见编码器可设置在介于2k - 500Hz之间；
15-11	RESERVED	-	-	-	预留
10	QDEC_INT_MODE	rw	0	0	1：根据 QDEC_SMP 配置产生定时标志与中断 0：当检测到输入信号发生变化时产生标志与中断
9-8	QDEC_MODE	rw	0	0	QDEC_MODE 0 0: 1 phase 模式，普通旋转编码器 0 1: 2 phase 模式，通常鼠标滑轮 1 2: 4 phase 模式，分辨率最高
7	PND	r	0	0	PND，只读。写入无效
6	CPND	w	0	0	CPND写入1清除PND， 并且计数器结果输送到寄存器 QDECx_DAT， QDECx_DBE 后重新计数； 读取 QDECx_DAT，QDECx_DBE 前需要把CPND写入1；读出永远为0
5-2	RESERVED	-	-	-	预留
1	QDEC_POL	rw	0	0	1: 把输入信号极性反转， 会影响输入信号正反识别
0	EN	rw	0	0	QDEC_EN 0: 模块关闭 1: 模块打开

16.3.2 QDECx_SMP: QDEC sample register

Bit	Name	RW	Default	RV	Description
7-0	QDEC_SMP	rw	0x0	0x0	当 QDEC_INT_MODE 为 1 时， 定时器时间为 $T_{smp} = (QDEC_SMP + 1) / sr_clk$

16.3.3 QDECx_DAT: QDEC control register

Bit	Name	RW	Default	RV	Description
7-0	QDEC_DAT	r	0x0	0x0	编码器步数计算器输出结果， 此寄存器为8位有符号数， 表示编码器正反的步数。 CPND写入1清除PND时， 步数计算器输出到QDEC_DAT后自动清0 重新计算。通常读取QDEC_DAT前需 CPND写入1 获取计步器结果； 如果长时间不读取数据清0， 会导致步数计算器溢出无法正确识别步数跳转； 需要注意内核总线同步问题， 想在CPND写入1后马上读结果， 需要加asm (“csync”)；

16.3.4 QDECx_DBE: QDEC double transition error

Bit	Name	RW	Default	RV	Description
7-0	QDEC_DBE	r	0x0	0x0	编码器步数错误计算器输出结果； 此寄存器为8位无符号数记录检测到编码； 器输入信号发生非连续跳步行进次数， 出现这些情况时， 会影响计步器输出结果QDEC_DAT的准确性； 通常 QDEC_SPND 设置采样率过慢导致跟不上信号变化， 或者过快受编码器模块输出信号毛刺影响。 当然编码器出现损坏， 输出信号有时不满足正交要求也可能导致问题 发生。同样CPND写入1清除PND时， 步数错误计算器结果输出到QDEC_DBE 后自动清0 重新计算，读取QDEC_DBE前需要CPND写 入1； 需要注意内核总线同步问题， 想在CPND写入1后马上读结果， 需要加asm (“csync”)；

16.4 DEMO CODE

```
void get_qdec_res( void)
{
    char qdec_cnt;
```



```
unsigned char qdec_error_cnt;  
JL_QDEC0->CON0 |= BIT(6);  
asm("csync");  
qdec_cnt = JL_QDEC0->DAT;  
qdec_error_cnt = JL_QDEC0->DBE;  
}
```

杰理科技授权潮汐专用

17 RAND64

17.1 概述

RAND64是一个产生64位伪随机序列的模块，它由片内RC振荡器驱动，RC振荡器振荡频率的不稳定性进一步确保了生成序列的随机性。

由于片内RC振荡器的振荡频率大约为250KHz，即每个周期4uS，因此RAND64生成的序列约每4uS自动更新一次。若软件有读取RAND64寄存器（RAND64H或RAND64L）的动作，则生成序列将立即更新，以确保每次读取均能获得不同的序列。

17.2 控制寄存器

寄存器列表	宽度	读写	复位值
RAND64H	32bit	只读	不确定
RAND64L	32bit	只读	不确定

18 SDC

18.1 概述

SD接口是一个标准的遵守SD协议的串行通讯接口，在上面传输的数据一Byte(8bit)为最小单位。

1. SD接口只支持主机模式。
2. SD接口支持1bit data和4bit data模式：
3. 1bit data模式：串行数据通过一根DAT线传输，一个字节数据需8个SD时钟
4. 4bit data模式：串行数据通过四根DAT线传输，一个字节数据需2个SD时钟
5. SD接口支持高速模式和低速模式，最高时钟速度可以达到50MHz。
6. SD接口支持命令DMA和数据DMA，两个DMA时独立的，即可以同时做DMA

18.2 数字模块控制寄存器

18.2.1 JL_SD0->CON0: sd0 control register 0

Bit	Name	RW	Default	RV	Description
15	DINT	r	0	-	SD数据pending 0: 数据未结束 1: 数据结束
14	CLR_DINT	w	0	1	清空SD数据pending（读为0） 0: N/A 1: 清空
13	LINE4	rw	0	-	4线模式控制位 0: 1线模式 1: 4线模式
12	DATCLK8	rw	0	-	SD发送/接收数据时时钟延长8个CLK 0: 延长关闭 1: 延长打开
11	DATCRC	r	0	-	接收数据的CRC标志位 0: 接收数据的响应CRC正常 1: 接收数据出错，CRC错误
10-8	SDDMODE	w	0x0	-	SD数据模式（kickstart） 000: no operation 100: 发送数据，不检查“busy”位 101: 发送数据，检查“busy”位 110: 接收数据，不检查“busy”位 111: 接收数据，检查“busy”位

Bit	Name	RW	Default	RV	Description
7	CINT	r	0	-	SD命令pending 0: 命令未结束 1: 命令结束
6	CLR_CINT	w	0	-	清空SD命令pending (读为0) 0: N/A 1: 清空
5	TIMEOUT	r	0	-	命令响应超时标志位 0: 命令响应正常 1: 命令响应超时
4	CMDCLK8	rw	0	-	SD发送/接收命令时钟延长8个CLK 0: 延长关闭 1: 延长打开
3	CMDCRC	r	0	-	CMDCRC: 接收响应的CRC标志位 0: 接收响应正确, CRC正确 1: 接收响应出错, CRC错误
2-0	SDCMODE	w	0x0	-	SD命令模式(kickstart) 000: no operation 001: send command, receive 6-byte response, with check busy 010: send command, receive 17-byte response, with check busy 011: send command, without receive response, with check busy 101: send command, receive 6-byte response, without check busy 110: send command, receive 17-byte response, without check busy 111: send command, without receive response, without check busy 上面后六种命令模式设置后, sdc控制器会进入命令收发状态

18.2.2 JL_SD0->CON1: sd0 control register 1

Bit	Name	RW	Default	RV	Description
15-8	SD_BAUD	rw	0x0	-	SD_BAUD: SD波特率 SD波特率= $F_{sys}/(SD_BAUD+1)$
7	DATIE	rw	0	-	DATIE: SD数据中断允许 0: 不允许 1: 允许
6	CMDIE	rw	0	-	CMDIE: SD命令中断允许 0: 不允许 1: 允许

Bit	Name	RW	Default	RV	Description
5	CLKALL	rw	0	-	CLKALL: SD时钟开关 0: SD时钟在不传送命令和数据时关闭 1: SD时钟常开
4	RESERVED	-	-	-	预留
3-1	CRCSTA	r	0x0	-	CRCSTA: CRS State 详细请查阅SD spec
0	SDCEN	rw	0	-	SDCEN: SD控制器允许位 0: SD控制器关闭 1: SD控制器打开

18.2.3 JL_SD0->CON2: sd0 control register 2

Bit	Name	RW	Default	RV	Description
15-9	RESERVED	-	-	-	预留
8-0	SDDCNT	rw	0x0	-	SDDCNT: 数据读写数目 000: 1byte 001: 2byte 002: 3byte ... 1ff: 512byte

18.2.4 JL_SD0->CTU_CON: sd0 continue control register

Bit	Name	RW	Default	RV	Description
15-11	RESERVED	-	-	-	预留
10-8	SDDMODE_KEEP	rw	0x0	-	SDDMODE_KEEP: SD连续数据读写模式 (连续模式开始后多次写进SDDMODE, 不是kickstart, kickstart是写SD_CTU_CNT) 000: N/A 100: 发送数据, 不检查“busy”位 101: 发送数据, 检查“busy”位 110: 接收数据, 不检查“busy”位 111: 接收数据, 检查“busy”位
7	CTU_PND	r	0	-	CTU_PND: SD连续读写结束标志 0: 未结束 1: 结束
6	CLR_CTU_PND	w	0	-	CLR_CTU_PND: 清空SD连续读写结束标志(读为0)【注意】: 先清SD数据pending, 再清这里) 0: N/A 1: 清空

Bit	Name	RW	Default	RV	Description
5	CTU_ERR	r	0	-	CTU_ERR: SD连续读写出错标志 0: 正确 1: 出错
4	CLR_CTU_ERR	w	0	-	CLR_CTU_ERR: 清空SD连续读写出错标志（读为0） （【注意】：先清SD数据pending, 再清这里） 0: N/A 1: 清空
3	CTU_BUSY	r	0	-	CTU_BUSY: SD数据连续读写状态 0: 空闲 1: 忙
2	CTU_PND_IE	rw	0	-	CTU_PND_IE: SD读写结束中断允许 0: 不允许 1: 允许
1	CTU_ERR_IE	rw	0	-	CTU_ERR_IE: SD读写出错中断允许（出错后会打断数据传输） 0: 不允许 1: 允许
0	SD_CTU_EN	rw	0	-	SD_CTU_EN: SD 数据多个BLOCK读写使能 0: 关闭 1: 打开

18.2.5 JL_SD0-> CTU_CNT: sd0 continual transfer counter register

Bit	Name	RW	Default	RV	Description
15-0	CTU_CNT	rw	0x0	-	SD 数据连续读写模式计数器（写寄存器kictstart数据传输）： 0: 开始1次SD数据读写 1: 开始2次SD数据读写 2: 开始3次SD数据读写 0xffff: 开始65536次SD数据读写 读寄存器可以看到剩下的数据传输次数： 0: 不再传输 1: 还剩1次传输 2: 还剩2次传输 0xffff: 还剩65535次传输

18.2.6 JL_SD0-> CPTR: sd0 command pointer register

Bit	Name	RW	Default	RV	Description
26-0	CPTR	w	0x0	-	SD命令地址

18.2.7 JL_SD0-> DPTR: sd0 data pointer register

Bit	Name	RW	Default	RV	Description
26-0	DPTR	w	0x0	-	SD数据地址

18.3 配置流程示例

18.3.1 发送命令、接收响应流程

1. 配置SD_CON0和SD_CON1进行初始化;
2. 往memory中写入长度为40bit的命令（实际命令长度为48bit，其中8bit是CRC和结束位由硬件自动进行计算并补齐），然后将数据的首地址配置给SDCPTR。（可将命令写入数组中，然后将数组首地址配置给SDCMDPTR，见2. SDCPTR的CMD responses 排列示例（CMD3））；
3. 配置SDCMODE（SD_CON0[2:0]）进行kickstart，开始发送命令；
4. 等待PENDING（查询CINT（SD_CON0[7]）），或等待中断到来；
5. 配置CLR_CINT（SD_CON0[6]），清除PENDING；
6. 查询CMDCRC（SD_CON0[3]），确认接收的响应正确；
7. 查看接收的响应。

18.3.2 发送数据流程（单块写CMD24）

1. 配置SD_CON0和SD_CON1进行初始化；
2. 配置SD_CON2进行配置将要发送数据的块长度；
3. 往memory中写入将要发送的数据，然后将数据的首地址配置给SDDPTR；
4. 配置SDDMODE（SD_CON0[10:8]）进行kickstart，开始发送数据；
5. 等待PENDING（查询DINT（SD_CON0[15]）），或等待中断到来；
6. 配置CLR_DINT（SD_CON0[14]），清除PENDING；
7. 查询CRCSTA（SD_CON1[3:1]），确认发送的数据正确。

18.3.3 接收数据流程（单块读CMD17）

1. 配置SD_CON0和SD_CON1进行初始化；
2. 配置SD_CON2进行配置将要接收数据的块长度；
3. 配置SDDPTR为数据保存在memory中的首地址；
4. 配置SDDMODE（SD_CON0[10:8]）进行kickstart，开始接收数据；

5. 等待PENDING（查询DINT（SD_CON0[15]）），或等待中断到来；
6. 配置CLR_DINT（SD_CON0[14]），清除PENDING；
7. 查询DATCRC（SD_CON0[11]），确认接收的数据正确；
8. 查看接收的数据。

【注意】：进行单块读CMD17时，需先进行kickstart接收数据，然后发送命令，最后等待数据PENDING，即在上述流程4和5之间进行发送命令。

18.3.4 发送数据流程（多块写CMD25）

1. 配置SD_CON0、SD_CON1和SD_CTU_CON进行初始化；
2. 配置SD_CON2为将要发送数据的块长度，配置SD_CTU_CNT为将要发送数据的块数量；
3. 往memory中写入将要发送的数据，然后将数据的首地址配置给SDDPTR；
4. 配置SDDMODE_KEEP（SD_CTU_CON[10:8]）进行kickstart，开始发送数据；
5. 等待PENDING（查询CTU_PND（SD_CTU_CON[7]）），或等待中断到来；
6. 配置CLR_CTU_PND（SD_CTU_CON[6]），清除PENDING；
7. 查询CTU_ERR（SD_CTU_CON[5]），确认发送数据过程没有发生错误（发送各块数据返回的CRC状态有一个错误都会标志在该位）。

18.3.5 接收数据流程（多块写CMD18）

1. 配置SD_CON0、SD_CON1和SD_CTU_CON进行初始化；
2. 配置SD_CON2为将要接收数据的块长度，配置SD_CTU_CNT为将要接收数据的块数量；
3. 配置SDDPTR为数据保存在memory中的首地址；
4. 配置SDDMODE_KEEP（SD_CTU_CON[10:8]）进行kickstart，开始接收数据；
5. 等待PENDING（查询CTU_PND（SD_CTU_CON[7]）），或等待中断到来；
6. 配置CLR_CTU_PND（SD_CTU_CON[6]），清除PENDING；
7. 查询CTU_ERR（SD_CTU_CON[5]），确认接收数据过程没有发生错误（接收各块数据的CRC有一个错误都会标志在该位）；
8. 查看接收的数据。

【注意】：进行多块读CMD18时，需先进行kickstart接收数据，然后发送命令，最后等待数据PENDING，即在上述流程4和5之间进行发送命令。

18.4 DEMO CODE

CMD responses 排列示例（CMD3）：

```
u32 __attribute__((aligned (16))) cmd_str[6];
u32 rca;
//send cmd
```



```
cmd_str[0] = ((0x40 + 3));
cmd_str[1] = 0x0;
JL_SD0->CPTR = (volatile u32)cmd_str;
.....
//obtain respond
sd0_cmd_finish_check();
rca = cmd_str[1] >> 24;
rca |= cmd_str[2] << 8;
rca &= 0x0000ffff;
```

cmd_str[0]	Cmd send byte 3	Cmd send byte 2	Cmd send byte 1	Cmd send byte 0
cmd_str[1]	Cmd res byte 1	Cmd res byte 0	Cmd send byte 5	Cmd send byte 4
cmd_str[2]	Cmd res byte 5	Cmd res byte 4	Cmd res byte 3	Cmd res byte 2
cmd_str[3]	Cmd res byte 9	Cmd res byte 8	Cmd res byte 7	Cmd res byte 6
cmd_str[4]	Cmd res byte 13	Cmd res byte 12	Cmd res byte 11	Cmd res byte 10
cmd_str[5]		Cmd res byte 16	Cmd res byte 15	Cmd res byte 14

（本质是接收响应的memory地址紧接在发送命令的memory地址后面）

19 SPI

19.1 概述

SPI接口是一个标准的遵守SPI协议的串行通讯接口，在上面传输的数据以Byte（8bit）为最小单位，且永远是MSB在前。

SPI接口支持主机和从机两种模式。

主机：SPI接口时钟由本机产生，提供给片外SPI设备使用。

从机：SPI接口时钟由片外SPI设备产生，提供给本机使用。

工作于主机模式时，SPI接口的驱动时钟可配置，范围为系统时钟~系统时钟/256。

工作于从机模式时，SPI接口的驱动时钟频率无特殊要求，但数据速率需要进行限制，否则易出现接收缓冲覆盖错误，在系统不繁忙情况下可以接近系统时钟速率。

SPI接口可独立地选择在SPI时钟的上升沿或下降沿更新数据，在SPI时钟的上升沿或下降沿采样数据。

SPI接口支持单向（Unidirection）和双向（Bidirection）模式

单向模式：使用SPICK和SPIDAT两组连线，其中SPIDAT为双向信号线，同一时刻数据只能单方向传输。

双向模式：使用SPICK，SPIDI和SPIDO三组连线，同一时刻数据双向传输。但DMA不支持双向数据传输，当在本模式下使能DMA时，也只有一个方向的数据能通过DMA和系统进行传输。

SPI单向模式支持1bit data、2bit data和4bit data模式，即：

1bit data模式：串行数据通过一根DAT线传输，一个字节数据需8个SPI时钟。

2bit data模式：串行数据通过两根DAT线传输，一个字节数据需4个SPI时钟。

4bit data模式：串行数据通过四根DAT线传输，一个字节数据需2个SPI时钟。

SPI双向模式只支持1bit data模式，即：

1bit data模式：串行数据通过两根DAT线进行双向传输，一个字节数据需8个SPI时钟。

SPI接口在发送方向上为单缓冲，在上一次传输未完成之前，不可开始下一次传输。在接收方向上为双缓冲，如果在下一次传输完成时CPU还未取走本次的接收数据，那么本次的接收数据将会丢失。

SPI接口的发送寄存器和接收寄存器在物理上是分开的，但在逻辑上它们一起称为SPIBUF寄存器，使用相同的SFR地址。当写这个SFR地址时，写入至发送寄存器。当读这个SFR地址时，从接收寄存器读出。

SPI传输支持由CPU直接驱动，写SPIBUF的动作将启动一次Byte传输。

SPI传输也支持DMA操作，但DMA操作永远是单方向的，即一次DMA要么是发送一包数据，要么是接收一包数据，不能同时发送并且接收一包数据，即使在双向模式下也是这样。每次DMA操作支持的数据量为 $1-(2^{32}-1)$ Byte。写SPIADR的动作将启动一次DMA传输。

19.2 特殊注意点

【注意】：该模块所有说明及配置仅作为SPIx模块使用，不适用到SFC中

19.3 数字模块控制寄存器

19.3.1 SPIx_CON: SPIx control register 0

Bit	Name	RW	Default	RV	Description
31-22	RESERVED	-	-	-	预留
21	OF_PND	r	0	-	dma fifo overflow中断，当spi做从机dma接收时，fifo满且dma响应慢，而新接收的数据到来时导致fifo发出溢出就会产生of_pnd 【注意】：仅做debug使用!!!
20	OF_PND_CLR	w	0	-	清除of_pnd，写1清零
19	OF_IE	rw	0	1	of_pnd使能位
18	UF_PND	r	0	-	dma fifo underflow中断，当spi做从机dma发送时，fifo空且dma响应慢，而主机端请求新数据时导致fifo读空就会产生uf_pnd 【注意】：仅做debug使用
17	UF_PND_CLR	w	0	-	清除uf_pnd，写1清零
16	UF_IE	rw	0	1	uf_pnd使能位
15	PND	r	0	0	中断请求标志，当1Byte传输完成或DMA传输完成时会被硬件置1。 有3种方法清除此标志 向PCLR写入‘1’ 写SPIBUF寄存器来启动一次传输 写SPICNT寄存器来启动一次DMA
14	CPND	w	0	1	软件在此位写入‘1’将清除PND中断请求标志。
13	IE	rw	0	1	SPI 中断使能 0: 禁止SPI中断 1: 允许中断允许
12	DIR	rw	0	0	在单向模式或DMA操作时设置传输的方向 0: 发送数据 1: 接收数据
11-10	DATW	r	0x0	0x0	SPI数据宽度设置 00: 1bit数据宽度 01: 2bit 数据宽度 10: 4bit数据宽度 11: NA，不可设置为此项
9-8	RESERVED	-	-	-	预留

Bit	Name	RW	Default	RV	Description
7	CSID	rw	0	0	SPICS信号极性选择 0: SPICS空闲时为0电平 1: SPICS空闲时为1电平
6	CKID	rw	0	0	SPICK信号极性选择 0: SPICK空闲时为0电平 1: SPICK空闲时为1电平
5	UE	rw	0	0	更新数据边沿选择 0: 在SPICK的上升沿更新数据 1: 在SPICK的下降沿更新数据
4	SE	rw	0	0	采样数据边沿选择 0: 在SPICK的上升沿采样数据 1: 在SPICK的下降沿采样数据
3	BDIR	rw	0	0	单向/双向模式选择 0: 单向模式，数据单向传输，同一时刻只能发送或者接收数据。 数据传输方向因收发而改变，所以由硬件控制，不受写IO口DIR影响。 1: 双向模式，数据双向传输，同时收发数据，但DMA只支持一个方向的数据传输。 数据传输方向设置后不改变，所以由软件控制，通过写IO口DIR控制。
2	CSE	rw	0	1	SPICS信号使能，always set为1
1	SLAVE	rw	0	0	从机模式
0	SPIEN	rw	0	0	SPI接口使能 0: 关闭SPI接口 1: 打开SPI接口

19.3.2 SPIx_BAUD: SPIx baudrate setting register

Bit	Name	RW	Default	RV	Description
7-0	SPI_BAUD	rw	0x0	0x0	SPI主机时钟设置寄存器 $SPICK = \text{system clock} / (\text{SPIBAUD} + 1)$

19.3.3 SPIx_BUF: SPIx buffer register

Bit	Name	RW	Default	RV	Description
7-0	SPI_BUF	rw	0x0	0x0	发送寄存器和接收寄存器共用此SFR地址。 写入至发送寄存器，从接收寄存器读出。

19.3.4 SPIx_ADR: SPIx DMA address register

Bit	Name	RW	Default	RV	Description
31-0	SPI_ADR	rw	0x0	0x0	SPI DMA起始地址寄存器，只写，读出为不确定值

19.3.5 SPIx_CNT: SPIx DMA counter register

Bit	Name	RW	Default	RV	Description
31-0	SPI_CNT	rw	0x0	0x0	SPI DMA计数寄存器，读出为当下剩余读写数量，此寄存器用于设置DMA操作的数目（按Byte计）并启动DMA传输。 如，需启动一次512Byte的DMA传输，写入0x0200，此写入动作将启动本次传输。

19.3.6 SPIx_CON1: SPIx control1 register

Bit	Name	RW	Default	RV	Description
31-2	RESERVED	-	-	-	预留
4	SPI_9B_DATA	rw	0	-	在9bit模式下，发送命令时，为0； 发送参数/数据时，为1。
3	SPI_9B_MODE	rw	0	-	0: 关闭9bit模式 1: 打开9bit模式
2	MIX_MODE	rw	0	-	3wire模式混合接线
1-0	SPI_BIT_MODE	rw	0	0	设置spi输入输出位流模式，默认0为标准格式 0: 【7,6,5,4,3,2,1,0】 1: 【0,1,2,3,4,5,6,7】 2: 【3,2,1,0,7,6,5,4】 3: 【4,5,6,7,0,1,2,3】

19.4 传输波形

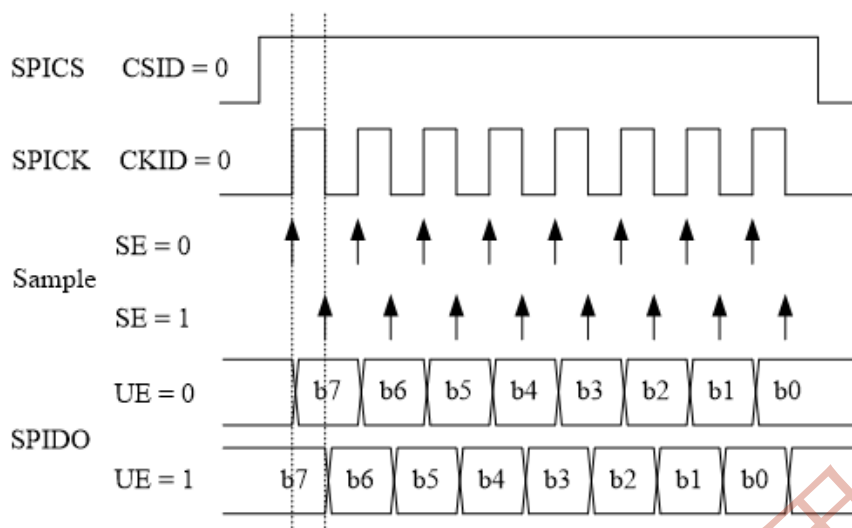


图 1-1 传输波形1

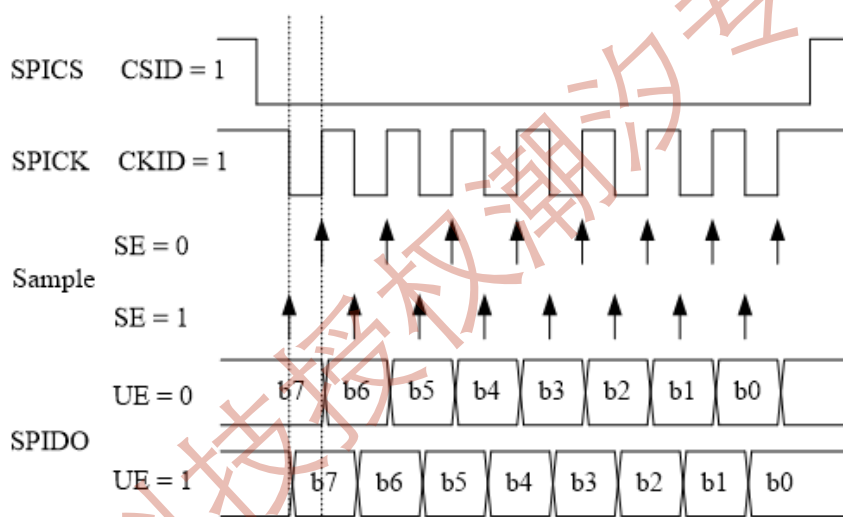


图 1-2 传输波形2

20 32位GPTIMER

20.1 概述

Timer是一个集成了定时/计数/捕获功能于一体的多功能定时器（计数位宽参见芯片规格）。它的计数时钟驱动源可以选择片内时钟或片外信号。它带有一个可配置的最高达64的异步预分频器，用于扩展定时时间或片外信号的最高频率。同时具有上升沿/下降沿捕获功能，可以方便的对片外信号的高电平/低电平宽度进行测量。

且每个Timer都具有独立的CAP_FLT硬件模块，使能后可用于去除掉从片外进入TIMER CAP信号上的窄脉冲信号（多用于提升红外接收解码的质量）；CAP FLT模块使用一个固定的时基对TIMER CAP进入的信号进行采样，改变CAP_FLT时基时钟的产生可兼容不同的系统工作状态，也可在一定范围内调整对CAP信号的过滤效果，滤波范围如下：

- 必须连续4次采样均为‘1’时，输出信号才会变为‘1’；
- 必须连续4次采样均为‘0’时，输出信号才会变为‘0’

脉宽小于3倍时基的窄脉冲将被滤除

TIMER 计数时钟源，CAP_FLT时基时钟源 参见 Clock_system 文档；

20.2 控制寄存器

寄存器列表	TIMER0	TIMER1	TIMER2	TIMER3
Tx_CON	T0_CON	T1_CON	T2_CON	T3_CON
Tx_CNT	T0_CNT	T1_CNT	T2_CNT	T3_CNT
Tx_PRD	T0_PRD	T1_PRD	T2_PRD	T3_PRD
Tx_PWM	T0_PWM	T1_PWM	T2_PWM	T3_PWM
Tx_IRFLT	T0_IRFLT	T1_IRFLT	T2_IRFLT	T3_IRFLT

该芯片只有TIMER0-TIMER3

20.3 寄存器说明

20.3.1 Tx_CON: timer x control register

Bit	Name	RW	Default	RV	Description
31-17	RESERVED	-	-	-	预留
16	DUAL_EDGE_EN	rw	0	-	DUAL_EDGE_EN:双边沿捕获模式，当MODE=2或3时，使能该位即变成双边沿捕获模式，在上沿和下沿会TxCNT的值写入TxPR。
15	PND	r	0	-	PND: 中断请求标志，当timer溢出或产生捕获动作时会被硬件置1，需要由软件清0。

Bit	Name	RW	Default	RV	Description
14	PCLR	w	x	-	PCLR: 软件在此位写入‘1’将清除PND中断请求标志。
13-10	SSEL	rw	0x0	-	SSEL3-0: TIMER驱动源选择见图1-2 【注意】: 选中的时钟需要使用上面PSET分频到 (lsb_clk/2)以下!
9	PWM_INV	rw	0	-	PWM_INV: PWM信号输出反向。
8	PWM_EN	rw	0	-	PWM_EN: PWM信号输出使能。此位置1后, 相应IO口的功能将会被PWM信号输出替代。
7-4	PSET	rw	0x0	-	PSET3-0: 预分频选择位 0000: 分频倍数为1; 0001: 分频倍数为4; 0010: 分频倍数为16; 0011: 分频倍数为64; 0100: 分频倍数为2; 0101: 分频倍数为8; 0110: 分频倍数为32; 0111: 分频倍数为128; 1000: 分频倍数为256; 1001: 分频倍数为1024; 1010: 分频倍数为4096; 1011: 分频倍数为16384; 1100: 分频倍数为512; 1101: 分频倍数为2048; 1110: 分频倍数为8192; 1111: 分频倍数为32768;
3-2	CAP_FLT_EN	rw	0x0	-	捕获信号滤波使能 (仅捕获模式使用) 0: 从IO 直接输入到TIMER CAP; 1: 从IO 输入后先经过FLT后再到TIMER CAP (需配合IRFLT寄存器使用)
1-0	MODE	rw	0x0	-	MODE1-0: 工作模式选择 00: TIMER关闭; 01: 定时/计数模式; 10: IO口上升沿捕获模式 (当IO上升沿到来时, 把TxCNT的值捕捉到TxPR中); 11: IO口下降沿捕获模式 (当IO下降沿到来时, 把TxCNT的值捕捉到TxPR中)。

20.3.2 Tx_CNT: timer x counter register

Bit	Name	RW	Default	Description
31-0	Tx_CNT	rw	x	Timer的计数寄存器

20.3.3 Tx_PR: timer x period register

Bit	Name	RW	Default	Description
31-0	Tx_PR	rw	x	Timer的周期寄存器

在定时/计数模式下，当Tx_CNT = Tx_PR时，Tx_CNT会被清0。

在上升沿/下降沿捕获模式下，TxPR是作为捕获寄存器使用的，当捕获发生时，Tx_CNT的值会被复制到Tx_PR中。而此时Tx_CNT自由的由0->(2³²-1)->0计数，不会和Tx_PR进行比较清0。

20.3.4 Tx_PWM: timer x PWM register

Bit	Name	RW	Default	Description
31-0	Tx_PWM	rw	x	Timer的PWM设置寄存器

在PWM模式下，此寄存器的值决定PWM输出的占空比。占空比N的计算公式如下：

$$N = (Tx_PWM / Tx_PR) * 100\%$$

此寄存器带有缓冲，写此寄存器的动作不会导致不同步状态产生的PWM波形占空比瞬间过大或过小的问题

20.3.5 Tx_IRFLT: timer x PWM register

Bit	Name	RW	Default	Description
7-4	PSEL	rw	x	PSEL[3-0]: 时基发生器分频选择 0000: 分频倍数为1; 0001: 分频倍数为2; 0011: 分频倍数为8; 0100: 分频倍数为16; 0101: 分频倍数为32; 0110: 分频倍数为64; 0111: 分频倍数为128; 1000: 分频倍数为256; 1001: 分频倍数为512; 1010: 分频倍数为1024; 1011: 分频倍数为2048; 1100: 分频倍数为4096; 1101: 分频倍数为8192; 1110: 分频倍数为16384; 1111: 分频倍数为32768;
3-2	TSRC	rw	x	TSRC[1-0]: 时基发生器驱动源选择，见Clock_System 文档
1	RESERVED	-	-	预留
0	FLT_EN	rw	0	FLT_EN: FLT使能 0: 关闭IRFLT; 1: 打开IRFLT; ;

PSEL选定的分频倍数N和TSRC选定的驱动时钟的周期Tc共同决定了IRFLT用于采样信号的时基Ts:

$$T_s = T_c * N$$

如选择24MHz的时基时钟，并且分频倍数为512时， $T_s = 21.3\mu s$ ，所有小于 $(21.3 * 3 = 63.9\mu s)$ 的窄脉冲信号，均会被滤除；

杰理科技授权潮汐专用

21 UART

21.1 概述

支持接收带循环Buffer的DMA模式和普通模式。

UART在DMA接收的时候有一个循环Buffer，UTx_RXSADR表示它的起始，UTx_RXEADR表示它的结束。同时，在接收过程中，会有一个**超时计数器（UTx_OTCNT）**，如果在指定的时间里没有收到任何数据，**则超时中断就会产生**。超时计数器是在收到数据的同时自动清空。

21.2 特殊注意点

【注意】：

- OT_PND触发流程，满足一下（1-4）顺序，则可产生OT_PND：
 - 写UTx_OTCNT，数值不为0启动OT功能；
 - 收到n个byte数据；
 - 从最后一个下降沿开始，等待OT超时（每来一个下降沿都会重新计时，时钟与接收baud_clk一致）；
 - 当OTCNT与超时时间相等，OT_PND置1（只能通过CLR_OTPND清零）。
- 读接收的数据数量时，先将RDC该bit写1，接着asm(“csync”)清空流水线，然后软件查询RDC_OVER置1，置1后即可读取HRXCNT数值，RDC_OVER只在RDC写1之后生效，平常为无效状态（不管0或1）；

21.3 数字模块控制寄存器

21.3.1 TX_CON0: uart tx control register 0

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	-	-	预留
15	TPND	r	1	-	TPND:TX Pending
14	RESERVED	-	-	-	预留
13	CL RTPND	w	0	-	CL RTPND: 清空TX Pending
12-3	RESERVED	-	-	-	预留
2	TXIE	rw	0	1	TXIE:TX中断允许 当TX Pending为1，而且TX中断允许为1，则会产生中断
1	RESERVED	-	-	-	预留
0	UTTXEN	rw	0	1	UTTXEN:UART模块发送使能

21.3.2 RX_CON0: uart rx control register 0

Bit	Name	RW	Default	RV	Description
31-15	RESERVED	-	-	-	预留
14	RPND	r	0	-	RPND:RX Pending& Dma_Wr_Buf_Empty, 数据接收不完Pending不会为1
13	RESERVED	-	-	-	预留
12	CLRRPND	w	0	-	CLRRPND: 清空 RX Pending
11	OTPND	r	0	-	OTPND:OverTime Pending
10	CLR_OTPND	w	0	-	CLR_OTPND: 清空OTPND
9	RESERVED	-	-	-	预留
8	RDC_OVER	r	0	-	用于判断写RDC后数据是否已存入内存标志, RDC写1时会清零, 当数据存入内存后置1。
7	RDC	w	0	-	RDC: 写1时, 将已经收到的数目写到UTx_HRXCNT, 已收到的数目清零写0无效
6	RX_MODE	rw	0	-	RXMODE (*): 读模式选择 0: 普通模式, 不用DMA; 1: DMA模式。
5	OT_IE	rw	0	1	OTIE:OT中断允许 0:不允许; 1: 允许。
4	DIVS	rw	0	-	DIVS:前3分频选择, 0为4分频, 1为3分频
3	RXIE	rw	0	1	RXIE:RX中断允许 当RX Pending为1, 而且RX中断允许为1, 则会产生中断
2	RESERVED	-	-	-	预留
1	UTRXEN	rw	0	1	UTRXEN:UART模块接收使能
0	RESERVED	-	-	-	预留

21.3.3 TX_CON1: uart tx control register 1

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	-	-	预留
15	CTSPND	r	0	0	CTSPND: CTS中断pending
14	RESERVED	-	-	-	预留

Bit	Name	RW	Default	RV	Description
13	CLR_CTSPND	w	0	-	CLR_CTSPND: 清除CTS pending
12-10	RESERVED	-	0	-	预留
9	TX_INV	rw	0	0	Tx发送数据电平取反 0: 不取反 1: 取反
8	RESERVED	-	-	-	预留
7	CTS_INV	rw	0	-	CTS流控有效电平选择（对方允许我方发送数据） 0: 高电平有效 1: 低电平有效
6-4	RESERVED	-	-	-	预留
3	CTSIE	rw	0	-	CTSIE: CTS中断使能 0: 禁止中断; 1: 中断允许
2	CTSE	rw	0	-	CTSE:CTS使能 0: 禁止CTS硬件流控制 1: 允许CTS硬件流控制
1-0	RESERVED	-	0	-	预留

21.3.4 RX_CON1: uart rx control register 1

Bit	Name	RW	Default	RV	Description
31-15	RESERVED	-	-	-	预留
14	RTSPND	r	0	0	RTSPND: RST pending
13	RESERVED	-	-	-	预留
12	CLR_RTSPND	w	0	-	CLRRTS: 清除RTS 0: N/A 1: 清空RTS
11-9	RESERVED	-	0	-	预留
8	RX_INV	rw	0	0	Rx接收数据电平取反 0: 不取反 1: 取反
7	RESERVED	-	-	-	预留

Bit	Name	RW	Default	RV	Description
6	RTS_INV	rw	0	-	RTS流控有效电平选择（允许对方发送数据） 0: 高电平有效 1: 低电平有效
5	RX_BYPASS	rw	1	1	Rx接收数据通路直通使能 0: 滤波输入 1: 直通输入
4	RX_DISABLE	rw	1	0	关闭数据接收 0: 开启输入（正常模式） 1: 关闭输入（输入固定为1）
3-1	RESERVED	-	-	-	预留
0	RTSE	rw	0	-	RTSE:RTS使能 0: 禁止RTS硬件流控制 1: 允许RTS硬件流控制

21.3.5 UTx_CON2: uart x control register 2

Bit	Name	RW	Default	RV	Description
15-3	RESERVED	-	-	-	预留
8	CHK_PND	r	0	-	校验中断标志
7	CLR_CHKPND	w	0	-	校验中断CHK_PND清除，置1清除
6	CHK_IE	rw	0	1	校验中断使能，置1使能
5-4	CHECK_MODE	rw	0	-	校验模式选择： 0: 常0 1: 常1 2: 偶校验 3: 奇校验
3	CHECK_EN	rw	0	0	校验功能使能位，置1使能
2	RB8	r	0	0	RB8:9bit模式时，RX接收的第9位
1	RESERVED	-	-	-	预留
0	M9EN	rw	0	0	M9EN: 9bit模式使能

21.3.6 UTx_BAUD: uart x baudrate register

Bit	Name	RW	Default	RV	Description
15-0	UTx_BAUD	w	0x0	0x0	注释如下所示

uart 的UTx_DIV的整数部分

串口频率分频器因子(UTx_DIV)的整数部分

当DIVS=0时，

$$\text{Baudrate} = \text{Freq_sys} / ((\text{UTx_BAUD}+1) * 4 + \text{BAUD_FRAC})$$

当DIVS=1时，

$$\text{Baudrate} = \text{Freq_sys} / ((\text{UTx_BAUD}+1) * 3 + \text{BAUD_FRAC})$$

(Freq_sys是apb_clk,指慢速设备总线的时钟，非系统时钟)

21.3.7 UTx_BUF: uart x data buffer register

Bit	Name	RW	Default	RV	Description
31-8	RESERVED	-	0	-	预留
7-0	UT_BUF	rw	0x0	-	uart的收发数据寄存器 写UTx_BUF可启动一次发送； 读UTx_BUF可获得已接收到的数据。

21.3.8 UTx_TXADR: uart x TX DMA buffer register

Bit	Name	RW	Default	RV	Description
31-25	RESERVED	-	0	-	预留
24-0	UTx_TXADR	rw	0x0	0x0	DMA发送数据的指针 写入时作为DMA发送数据的 起始地址，读出时时DMA正 在操作的位置

UTx_TXADR的读出值不是操作的实时地址

21.3.9 UTx_TXCNT: uart x TX DMA counter register

Bit	Name	RW	Default	RV	Description
31-0	UTx_TXCNT	rw	0x0	0x0	写UTx_TXCNT，控制器产生一次DMA的操作，同时清空中断（写该寄存器需要开启RX_EN才有效），当uart需要发送的数据达到UTx_TXCNT的值，控制器会停止发送数据的操作，同时产生中断（UTx_CON[15]）。

21.3.10 UTx_RXCNT: uart x RX DMA counter register

Bit	Name	RW	Default	RV	Description
-----	------	----	---------	----	-------------

Bit	Name	RW	Default	RV	Description
31-0	UTx_RXCNT	rw	0x0	0x0	写UTx_RXCNT，控制器产生一次DMA的操作，同时清空中断，当uart需要接收的数据达到UTx_RXCNT的值，控制器会停止接收数据的操作，同时产生中断（UTx_CON[14]）。

21.3.11 UTx_RXSADR: uart x RX DMA start address register

Bit	Name	RW	Default	RV	Description
31-25	RESERVED	-	0	-	预留
24-0	UTx_RXSADR	rw	0x0	0x0	DMA接收数据时，循环buffer的开始地址，需4byte地址对齐

21.3.12 UTx_RXEADR: uart x RX DMA end address register

Bit	Name	RW	Default	RV	Description
31-25	RESERVED	-	0	-	预留
24-0	UTx_RXEADR	rw	0x0	0x0	DMA接收数据时，循环buffer的结束地址。需4byte地址对齐 【注意】：UTx_RXSADR=UTx_RXEADR时没有循环buffer，接收地址会递增

21.3.13 UTx_HRXCNT: uart x have RX DMA counter register

Bit	Name	RW	Default	RV	Description
31-0	UTx_HRXCNT	r	0x0	0x0	当设这RDC（UTx_CON[7]）=1时，串口设备会将当前总共收到的字节数记录到UTx_HRXCNT里。

21.3.14 UTx_OTCNT: uart x Over Timer counter register

Bit	Name	RW	Default	RV	Description
31-0	UTx_OTCNT	rw	0x0	0x0	设置串口设备在等待RX下降沿的时间，在设置的时间内没收到RX下降沿，则产生OT_PND Time(ot)= Time(uart_clk) * UTx_OTCNT; 例如：接收uart_clk的频率为10MHz，UTx_OTCNT = 10，那么，OT的时间就为1ms

21.3.15 UTx_ERR_CNT: uart x error byte counter register

Bit	Name	RW	Default	RV	Description
31-0	UTx_ERR_CNT	r	0x0	-	校验错误数量计数，当打开CHECK_EN后，当第一次校验出错时，该寄存器会记录当前byte对应的数量。 【注意】：只记录第一次出错的数量，清除CHK_PND会重新记录。 若清除CHK_PND后，没有出现新的校验错误数据，则该寄存器会一直保持上一次记录的错误数量计数数据。

杰理科技授权潮汐专用

22 WWDG(Window watch dog)

22.1 概述

WWDG(Window WatchDog) 窗口看门狗，是一个用于监视应用程序运行的看门狗，如果应用程序没有在WWDG的递减计数器到达0x3f前重载计数器，那么WWDG会产生一个复位；如果在递减寄存器到达窗口值（WIN）前重载计数器，同样也会产生一个复位。WWDG可以用来监控应用程序运行期间由于不可预见的错误，造成的程序脱离原先的运行顺序的软件故障。

另外的，WWDG可以配置提前唤醒中断（EWI），在计数器为0x40时可以拉起一个中断，可以利用该中断在中断服务程序中保存应用程序复位前的数据。

WWDG接收外部时钟信号，并且选择一个时钟源，经过可配置的预分频器来配置为合适的计数器递减时钟信号。

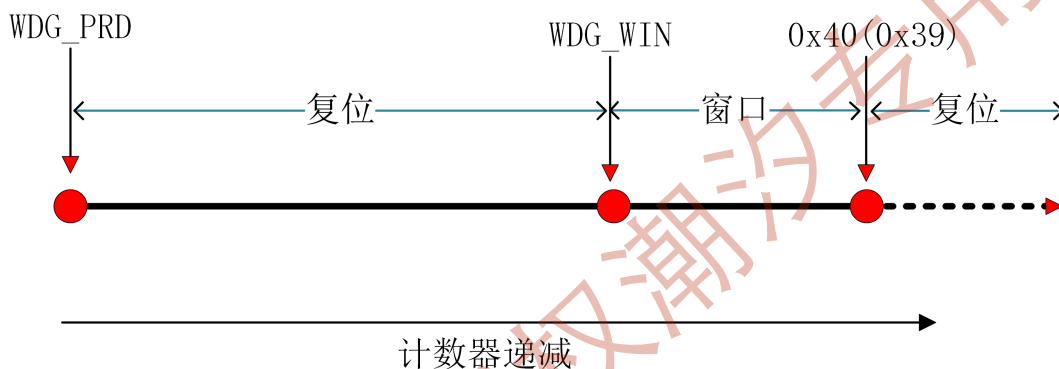


Figure 1 计数器行为图示

22.2 主要特性

- 要配置寄存器的值需要先往JL_WWDG->KEY寄存器中写入规定的KEY码0xa6a0。
- 要重载（reload）计数器的值需要往JL_WWDG->KEY寄存器中写入规定的KEY码0xa6a1。
- 当WWDG使能，并且使能了EWI（提前唤醒中断），当计数值到达0x40，会产生一个中断信号。
- 产生一个复位的条件：
 - 计数器在没到达窗口值WIN之前就被重载。
 - 计数器计数值变为0x3f时。
- 写入CR->WWDG_PRD的值，要大于0x40，小于或等于0x7f。
- 写入CFG->WWDG_WIN的值，大于0x40，小于0x7f与WWDG_PRD，不然无意义。

22.3 框架图



22.4.1 KEY

22.4.2 CR

22.4.3 CFG

Bit	Name	RW	Default	RV	Description
31-16	RESERVED	-	-	-	预留
15-14	WDG_SSEL	rw	—	—	时钟源选择，具体见《Clock_system》
13-11	WDG_PSEL	rw	—	—	预分频数，在4096的基础上分频出 $2^{\text{WDG_PSEL}}$ 的时钟 0: $\text{clk_src} \div 4096 \div 1$ 1: $\text{clk_src} \div 4096 \div 2$ 2: $\text{clk_src} \div 4096 \div 4$ 3: $\text{clk_src} \div 4096 \div 8$ 4: $\text{clk_src} \div 4096 \div 16$ 5: $\text{clk_src} \div 4096 \div 32$ 6: $\text{clk_src} \div 4096 \div 64$ 7: $\text{clk_src} \div 4096 \div 128$
10	RESERVED	-	—	-	预留
9	WDG_EWI	rw	—	—	提前唤醒中断模式使能标志。 1: 使能提前唤醒中断模式 0: 失能提前唤醒中断模式
8-7	RESERVED	—	—	-	预留
6-0	WDG_WIN	rw	—	—	递减计数器窗口值

22.4.4 SR

Bit	Name	RW	Default	RV	Description
31-1	RESERVED	-	-	-	预留
0	WDG_INT	ro	0x0	—	中断标志位，重载KEY寄存器清除该中断

22.5 窗口时间计算

对于用户而言，窗口时间如下。窗口时间下窗口是固定的，为0x40，而上窗口是自己设置的，为WDG_WIN[6:0]，实际窗口时间为WDG_WIN[5:0]。

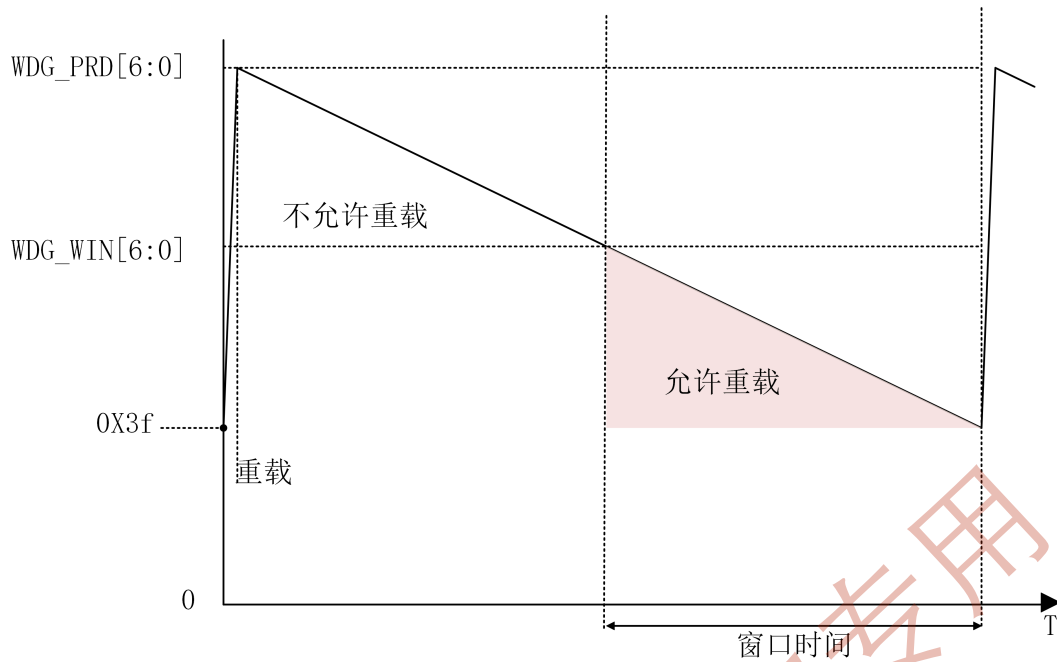


Figure 3 重载窗口时间

记WWDG模块时钟clk为的时钟周期时间为 T_{clk} ，那么窗口时间 $T_{win,max}$ 可以表示为： $T_{win,max} = T_{clk} * 4096 * 2^{WDG.PSEL[2:0]} * (WDG.PRD[5:0] + 1)$

$T_{win,min}$ 可以表示如下：

$$T_{win,min} = T_{clk} * 4096 * 2^{WDG.PSEL[2:0]} * (WDG.PRD[5:0] - WDG.WIN[5:0])$$

即，重载计数器之后，最少要经过 $T_{win,min}$ 时间，在到达 $T_{win,max}$ 时间之内，才可以再次重载计数器。

以 $T_{clk} = 62.5ns$, $F_{clk} = 16MHz$ 的源时钟，WDG_PSEL[2:0]=7, WDG_WIN[6:0]=0x42的配置为例，可以得出，最大的窗口时间 T_{max} 如下： $T_{max} = 62.5 * 4096 * 2^7 * (63+1) = 2097.152 ms$

最小的窗口时间 T_{min} 如下： $T_{min} = 62.5 * 4096 * 2^7 * (63-2) = 1998.848 ms$

即WWDG至少经过1998848000 ns后，在2097152000 ns内，重载WWDG的递减计数器，来定时喂狗，否则会产生一个复位。

22.6 示例代码

```
#define RELOAD_KEY 0xa6a1
#define CONF_KEY 0xa6a0

#define cr_con (\
    /* 31 - 8 ,reserved*/ (0 << 8) |\
    /* 7      , WDG_EN */ (0 << 7) |\
    /* 6 - 0 ,WDG_PRD */ (42<< 0) )

#define cfg_con (\
    /* 31 - 16, reserved*/ (0 << 16) |\
    /* 15 - 14, WDG_SSEL*/ (0 << 0) |\
```

```
/* 13 - 11, WDG_PSEL*/ (0 << 11)|\
/* 10      , reserved*/ (0 << 10)|\
/* 9      , WDG_EWI */ (1 << 9)|\
/* 8 - 7, reserved*/ (0 << 7)|\
/* 6 - 0, WDG_WIN */ (41<< 0))

__attribute__((interrupt("")))
void wwdg_isr(void) {
    //handle
    delay(0xf);

    //clear the intr flag,and reload wwdg
    JL_WWDG->KEY = RELOAD_KEY;
}

void init_intr(void){
    enable_int();
    INTALL_HWI(26,wwdg_isr,0);
}

int main(void){
    init_intr();

    JL_WWDG->KEY = CONF_KEY;
    JL_WWDG->CFG = cfg_con ;

    JL_WWDG->KEY = CONF_KEY;
    JL_WWDG->CR  = cr_con;

    JL_WWDG->KEY = RELOAD_KEY;

    JL_WWDG->KEY = CONF_KEY;
    JL_WWDG->CR |= BIT(7); //enable wwdg

    delay(0x3000);
    SIMEND;

    while(1);
}
```

23 AUDIO LINK

23.1 概述

1. Audio Link接口(以下简称ALNK)是一个通用的双声道音频接口,用于连接片外的DAC或ADC,连接信号有MCLK, SCLK, LRCK, DATA。其中LRCK即为采样率;MCLK是音频接口的主时钟,Delta-Sigma类型的DAC/ADC都是需要此信号的。MCLK可由ALNK提供给DAC/ADC,也可由DAC/ADC提供给ALNK;SCLK为位时钟。
2. Audio Link支持三种基本模式(IIS/左对齐/右对齐),及扩展模式(DSP0短帧模式,DSP1短帧模式,DSP0长帧模式,DSP1长帧模式)共7种模式,三种基本模式原生支持16/24/32bit数据位宽,扩展模式原生支持16/20/24/32bit数据位宽,可对18bit位宽的设备提供兼容支持。
3. ALNK通过DMA的方式与片内系统进行数据连接,不论输入或输出,每条通道占用buffer的大小可由软件配置。
4. ALNK可配置为主机模式或从机模式。

主机模式是指SCLK和LRCK由本模块提供,此时需从外部提供相应采样率参考时钟。有3种方式可供选择:

1. 挂接22.5792MHz晶振以支持44.1KHz组别的采样率。
2. 挂接24.576MHz晶振以支持48/32KHz组别的采样率。
3. MCLK可通过corssbar IO供入。

从机模式是指SCLK和LRCK由外部DAC/ADC提供,此时采样率由外部提供的LRCK确定,因此芯片不需外接晶振来提供采样率参考时钟。

23.2 特殊注意点

ALNK具备4条独立的通道,可相互独立地工作,每条通道都可独立配置为输入或输出,也可配置为不同的连接模式。需注意的是它们共用了MCLK/SCLK/LRCK信号,因此使用上有一定的限制。

1. 4条通道只能同时工作于基本模式或扩展模式。不可一些通道工作于基本模式,另一些通道工作于扩展模式,
2. 位宽配置由四条通道共用。
3. 基本模式下每条通道都只支持双声道立体声。
4. 扩展模式分为短帧模式和长帧模式,短帧模式下,lrck高电平为一个SCLK周期;长帧模式下,LRCK高电平为位宽个SCLK周期;扩展模式通道数可配置,短帧模式通道数可支持1-8声道(slot),当工作于单声道时,只有左声道;而长帧模式通道数可支持2-8声道(slot),不支持1声道。
5. ALNK与SRC_SYNC通信仅支持三种基本模式,扩展模式不支持。

23.3 ALNK采样率配置说明

1. SCLK计算说明: $SCLK = \text{位宽}(\text{bit}) \times \text{声道}(\text{slot}) \times \text{LRCK}(\text{kHz})$ 。
2. MCLK计算说明:
3. 三种基本模式: $MCLK = N \times \text{LRCK}(\text{kHz})$, 常用为64fs, 128fs, 192fs, 256fs, 384fs, 512fs, 768fs。
4. 扩展模式: MCLK无固定要求。
5. 注意: 时钟配置频率精度须达到0.1%, 否则需要通过SRC进行采样率转换得到准确采样率:

23.3.1 ALNK模式说明

1. IIS模式

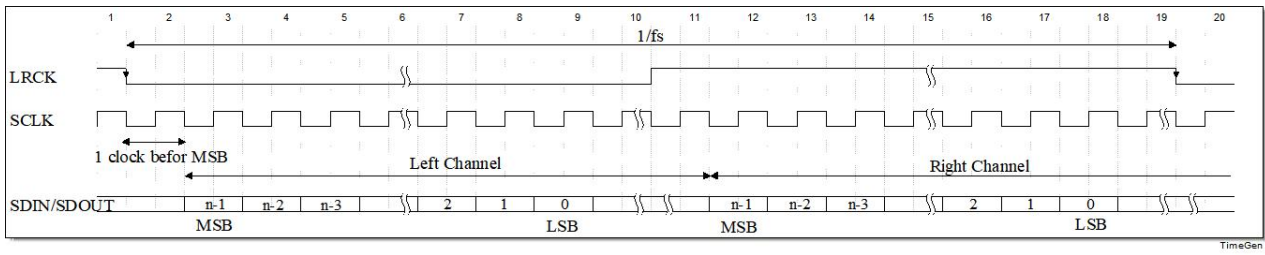


图2 ALNK IIS模式时序图

2. LEFT模式

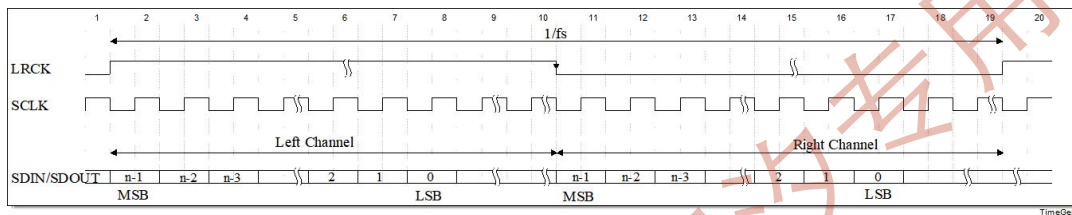


图3 ALNK LEFT模式时序图

3. RIGHT模式

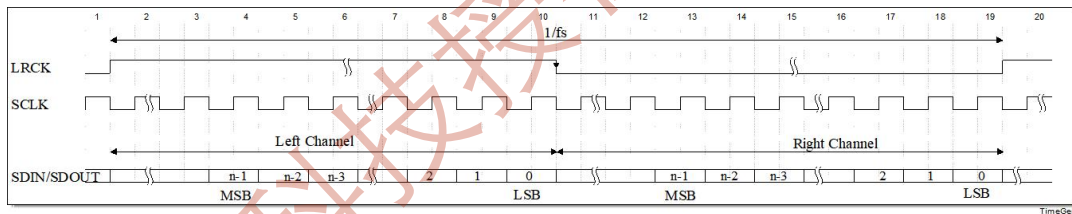


图4 ALNK RIGHT模式时序图

4. DSP0短帧模式

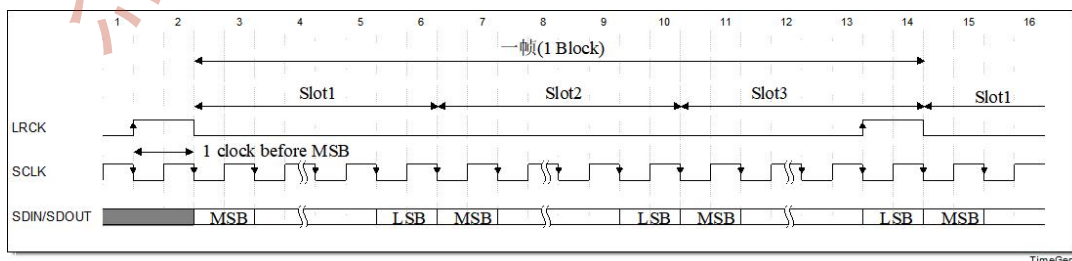


图5 ALNK DSP0短帧模式时序图

5. DSP1短帧模式

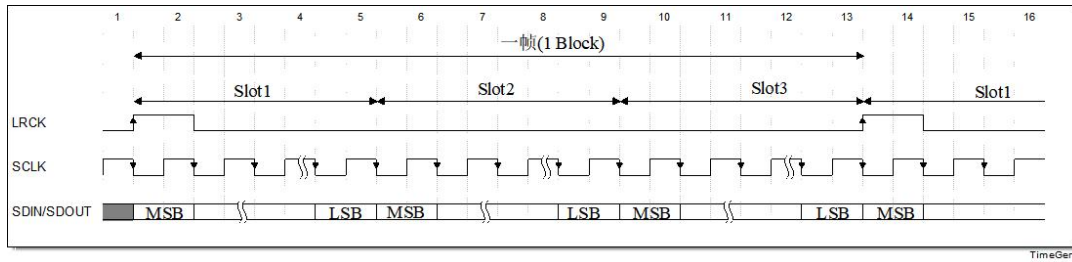


图6 ALNK DSP1短帧模式时序图

6. DSP0长帧模式

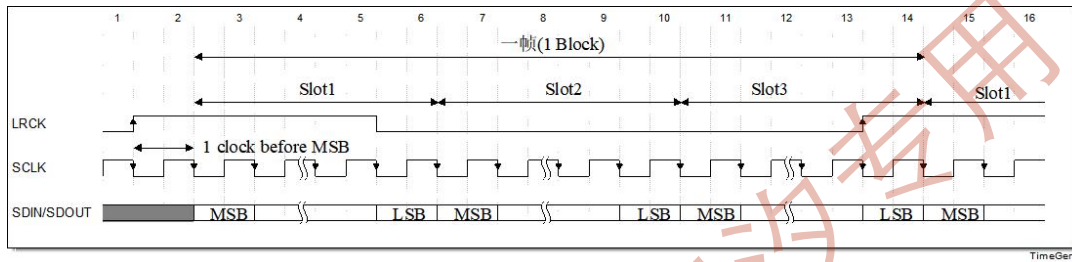


图7 ALNK DSP0长帧模式时序图

7. DSP1长帧模式

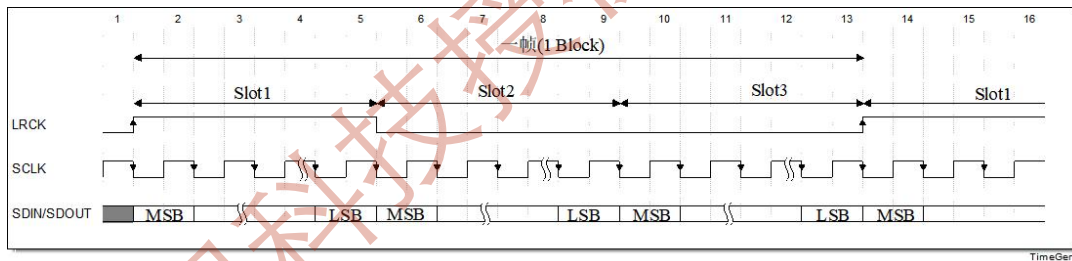


图8 ALNK DSP1长帧模式时序图

23.4 数字模块控制寄存器

23.4.1 ALNK_CON0: control register 0

Bit	Name	RW	Default	RV	Description
15	FLAG3	r	0	x	
14	FLAG2	r	0	x	
13	FLAG1	r	0	x	
12	FLAG0	r	0	x	

Bit	Name	RW	Default	RV	Description
11	ALNKE	rw	0	1	
10	SCKINV	rw	0	0	
9	RESERVED	-	-	-	
8	MOE	rw	0	0	
7	SOE	rw	0	0	
6-5	DSPEN	rw	0x0	x	
4-3	RESERVED	-	-	-	
2	DMA_MODE	rw	0	x	
1-0	DA2BTSRC	rw	0x0	x	

23.4.2 ALNK_CON1: control register 1

Bit	Name	RW	Default	RV	Description
31-14	RESERVED	-	-	-	
13-12	BWLEN	rw	0x0	x	
11	T3DIR	rw	0	x	
10-9	T3MOD	rw	0x0	x	
8	T2DIR	rw	0	x	
7-6	T2MOD	rw	0x0	x	
5	T1DIR	rw	0	x	
4-3	T1MOD	rw	0x0	x	
2	T0DIR	rw	0	x	
1-0	T0MOD	rw	0x0	x	

23.4.3 ALNK_CON2: control register 2

Bit	Name	RW	Default	RV	Description
15	IE3	rw	1	1	

Bit	Name	RW	Default	RV	Description
14	IE2	rw	1	1	
13	IE1	rw	1	1	
12	IE0	rw	1	1	
11-8	RESERVED	-	-	-	
7	PND3	r	0	x	
6	PND2	r	0	x	
5	PND1	r	0	x	
4	PND0	r	0	x	
3	CPND3	w	-	x	
2	CPND2	w	-	x	
1	CPND1	w	-	x	
0	CPND0	w	-	x	

23.4.4 ALNK_CON3: control register 3

Bit	Name	RW	Default	RV	Description
31-30	RESERVED	-	-	-	预留
29-26	SW_MUTE_EN	rw	0x0	-	各通道sw_mute_en [3]: ch3 sw_mute_en [2]: ch2 sw_mute_en [1]: ch1 sw_mute_en [0]: ch0 sw_mute_en
25-23	SLOT	rw	0x0	x	配置范围: 0-7 (配置值分别表示1-8通道) IIS/LEFT/RIGHT: SLOT=2 DSP0/DSP1短帧模式: SLOT=1-8 DSP0/DSP1长帧模式: SLOT=2-8
22-17	LRCK_DUTY	rw	0x0	x	LRCK的高电平脉宽为LRCK_DUTY个SCLK周期 IIS/LEFT/RIGHT或DSP0/DSP1长帧模式: LRCK_DUTY=位宽(BWLEN) DSP0/DSP1短帧模式: LRCK_DUTY=1
16-8	LRCK_PRD	rw	0x0	x	$LRCK = SCLK / (LRCK_PRD + 1)$ LRCK_PRD = 位宽(bit)*声道(slot)-1

Bit	Name	RW	Default	RV	Description
7	SSEL	rw	0	x	0: 从外部输入SCLK和LRCK 1: SCLK为MCKD的分频, 结合SDIV进行配置 LRCK为SCLK的分频, 结合LRCK和LRCK_DUTY进行配置
6-2	SDIV	rw	0x0	x	SCLK前置分频器设置 配置范围: 0-31表示SCLK为MCKD的1-32倍分频。
1-0	MSRC	rw	0x0	0x3	设置MCLK的来源 00: 从外部输入MCLK 01: 系统时钟 10: OSC CLK 11: PLL ALNK CLK

23.4.5 ALNK_CON4: control register 4

Bit	Name	RW	Default	RV	Description
31-8	RESERVED	-	-	-	预留
7-4	HW_MUTE_S	r	0x0	0xx	各通道hw_mute 标志位 [3]: ch3 hw_mute 标志 [2]: ch2 hw_mute 标志 [1]: ch1 hw_mute 标志 [0]: ch0 hw_mute 标志
3-0	SW_MUTE_S	r	0x0	0xx	各通道sw_mute 标志位 [3]: ch3 sw_mute 标志 [2]: ch2 sw_mute 标志 [1]: ch1 sw_mute 标志 [0]: ch0 sw_mute 标志

23.4.6 ALNK_ADRX: alnk dma start address register

ALNK DMA操作起始地址寄存器, 在使用ALNK之前, 必须由软件初始化对齐至4Byte。4个通道的ALNK DMA操作起始地址可以独立设置, 其中, x取0-3。

23.4.7 ALNK_LEN: alnk dma sample length register

通道0-3 DMA数据长度设置寄存器。不同缓存模式下的含义不同。

1. 乒乓缓存模式

ALNK DMA样点长度寄存器, 在使用ALNK之前, 必须由软件初始化为2的倍数, 允许写入 值为2-32768, 超出此范围的设置值可能导致不可预料的错误。

例如, 当ALNK_LEN设置为100时, 则ALNK每条通道每次DMA过程消耗100个样点。此时每通道

DMA buffer占用的空间为:

$$\begin{aligned} 16\text{bit data: } & 100 * 2(\text{CH}) * 2(\text{Byte}) * 2(\text{dual buffer}) = 800 \text{ Byte} \\ 24\text{bit data: } & 50 * 2(\text{CH}) * 4(\text{Byte}) * 2(\text{dual buffer}) = 800 \text{ Byte} \end{aligned}$$

【注意】：2(CH)代表左、右声道。

2. 循环缓存模式

ALNK DMA样点长度寄存器，指定循环Buffer 样点空间大小，在使用ALNK之前，必须由软件初始化，允许写入值为2-32768，超出此范围的设置值可能导致不可预料的错误。

例如，当ALNK_LEN设置为100时，则ALNK每条通道循环Buffer 样点空间为100个样点。此时每通道DMA buffer占用的空间为:

$$\begin{aligned} 16\text{bit data: } & 100 * 2(\text{CH}) * 2(\text{Byte}) = 400 \text{ Byte} \\ 24\text{bit data: } & 50 * 2(\text{CH}) * 4(\text{Byte}) = 400 \text{ Byte} \end{aligned}$$

【注意】：2(CH)代表左、右声道。

23.4.8 ALNK_PNS: alnk ring buff pnding set number

Bit	Name	RW	Default	RV	Description
31-16	OPNS	rw	x	-	当发送循环缓存中剩余数据少于 N 点时，标志位置
15-0	IPNS	rw	x	-	当接收循环缓存中有效数据大于 N 点时，标志位置

23.4.9 ALNK_BLOCK: alnk_block number set

通道0-3通道帧长度设置寄存器，需要根据ALNK_LEN的大小进行配置。计算方式如下所示：
16bit: block = (ALNK_LEN * 2) / slot 20/24/32bit: block = (ALNK_LEN * 1) / slot

23.4.10 ALNK_HWPTRX: alnk hardware rd/wr pointer

4个通道的硬件写循环缓存器位置可以独立设置，其中，x取0-3。

Bit	Name	RW	Default	RV	Description
15-0	HWPTRX	rw	0x0	-	指示硬件当前读写循环缓存器位置，需要初始化，后硬件自动更新

23.4.11 ALNK_SWPTRX: alnk software rd/wr pointer

4个通道的软件写循环缓存器位置可以独立设置，其中，x取0-3。

Bit	Name	RW	Default	RV	Description
-----	------	----	---------	----	-------------

Bit	Name	RW	Default	RV	Description
15-0	SWPTRX	rw	0x0	-	指示软件当前读写循环缓存器位置，需要初始化，写ALNK_SHN时，指针位置硬件自动增加向ALNK_SHN 写入数据

23.4.12 ALNK_SHNX: alnk software handle data number

4个通道的软件处理数据数目寄存器可以独立设置，其中，x取0-3。

Bit	Name	RW	Default	RV	Description
15-0	SHNX	rw	x	-	当通道为接收模式时：写，软件从循环buff中读取了N个数据，ALNK_SWPTR 自动增加相应数值；读，指示循环buff中还有N点数据可以读取。当通道为发送模式时：写，软件往循环buff中写了N个数据，ALNK_SWPTR 自动增加相应数值；读，指示循环buff中还有N点空间可供软件写入。

23.5 SW_MUTE参考代码

```
void sw_mute_func()
{
    SFR(JL_ALNK0->CON2, 12, 1, 0); // close ch0 ie
    SFR(JL_ALNK0->CON3, 26, 1, 01); // sys_clk : sw mute sfr = 1
    while(!(JL_ALNK0->CON4 & BIT(0))); //wait sclk : ch0 sw_mute == 1 -> sw_mute 生效
    #if(dma_mode == 循环buffer) // 初始化DMA指针
    JL_ALNK0->HWPTR0 = 0;
    JL_ALNK0->SWPTR0 = 0;
    JL_ALNK0->SHN0 = ALNKLEN-1;
    #end
    JL_ALNK0->CON2 |= BIT(0); // clr ch0 pnd
    SFR(JL_ALNK0->CON2, 12, 1, 1); //open ch0 ie
    SFR(JL_ALNK0->CON3, 26, 1, 0); //释放sw_mute
}
```

注意：乒乓buffer初始化不要点对DMA相关指针进行操作

23.6 数据通路

23.6.1 乒乓缓存

ALNK的数据都是通过DMA的方式与片内系统连接的，使用了dual-buffer（乒乓缓冲）的方式，每条通道的buf0/buf1容纳样点数由ALNK_LEN寄存器指定，总buffer需求为：

16bit data: $ALNK_LEN * 2(CH) * 2(Byte) * 2(dual\ buffer) = ALNK_LEN * 8\ Byte$

24bit data: $ALNK_LEN / 2 * 2(CH) * 4(Byte) * 2(dual\ buffer) = ALNK_LEN * 8\ Byte$

当四条通道一起使用时，共需要4倍大小的buffer。

特别的：

1. 上述的ALNK_LEN在此处只能理解为16bit数据宽度下的样点数，当数宽度为24bit时，样点数为ALNK_LEN/2；
2. 在单声道状态下，只有左声道会被使用到，右声道将被停用。

16bit数据宽度时，数据组织如图1-1所示。

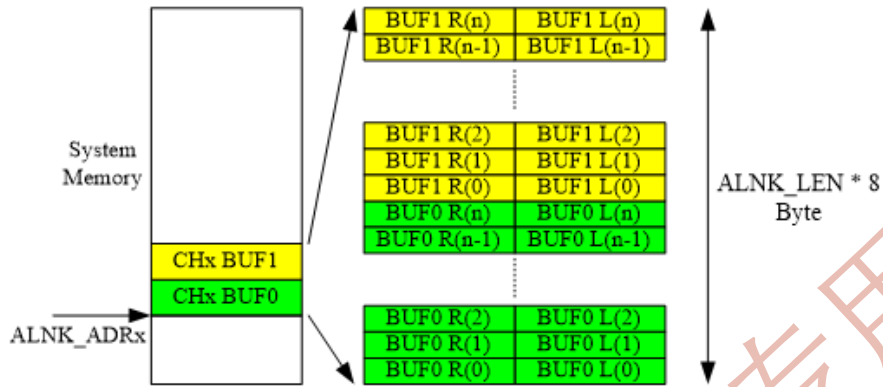


图9 16bit数据宽度乒乓缓冲结构示意图

24bit数据宽度时，数据组织如图1-2所示。

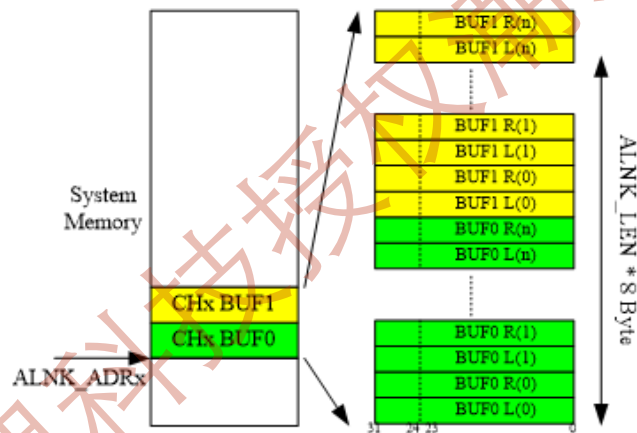


图10 24bit数据宽度乒乓缓冲结构示意图

23.6.2 循环缓存

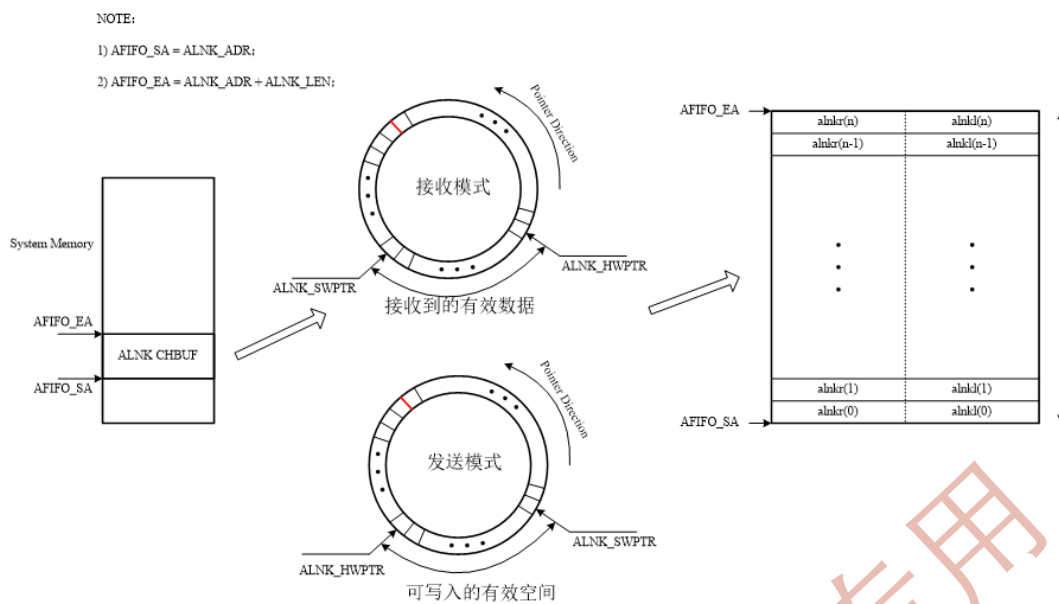


图11 循环缓冲结构示意图